

gold
silver
bronze
raw files

Learning handbook for Mike · Senior Data Engineer

Databricks Certified Data Engineer Professional

Learn the platform the way a senior engineer uses it, then prove it on the exam. Foundations first, then all nine exam domains with plain-language explanations, production advice, hands-on labs, and 166 practice questions with every option explained.

Exam version	**New outline, live from October 9, 2026**
Format	**60 scored questions · 120 minutes**
Free courses mapped	**Databricks Academy · Professional DE pathway**
This edition	**Balanced answer keys · expanded concepts**

Second edition
September 2026

Independent study aid. Not affiliated with or endorsed by Databricks.

Start here

How to use this handbook

Two jobs, one book: become the engineer who understands why things work, and pass the Professional exam on the first attempt.

Two ways to read it

This edition is built for **learning first**. The first edition was mostly an exam summary: dense tables and questions. That works for someone who already knows the material. It does not work well for building real understanding, and understanding is what makes you a senior engineer (and, not by accident, what makes exam scenarios easy).

Mode	When	What to read
Learn mode	First pass through a topic, or when something feels like magic	Part I Foundations, then each domain chapter from the top: the "In plain words" boxes, diagrams, code, and "On the job" advice. Do the lab. Write in the notes pages.
Exam mode	Second pass, last two weeks before the exam	The chapter opener checklist, "Remember" boxes, "Exam trap" boxes, practice questions, the cheat sheets and the mock exam.

A good rhythm per chapter: read it in learn mode (45–90 min), do the lab in Databricks Free Edition (30–60 min), answer the practice questions cold, then read the answer key including the notes on wrong options.

What each box means

In plain words

The idea without jargon. If you can explain this box to a junior colleague, you understand the topic.

On the job

What a senior engineer does with this in production: design choices, failure modes, cost, and healthcare-specific concerns such as PHI.

Exam trap

A pattern the exam uses to make a wrong answer look right. Recognising the trap is often faster than proving the right answer.

Remember

The three to six facts that you should be able to recite the night before the exam.

> **Try it yourself (Free Edition)**
> A short hands-on exercise that works in Databricks Free Edition. Hands-on memory lasts much longer than reading memory.

What was fixed in this edition

The first edition had a serious flaw in its practice questions: **the longest answer was almost always the correct one.** In 139 of 146 questions the correct option was the longest, and the correct letter was D only once. A test-wise reader could score highly without knowing anything, which makes the practice questions useless as a signal of readiness and trains a habit that fails on the real exam, where option length tells you nothing.

What changed:

- **Every question was rewritten** so wrong options are as specific and plausible as the right one. Distractors now contain real Databricks terms, real settings and half-true statements, the way the real exam writes them.
- **Correct letters are balanced** across A, B, C and D, and assigned by a script rather than by hand.
- **Option length was checked by script.** The correct option is now the longest in roughly one question out of four, which is what chance would give. Do not use length as a clue; it no longer works here, and it never worked on the real exam.
- **Every option is explained** in the answer keys, not only the correct one.
- **Four new Foundations chapters** (lakehouse and Unity Catalog, Spark execution, Delta Lake internals, Structured Streaming) with their own self-check questions.
- **Concept sections were expanded** with plain-language explanations, diagrams, production advice and labs, and mapped to the free Databricks Academy courses.

> **How to read a question on the real exam**
> Read the last sentence first: it tells you what is being optimised ("least operational overhead", "lowest cost", "without rewriting data", "exactly once"). Then read the scenario for constraints. Only then read the options, and eliminate options that describe behaviour Databricks does not have. Length, confident wording and the word "always" are not evidence.

Contents

How to use this handbook 1

The exam, the free courses, and your plan 4

Part I · Foundations 8

F1 — The lakehouse and Unity Catalog 9

F2 — How Spark runs your code 17

F3 — Delta Lake under the hood 24

F4 — Structured Streaming from first principles 30

Part II · The nine exam domains 36

23% 1 — Developing Code for Data Processing 37

12% 2 — Data Ingestion and Acquisition 51

12% 3 — Data Manipulation 63

10% 4 — Monitoring and Alerting 73

15% 5 — Cost and Performance Optimization 83

8% 6 — Ensuring Data Security and Compliance 93

5% 7 — Data Governance 103

10% 8 — Debugging and Deploying 111

5% 9 — Data Modeling 120

Part III · Reference and practice 129

Cheat sheets 130

Glossary and renamed products 133

Final mixed mock exam 135

The senior path 143

Extra notes pages 146

Percentages are the official exam weights for the exam version live from October 9, 2026.

Start here

The exam, the free courses, and your plan

What the exam tests, which free Databricks Academy courses teach it, and a calendar that fits the Advanced Learning Festival voucher window.

What you will be able to do

- Know exactly what the new (October 9, 2026) exam covers and how it is weighted
- Know which free Academy course teaches which domain, and what the courses do not give you
- Follow a dated plan: festival pathway, voucher, practice, booking

Matching free course

- All self-paced Academy courses are free with a Customer Academy account
- Hands-on labs inside Academy are a paid subscription: use Databricks Free Edition instead

Study checklist

- ☐ Create or log in to a Customer Academy account
- ☐ Create a Databricks Free Edition workspace
- ☐ Enroll in the four Professional Data Engineering courses
- ☐ Use the same email for WebAssessor as for Academy

Exam facts at a glance

Item	New exam (live from October 9, 2026)
Format	60 scored multiple-choice questions, plus up to 10 unscored pilot items you cannot identify. Four options, one correct answer. No labs.
Duration	120 minutes: just under 2 minutes per question.
Fee	USD 200. The festival voucher gives 50% off.
Language	English only.
Delivery	Online proctored (WebAssessor/Kryterion) or test centre. Webcam, ID, clean desk, no notes or documentation.
Passing score	Not published. Aim for 85% or more on timed practice before booking.
Prerequisites	None enforced. Databricks recommends about one year of hands-on data engineering on the platform.
Validity	2 years; recertify by passing the then-current exam.

The nine domains and their weights

#	Domain	Weight	In one sentence
1	Developing Code for Data Processing (Python and SQL)	23%	Bundles project structure, dependencies, UDFs, Declarative Pipelines, Lakeflow Jobs, serverless, testing, streaming state.

#	Domain	Weight	In one sentence
2	Data Ingestion and Acquisition	12%	File formats incl. Iceberg, Auto Loader, message buses, Lakeflow Connect CDC, OpenSharing, Clean Rooms, Federation.
3	Data Manipulation	12%	Windows, joins, aggregations, VARIANT, AI functions, expectations and quarantine.
4	Monitoring and Alerting	10%	System tables, REST/CLI/SDK, event logs, Lakehouse alerts, Jobs monitoring.
5	Cost and Performance Optimization	15%	Managed tables, Predictive Optimization, liquid clustering, deletion vectors, caching, CDF, query profiles.
6	Ensuring Data Security and Compliance	8%	Least privilege, row filters, column masks, ABAC with governed tags, PII handling, GDPR purge.
7	Data Governance	5%	Tags, comments, discoverability, permission inheritance.
8	Debugging and Deploying	10%	Spark UI, logs, repair runs, Declarative Automation Bundles, Git folders and CI/CD.
9	Data Modeling	5%	Physical layouts for Delta and Iceberg, dimensional models, materialized views, Metric Views.

Domains 1, 5, 2 and 3 are 62% of the exam. If time runs short, they come first.

> **Names changed. The exam uses the new ones.**
>
> Lakeflow Spark Declarative Pipelines = Delta Live Tables (DLT). Declarative Automation Bundles = Databricks Asset Bundles (DABs). AUTO CDC = APPLY CHANGES. OpenSharing = Delta Sharing. Lakeflow Jobs = Workflows. Git folders = Repos. Data quality monitoring = Lakehouse Monitoring. The glossary in Part III has the full list.

The free Databricks Academy courses, mapped to this book

All self-paced courses in Databricks Academy are free with a Customer Academy account. The Professional Data Engineering pathway is the one the exam guide recommends, and it is also the pathway that qualifies for the festival voucher.

Professional Data Engineering pathway (the exam pathway)

Course	What it teaches (official outline)	Read with
Advanced Techniques with Spark Declarative Pipelines (about 2 h)	Multi-flow pipelines that land many sources in one bronze table; liquid clustering and expectations in silver and gold; the Multiplex streaming pattern with Delta sinks and Iceberg reads (UniForm); SCD Type 2 with `AUTO CDC INTO`; zero-data-loss quarantine pipelines.	F2, F4, Ch. 1, 2, 3
Databricks Data Privacy	Regulatory compliance; auditing data; data isolation and securing data in Unity Catalog; pseudonymisation and anonymisation; PII security; capturing changed data; deleting data and propagating deletes with Change Data Feed.	Ch. 6, 7, 5.5
Databricks Performance Optimization	Spark UI; designing the foundation and file explosion; data skipping and liquid clustering; skew, shuffles, spill, exploding joins; serialization and UDFs; choosing cluster and instance types.	F2, F3, Ch. 5, 8, 9

Course	What it teaches (official outline)	Read with
Automated Deployment with Declarative Automation Bundles	DevOps and CI/CD review; deploying with REST API, SDK and CLI; bundle structure; variables and multiple environments; templates; CI/CD with bundles; local development in VS Code; GitHub Actions.	Ch. 1, 8

Refresher and depth courses (also free)

Pathway	Courses	Why a senior engineer should take it
Associate Data Engineering	Data Ingestion with Lakeflow Connect · Deploy Workloads with Lakeflow Jobs · Build Data Pipelines with Lakeflow Spark Declarative Pipelines · DevOps Essentials for Data Engineering	The prerequisites the Professional courses assume. Skim at 1.5x speed if you already build pipelines daily.
Apache Spark Developer	Introduction to Apache Spark · Developing Applications with Apache Spark · Stream Processing and Analysis with Apache Spark · Monitoring and Optimizing Apache Spark Workloads on Databricks	The deepest free material on how Spark executes. This is what separates "writes PySpark" from "tunes PySpark".
Data Warehousing Practitioner	SQL Programming and Procedural Logic · Data Modeling Strategies · Building Semantic Models with UC Metric Views · Architecting Data Warehouses for Large-Scale Deployments	Gold-layer design, dimensional modelling and semantic layers: the part of the job that decides whether analysts trust your data.
Fundamentals	Databricks Fundamentals (about 1 h) and the accreditation	Quick orientation; useful vocabulary for explaining the platform to stakeholders.

> **What the free courses do not give you**
>
> The self-paced courses are lectures (increasingly PDF slides with notes) and demo videos. The interactive lab environment is part of the paid Databricks Academy Labs subscription. So the hands-on part is on you: every chapter here has a lab you can do in **Databricks Free Edition**, which is free and replaced Community Edition.

Other free resources worth bookmarking

- **Databricks documentation** (docs.databricks.com): the source of truth for syntax. When this book and the docs disagree, trust the docs; features change monthly.
- **The official exam guide** for the new outline: includes sample questions aligned to the October 2026 version. Do them in the final week.
- **Databricks Community** (community.databricks.com): the Academy Learners group and the Certifications board, where people report exam experiences.
- **Databricks Free Edition** (databricks.com/learn/free-edition): serverless compute, Unity Catalog, pipelines and jobs, enough for every lab in this book.

Your calendar

Today is late September 2026. The Advanced Learning Festival runs **September 16 – October 14, 2026**. Completing all four courses of one pathway inside that window earns a 50% certification voucher and 20% off Academy Labs, emailed on **October 19, 2026** to your Academy address; the voucher must be used before mid-January 2027. Register at WebAssessor with the same email.

Dates	Goal	Do
Sep 25 – Oct 1	Foundations + pipelines	Course: Advanced Techniques with SDP. Book: F1–F4, Ch. 1, 2. Labs F2, F3, 1, 2.
Oct 2 – Oct 8	Performance + data	Course: Performance Optimization. Book: Ch. 3, 5, 9. Labs 3, 5, 9.
Oct 9 – Oct 14	Privacy + deployment; finish pathway	Courses: Data Privacy, Declarative Automation Bundles. Book: Ch. 6, 7, 8. Confirm all modules show "completed" in My Activities.
Oct 15 – Oct 25	Monitoring + first full practice	Book: Ch. 4. Answer every chapter's questions cold, log scores. Re-read weak chapters.
Oct 26 – Nov 5	Exam readiness	Mock exam timed (40 min). Official sample questions. Cheat sheets. Book the exam once you score 85%+ twice.
Nov – Jan	Exam, then keep growing	Take the exam with the voucher. Then follow the senior growth path in Part III.

Protect the time

Two focused 60–90 minute sessions a day beat one long weekend session. Put them in your calendar like meetings. If work gets busy, cut reading time, not lab time.

My goals

Part I

Foundations

The mental models every exam question quietly assumes. Read these first even if you use Databricks daily: most "tricky" exam questions are easy once you can picture what happens underneath.

F1 · The lakehouse and Unity Catalog

F2 · How Spark runs your code

F3 · Delta Lake under the hood

F4 · Structured Streaming from first principles

Foundations

F1 The lakehouse and Unity Catalog

Where your data physically lives, who can see it, and which engine does the work.

What you will be able to do

- Draw the lakehouse stack from cloud storage up to dashboards, and say what each layer is responsible for
- Explain managed vs external tables, volumes, and why DBFS root is legacy
- Pick the right compute type (serverless, job, all-purpose, SQL warehouse) and access mode for a workload
- Design bronze, silver and gold layers for a real healthcare feed

Matching free course

- Databricks Fundamentals (free, about 1 h)
- Data Management and Governance with Unity Catalog

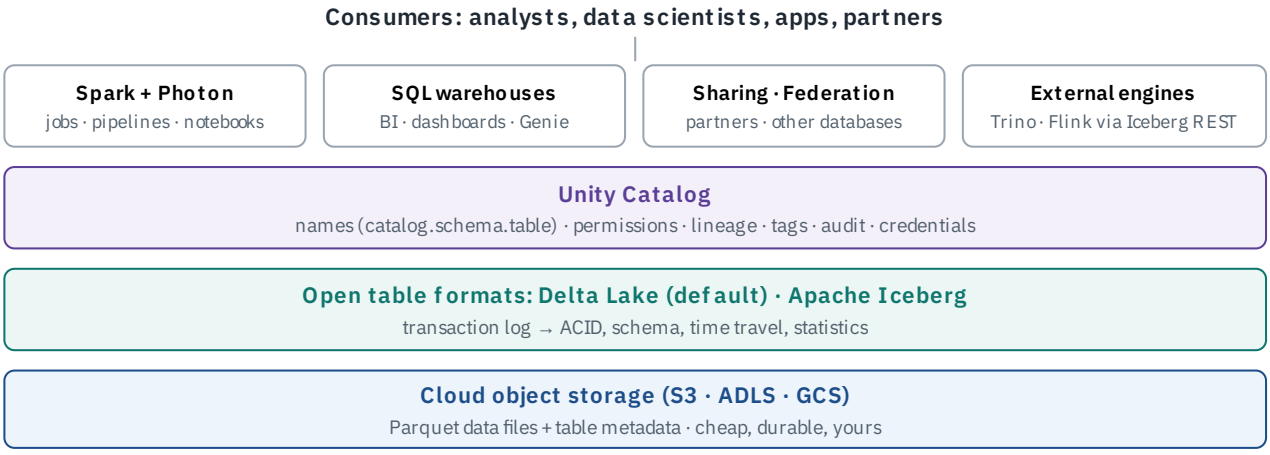
Study checklist

- ☐ Read F1
- ☐ Lab F1: catalog, schema, volume, managed table
- ☐ Self-check questions F1
- ☐ Write 3 notes in your own words

F1.1 The lakehouse in one picture

In plain words

A data warehouse gives you reliable tables, SQL and security, but stores data in its own closed format. A data lake stores anything cheaply as files, but files alone are not tables: there are no transactions, no schema rules and no fine-grained security. A **lakehouse** keeps the cheap open files of a lake and adds a thin layer (a table format plus a catalog) that makes those files behave like warehouse tables.



Each layer has one job. Storage keeps bytes, the table format makes them transactional, Unity Catalog governs them, engines compute.

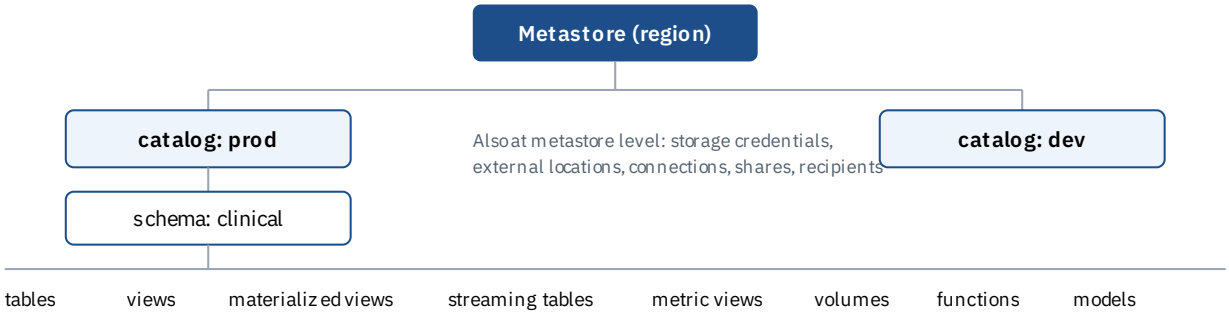
Two consequences follow from this picture, and the exam leans on both:

1. **Storage and compute are separate.** You can shut down every cluster and your tables still exist. You can point several engines at the same table at once. You pay for storage continuously and for compute only while it runs.

2. **Governance lives in one place.** Because every engine goes through Unity Catalog to find and open a table, one `GRANT` or one column mask protects the data for SQL warehouses, notebooks, pipelines and BI tools alike. That is why the "right" exam answer for security is almost always a Unity Catalog feature rather than a filter in a dashboard.

F1.2 Unity Catalog: the object model

Unity Catalog (UC) has one **metastore** per cloud region, attached to many workspaces. Inside it, everything has a three-level name: `catalog.schema.object`.



Object	What it is	Typical use
Managed table	UC decides where the files live and manages their lifecycle. `DROP TABLE` deletes the data (after a short recovery window).	The default. Gets Predictive Optimization, automatic liquid clustering, faster metadata.
External table	You give a `LOCATION` inside an external location. `DROP TABLE` removes only the metadata.	Data that other systems write to a path you must keep, or migrations.
View	A saved query. Runs with the view owner's rights on the base tables.	Present a subset or reshaped data to an audience.
Materialized view / streaming table	Tables maintained by a Declarative Pipeline.	Incremental ingestion (streaming tables) and precomputed results (MVs).
Metric view	A governed semantic definition of dimensions and measures.	One definition of "net revenue" for every dashboard.
Volume	Governed folder of non-tabular files: `/Volumes/catalog/schema/volume/path`.	Landing raw files, libraries, checkpoints, logs. Replaces DBFS paths.
Function	SQL or Python UDF registered in UC with `EXECUTE` permission.	Shared logic, row filters, column masks, ABAC policies.
Connection / foreign catalog	Credentials to an external database and its mirrored catalog.	Lakehouse Federation and Lakeflow Connect.
Storage credential / external location	Cloud identity plus a path it may use.	Governed access to your own buckets.

DBFS root is not a governed place for data

Paths like `dbfs:/FileStore/...` or `/mnt/...` bypass Unity Catalog permissions and are legacy. If an answer stores libraries, checkpoints or raw files "in DBFS root", it is almost certainly a distractor. The modern answer is a Unity Catalog volume.

Identities

Users, groups and service principals are defined at the **account** level and synced into workspaces. Unity Catalog grants are made to account-level principals. Two consequences show up in exam questions: use `is_account_group_member()` (not the older workspace-scoped `is_member()`) inside masks and filters, and run production jobs as a **service principal** so they keep working when a person leaves the company.

F1.3 Compute: which engine runs your code

Compute	Starts	Billing	Best for	Watch out
Serverless (notebooks, jobs, pipelines)	Seconds	Per use, no idle cost	Most new jobs and pipelines; spiky or short workloads	No init scripts or cluster libraries: dependencies go in the task's environment
Job compute (classic)	Minutes	Cheaper DBU rate than all-purpose; exists only for the run	Scheduled batch where you need custom configs, specific instance types or init scripts	Cold start each run
All-purpose (classic)	Minutes	Highest DBU rate; bills while idle	Interactive development	Running scheduled jobs on it is a classic cost mistake
SQL warehouse (serverless or pro)	Seconds (serverless)	Per use	SQL, BI, dashboards, alerts	SQL only

Access modes decide whether classic compute enforces Unity Catalog:

- **Standard** (formerly "shared"): many users, full UC enforcement, Python and SQL. The default for most teams.
- **Dedicated** (formerly "single user"): one user or group; supports everything including ML runtimes.
- **No isolation** (legacy): no UC enforcement at all. Masks, filters and lineage do not work. Appears in exam questions as the reason why "lineage is missing" or "the mask did not apply".

Photon is Databricks' vectorised C++ engine for SQL and DataFrame operations. It speeds up scans, joins and aggregations; it does not speed up Python UDFs or RDD code, because those run outside the engine.

Choosing compute at work

Default to serverless for new jobs and pipelines; it removes cluster sizing, idle cost and most "which runtime?" problems. Reach for classic job compute when you need something serverless does not offer (an init script for an OS library, a specific GPU type, a custom Spark config). Never schedule production jobs on an all-purpose cluster: it is the most expensive rate, and a colleague restarting it breaks your job.

F1.4 The medallion architecture

The medallion architecture is not a feature; it is a convention for **what quality promise each layer makes**.

Layer	Promise	Contents	Typical write pattern
Bronze	"Exactly what arrived, nothing lost."	Raw records plus ingestion metadata (file path, load time, Kafka offset). Often a raw string or VARIANT payload.	Append-only streaming table
Silver	"Clean, conformed, one row per real thing."	Typed, validated, deduplicated, joined to reference data; CDC applied; PII handled.	Streaming table, AUTO CDC target, MERGE
Gold	"Ready to answer a business question."	Aggregates, star schemas, feature tables, metric views.	Materialized views, aggregated tables

Why keep bronze append-only and raw? Because parsing logic has bugs. If bronze holds the original bytes, you can fix the silver logic and replay from bronze. If you parsed and threw away the raw payload, the data is gone.

> **A healthcare example**
>
> An 837 claims feed arrives as files every hour and an EHR database emits changes. Bronze: one append-only streaming table per source with the raw payload, `_metadata.file_path` and load time. Silver: parsed claims with typed amounts and codes, member records maintained with AUTO CDC (SCD Type 2 so address history is kept), PHI columns tagged and masked by policy, invalid rows quarantined, not dropped. Gold: a claims fact and member dimension, materialized views for dashboard grains, and a metric view defining "paid amount" and "denial rate" once.

F1.5 Batch, incremental and streaming are one spectrum

Beginners think of "batch jobs" and "streaming jobs" as different worlds. On Databricks they are the same engine at different settings. A streaming query with `trigger(availableNow=True)` behaves like a batch job: it processes everything new since the last run, then stops. The same code with `trigger(processingTime="1 minute")` runs continuously. The checkpoint remembers progress either way. This is why the exam likes "incremental" answers: they give batch-like cost with streaming-like correctness.

> **Remember**
>
> - Lakehouse = open files + table format (Delta/Iceberg) + Unity Catalog + many engines.
> - Three-level names: `catalog.schema.object`. Managed tables are the default; external when another system owns the path.
> - Volumes replace DBFS paths for files, checkpoints and libraries.
> - Serverless for most work, job compute for special needs, all-purpose only for development.
> - Standard or dedicated access modes enforce UC; no-isolation mode does not.
> - Bronze keeps raw and append-only so you can replay; silver cleans and conforms; gold serves questions.

Lab F1 · Your first governed table (20 min)

1. In Free Edition, run `CREATE CATALOG IF NOT EXISTS learn; CREATE SCHEMA IF NOT EXISTS learn.bronze; CREATE VOLUME learn.bronze.landing;`
2. Upload any small CSV to the volume through Catalog Explorer.
3. `CREATE TABLE learn.bronze.sample AS SELECT *, _metadata.file_path AS src FROM read_files('/Volumes/learn/bronze/landing/', format => 'csv', header => true);`
4. Run `DESCRIBE DETAIL learn.bronze.sample` and `DESCRIBE EXTENDED learn.bronze.sample`. Find the table type (MANAGED), the location UC chose, and the format.
5. Drop the table and run `UNDROP TABLE learn.bronze.sample`. Managed tables can be recovered for a few days.

Self-check · F1 The lakehouse and Unity Catalog

Answer first, check later. Score: _____ / 5 · Date: _____ · Retake date: _____

F1.1 A team drops a **managed** Unity Catalog table by mistake on Tuesday and notices on Wednesday. What is true?

- (A) Only the metadata was removed; the Parquet files stay in the table's path until someone deletes them manually.
- (B) Nothing can be recovered, because a dropped managed table is purged from cloud storage in the same transaction.
- (C) Its files are only scheduled for deletion, so `UNDROP TABLE` can recover it within the retention window.
- (D) The table can be restored with `RESTORE TABLE ... TO VERSION AS OF`, since time travel survives a drop.

F1.2 A scheduled ETL job runs for 15 minutes every two hours on an all-purpose cluster that three analysts also use during the day. Which change is the best first step?

- (A) Enable Photon on the all-purpose cluster so the job finishes sooner and the cluster idles less.
- (B) Pin the analysts to a SQL warehouse and give the ETL job exclusive use of the all-purpose cluster.
- (C) Keep the all-purpose cluster but raise auto-termination so the job never waits for a cold start.
- (D) Move the scheduled job off the shared cluster to serverless job compute or a per-run job cluster.

F1.3 A column mask on `prod.clinical.patients.ssn` works in SQL warehouses, but a data scientist sees clear SSNs from a notebook. The cluster was created years ago. What is the most likely cause?

A. Column masks are evaluated by SQL warehouses only; notebooks need a dynamic view on top of the table.

B. The cluster uses the legacy no-isolation access mode, which does not enforce Unity Catalog governance.

C. The data scientist owns the masking function, and function owners are always exempt from the mask.

D. The mask is cached per session, so the notebook still sees the values from before the mask was set.

F1.4 Where should raw vendor files, streaming checkpoints and wheel files be stored in a Unity Catalog workspace?

A. In workspace files next to the notebooks, because they are versioned together with the code.

B. Inside a managed Delta table as binary columns, so that access control comes from table grants.

C. In the DBFS root under `/FileStore/`, because every cluster can reach it without extra configuration.

D. In Unity Catalog volumes such as `/Volumes/prod/landing/files/`, governed by volume privileges.

F1.5 Why should bronze tables keep the raw payload instead of only the parsed columns?

A. So that bugs in parsing can be fixed by replaying silver from bronze without re-acquiring data.

B. Because Delta Lake requires one STRING or VARIANT column for schema evolution to work on bronze.

C. Because silver tables cannot read parsed columns from a streaming source without the original string.

D. Because Unity Catalog only tracks lineage for tables that keep the original input unchanged.

Self-check · F1 The lakehouse and Unity Catalog — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

F1.1 Answer C

Dropping a managed table removes it from the catalog and schedules the underlying files for deletion after a recovery period, during which `UNDROP TABLE` brings it back.

A That describes an **external** table. For managed tables UC owns the files and deletes them after the recovery window.

B Managed tables have a recovery window (days) during which `UNDROP` works.

D `RESTORE` works on an existing table's history; after a drop you need `UNDROP` first.

F1.2 Answer D

Scheduled production work belongs on job or serverless compute: a cheaper rate, no idle billing, and isolation from interactive users.

A Photon may help runtime, but the rate and idle-cost problem of all-purpose compute remains.

B Still pays the all-purpose rate and idle cost for a scheduled job.

C A longer idle time increases cost at the highest DBU rate and keeps the shared-cluster risk.

F1.3 Answer B

Unity Catalog features (grants, masks, row filters, lineage) are enforced only on UC-enabled compute: standard or dedicated access mode, or serverless.

A Masks are enforced for every UC-enabled engine, not just warehouses.

C Owning the function does not exempt a user; the mask logic decides who sees what.

D There is no such cache; enforcement happens on every query.

F1.4 Answer D

Volumes are the Unity Catalog object for non-tabular files; they carry permissions, lineage and a stable path.

A Workspace files suit small code assets, not data, checkpoints or large artifacts.

B Tables are for tabular data; volumes are the governed home for files.

C DBFS root is ungoverned and legacy; it bypasses UC permissions.

F1.5 Answer A

Bronze is the system's memory. Keeping the original input (plus source metadata) means any downstream mistake is recoverable by reprocessing.

B No such requirement exists; schema evolution works on typed columns.

C Silver can read parsed columns; the reason for raw data is recoverability, not a technical limit.

D Lineage is tracked regardless of what columns a table keeps.

Notes · F1 The lakehouse and Unity Catalog

What clicked, what did not, commands to try, questions to ask a colleague.

Foundations

F2 How Spark runs your code

Driver, executors, stages, shuffles and the optimizer: the model behind every performance question.

What you will be able to do

- Explain what happens between `df.write` and files appearing in storage
- Tell narrow from wide transformations and predict where a shuffle happens
- Read the Spark UI: jobs, stages, tasks, skew, spill
- Explain what Catalyst, AQE and Photon each do, and what they cannot fix

Matching free course

- Introduction to Apache Spark (Spark Developer pathway)
- Databricks Performance Optimization (Professional pathway)

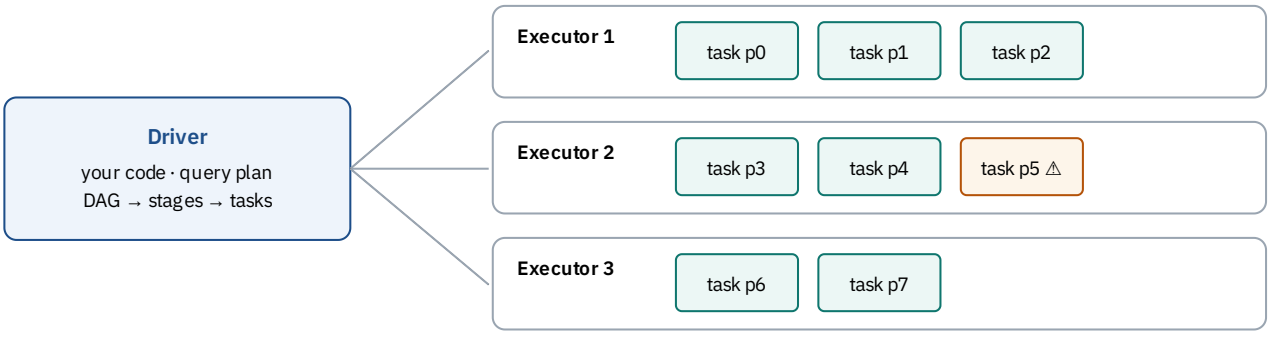
Study checklist

- ☐ Read F2
- ☐ Lab F2: cause and read a shuffle and a skew
- ☐ Self-check questions F2
- ☐ Explain 'stage boundary' to a colleague

F2.1 Driver, executors, tasks

In plain words

Spark splits your data into many pieces (partitions) and sends the same piece of work to many machines at once. One machine, the **driver**, plans the work and hands out tasks. The other machines, **executors**, do the work, one task per partition per CPU core. Your notebook code runs on the driver; the heavy lifting runs on executors.



One task per partition. A stage is finished only when its slowest task is finished, so one oversized partition (p5) delays everything.

Three rules explain most performance behaviour:

1. **Parallelism = number of partitions (up to the number of cores).** 8 partitions on 64 cores leaves 56 cores idle. 100,000 tiny partitions spends more time scheduling than working.
2. **A stage is as slow as its slowest task.** This is why skew hurts so much.

3. **Only the driver sees Python objects.** `collect()`, `toPandas()` and building a pandas DataFrame pull data onto the driver, which is the most common cause of driver out-of-memory errors.

F2.2 Lazy evaluation and the plan

Transformations (`filter`, `select`, `join`, `groupBy`) only build a plan. Nothing runs until an **action** (`write`, `count`, `collect`, `display`, `saveAsTable`). When the action arrives, the **Catalyst optimizer** rewrites the plan: it pushes filters down to the scan, prunes unused columns, reorders joins, and picks join strategies. That is why a filter written after a join can still skip files: Catalyst moves it.

```
df = (spark.read.table("prod.silver.claims")      # nothing read yet
        .join(spark.read.table("prod.silver.members"), "member_id")
        .filter("service_date >= '2026-01-01'")      # Catalyst pushes this into the claims scan
        .groupBy("plan_code").agg(F.sum("paid_amount").alias("paid")))
df.explain(mode="formatted")   # read the physical plan: PushedFilters, BroadcastHashJoin, Exchange
df.write.mode("overwrite").saveAsTable("prod.gold.paid_by_plan")   # action: now it runs
```

> **Python code is a black box to Catalyst**
>
> Catalyst can optimise built-in functions and SQL expressions. It cannot look inside a Python UDF. A UDF blocks filter pushdown through it, prevents Photon from running that step, and pays serialization cost per row. "Replace the UDF with a built-in function" is right far more often than "add more nodes".

F2.3 Narrow vs wide: where shuffles happen

A **narrow** transformation works on each partition independently: `select`, `filter`, `withColumn`, `union`, map-side work. A **wide** transformation needs rows with the same key to meet on the same machine: `groupBy`, `join` (non-broadcast), `distinct`, `orderBy`, window functions with `PARTITION BY`, `repartition`. To do that, Spark writes data to local disk and sends it across the network: a **shuffle**. Every shuffle creates a **stage boundary**.

Symptom in the Spark UI	Meaning	Typical fix
Large "Shuffle Write/Read" on a join stage	Both sides were shuffled (sort-merge join)	Broadcast the small side; filter/aggregate before the join
One task runs 20× longer than the median	**Skew**: one key holds a huge share of rows	AQE skew join, salting, broadcast, isolate the hot key
"Spill (disk)" greater than zero	Partition did not fit in execution memory	More, smaller partitions; fix skew; bigger memory instances
Thousands of tasks reading ~1 MB each	Small files / over-partitioning	Compaction (OPTIMIZE), let AQE coalesce
Only a few tasks, most cores idle	Under-partitioned data or a window without `PARTITION BY`	Repartition by a key; add partition keys to windows

`spark.sql.shuffle.partitions` (default 200) sets how many partitions a shuffle produces. With **Adaptive Query Execution (AQE)**, on by default, Spark measures real shuffle sizes at runtime and then coalesces small partitions, splits skewed ones, and can switch a sort-merge join to a broadcast join when one side turns out small.

F2.4 Join strategies

Strategy	Chosen when	Cost
Broadcast hash join	One side is small (below `spark.sql.autoBroadcastJoinThreshold`, default 10 MB, or hinted)	No shuffle of the big side; small side copied to every executor
Sort-merge join	Two large sides with an equi-join key	Shuffle and sort both sides
Shuffle hash join	Medium side, equi-join	Shuffle both, hash one per partition
Broadcast nested loop	Non-equi or `OR` conditions	Very expensive: compares every pair

F2.5 Memory, caching, Photon

Executor memory is split between **execution** (shuffles, joins, sorts) and **storage** (cached data). When execution memory is exhausted, Spark spills to disk: slower, but it does not crash. Crashes (OOM) usually happen on the driver (`collect`, big pandas objects) or with extreme skew.

Photon is the native vectorised engine: it runs scans, filters, joins, aggregations and Delta writes much faster. It does not accelerate Python UDFs, pandas code or RDDs. On the exam, "enable Photon" is a distractor whenever the bottleneck is Python.

> **How a senior engineer tunes**
>
> Measure first: open the Spark UI or Query Profile and find the one stage that dominates. Then ask in order: is it reading too much (fix layout and skipping), shuffling too much (broadcast, pre-aggregate, filter earlier), skewed (AQE, salting), spilling (partition sizing), or running Python (native functions, Pandas UDF)? Scaling up the cluster is the last step, not the first.

> **Remember**
>
> - Partitions are the unit of parallelism; a stage waits for its slowest task.
> - Transformations are lazy; actions run the plan; Catalyst optimises only what it can see.
> - Wide transformations shuffle and create stage boundaries.
> - Skew = one long task; spill = memory pressure; many tiny tasks = small files.
> - AQE coalesces partitions, splits skew and converts joins to broadcast at runtime.
> - Photon speeds up SQL/DataFrame work, never Python UDFs.

Lab F2 · See a shuffle and a skew (30 min)

1. `big = spark.range(0, 50_000_000).withColumn("k", F.when(F.col("id") % 100 < 60, 0).otherwise(F.col("id") % 1000))` : 60% of rows get key 0.
2. `small = spark.range(0, 1000).withColumnRenamed("id", "k")`
3. Run `big.join(small.hint("merge"), "k").count()` and open the job in the Spark UI (classic compute) or the query profile (serverless). Find the join stage and compare max vs median task duration.
4. Run `big.join(F.broadcast(small), "k").count()` and compare shuffle bytes.
5. Write one sentence in your notes on why the broadcast version has no skew problem.

Self-check · F2 How Spark runs your code

Answer first, check later. Score: _____ / 5 · Date: _____ · Retake date: _____

F2.1 A notebook builds a 40 GB pandas DataFrame with `df.toPandas()` and fails with an out-of-memory error. Where did it run out of memory?

- A. On the driver, because `toPandas()` collects every row from all executors into one Python process.
- B. On the executors, because pandas conversions happen inside each task before the results are merged.
- C. In the shuffle service, because converting to pandas triggers a full repartition of the data by index.
- D. In the Photon engine, which cannot allocate vectors for pandas types larger than executor memory.

F2.2 Which operation does **not** cause a shuffle?

- A. `df.groupBy("member_id").agg(F.sum("paid"))`
- B. `df.filter("amount > 0").withColumn("y", F.year("service_date"))`
- C. `df.orderBy("service_date")` before writing the result to a table
- D. `claims.join(members, "member_id")` where both tables are several terabytes

F2.3 In the Spark UI, a join stage has 200 tasks. The median takes 30 seconds, one takes 38 minutes and shows large disk spill. What is the diagnosis?

- A. Photon is disabled, so the join falls back to a slower row-by-row path.
- B. Too many small input files, which makes the scheduler wait between tasks.
- C. Data skew: a single join key holds a very large share of the stage's rows.
- D. The driver is garbage-collecting, which pauses all tasks in the stage evenly.

F2.4 A Python UDF normalises a code column in a 3-billion-row job and is the slowest step. Which change is most likely to help most?

- (A) Cache the input DataFrame in memory before the UDF so the data is read only once.
- (B) Enable Photon so the UDF is compiled into native vectorised code at runtime.
- (C) Increase `spark.sql.shuffle.partitions` so each UDF call processes fewer rows per task.
- (D) Rewrite it with built-in functions such as `regexp_replace` and `upper`.

F2.5 What does Adaptive Query Execution (AQE) do?

- (A) Caches frequently used tables on local SSDs so that later queries in the session skip cloud reads.
- (B) Chooses the number of executors for each stage based on the size of the input tables in the catalog.
- (C) Re-plans at runtime using real shuffle statistics: coalescing, splitting skew, switching join types.
- (D) Rewrites Python UDFs into SQL expressions when their logic can be expressed with built-ins.

Self-check · F2 How Spark runs your code — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

F2.1 Answer A

Collect-style actions (`collect`, `toPandas`, driver-side pandas) move data to the driver. Keep data distributed or use high-memory driver settings as a stop-gap.

B Executors produce Arrow batches, but the pandas object is assembled on the driver.

C `toPandas()` does not shuffle; it collects to the driver.

D Photon is not involved in building pandas objects.

F2.2 Answer B

Filter and column expressions are narrow: each partition is processed on its own, with no data exchange.

A Grouping needs all rows of a key together: a shuffle.

C A global sort is a wide operation.

D A large-large equi-join is a sort-merge join with shuffles.

F2.3 Answer C

One outlier task with spill is the signature of skew. Fix with AQE skew handling, salting, broadcasting the other side, or handling the hot key separately.

A Photon affects all tasks similarly; it does not create a single outlier.

B Small files cause many short tasks, not one very long one.

D Driver GC would slow all tasks, not one.

F2.4 Answer D

Built-in functions run inside the JVM/Photon with no serialization and are visible to Catalyst. If Python is unavoidable, a Pandas UDF is the next best option.

A Caching avoids re-reading input; it does not speed up the UDF itself.

B Photon cannot execute Python UDFs; they stay in Python workers.

C The cost is per-row Python execution, not partition count.

F2.5 Answer C

AQE uses statistics from completed shuffle stages to adjust the rest of the plan: fewer tiny partitions, split skewed partitions, broadcast joins when a side is small.

A That describes the disk (Delta) cache.

B Executor count is autoscaling, not AQE.

D Spark does not translate UDF bodies.

Notes · F2 How Spark runs your code

What clicked, what did not, commands to try, questions to ask a colleague.

Foundations

F3 Delta Lake under the hood

The transaction log explains time travel, concurrency conflicts, VACUUM, deletion vectors and data skipping.

What you will be able to do

- Explain how a folder of Parquet files becomes an ACID table
- Predict what `VACUUM`, `OPTIMIZE`, `RESTORE` and time travel do to files and the log
- Explain optimistic concurrency and why two writers conflict
- Explain data skipping from file statistics, and what deletion vectors change

Matching free course

- Databricks Performance Optimization
- Data Ingestion with Lakeflow Connect (Delta basics)

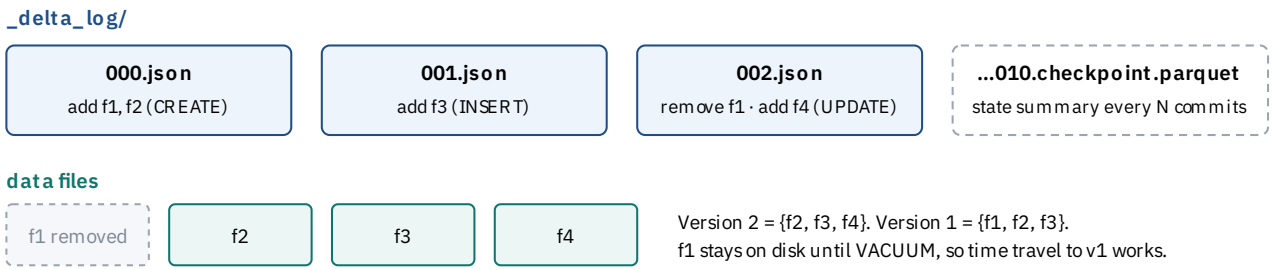
Study checklist

- ☐ Read F3
- ☐ Lab F3: read the _delta_log yourself
- ☐ Self-check questions F3

F3.1 A table is files plus a log

In plain words

A Delta table is a folder of ordinary Parquet files plus a folder called `_delta_log`. The log is a numbered diary: each commit is a JSON file that says "add these files, remove those files". To read the table, an engine reads the diary and uses only the files that are currently added. The Parquet files themselves are never edited: changes are written as new files, and old ones are marked removed.



Everything else follows from this design:

Feature	How the log makes it work
Atomic commits	A commit exists only when its JSON file is written. Readers never see half a write.
Time travel	Replay the log up to version N: `SELECT * FROM t VERSION AS OF 12` or `TIMESTAMP AS OF`. Works only while the old files still exist.
RESTORE	Writes a new commit that re-adds the files of an old version. History is kept.
VACUUM	Physically deletes files that are no longer referenced **and** older than the retention period (default 7 days). After it, time travel to older versions fails.

Feature	How the log makes it work
Schema enforcement	The schema is in the log; writes with wrong columns or types are rejected unless you allow evolution (`mergeSchema` , or `ALTER TABLE`).
Statistics	Each "add" records per-file min/max/null counts for the first 32 columns (or clustering columns). Readers skip files whose range cannot match a filter.
Change Data Feed	When enabled, commits also write row-level change files so consumers can read inserts, updates and deletes incrementally.

F3.2 Concurrency: optimistic, per table

Two writers both read version 5 and try to commit version 6. Delta lets the first one win. The second checks: did the winner touch files I read or wrote? If not (for example, two appends), it retries as version 7 automatically. If yes, it fails with a conflict exception such as `ConcurrentAppendException` or `ConcurrentDeleteReadException` .

Ways to reduce conflicts: make writers touch disjoint data (partition or cluster so a MERGE reads only its slice), use **deletion vectors** with liquid clustering or managed tables (row-level concurrency), and add retries. There are **no transactions across tables**: if a job writes table A and then fails before writing table B, A's commit stays.

F3.3 Deletion vectors: merge-on-read

Without deletion vectors, deleting one row rewrites the whole Parquet file that contains it (copy-on-write). With **deletion vectors (DVs)**, Delta writes a tiny file that marks rows as deleted; readers skip those rows. Deletes, updates and merges become much faster. The cost: the deleted bytes are still physically present until files are rewritten (`OPTIMIZE` , or `REORG TABLE t APPLY (PURGE)`) and the old files are vacuumed. This matters for GDPR erasure (Ch. 6).

F3.4 File layout and skipping

Data skipping only works if values are **co-located**: if every file contains every `member_id` , every file's min/max range matches and nothing is skipped. Layout techniques put similar values in the same files:

Technique	Idea	Status
Partitioning	One folder per value of a low-cardinality column	Only for large tables with a natural coarse key (e.g. month)
Z-order	Rewrite files so several columns are co-located	Legacy; full rewrite each time
Liquid clustering	`CLUSTER BY (cols)` or `CLUSTER BY AUTO` ; incremental, keys changeable	Recommended default

`OPTIMIZE` compacts small files (and clusters them). **Predictive Optimization** runs `OPTIMIZE` , `VACUUM` and `ANALYZE` automatically for managed tables.

Common log myths in distractors

"VACUUM restores old versions" (it deletes them). "RESTORE deletes history" (it adds a commit). "Deletion vectors physically remove rows" (they mask them). "A failed multi-table job rolls back all tables" (each table commits alone). "Time travel works forever" (only until VACUUM and log retention remove what it needs).

In production

Set retention deliberately per table: long enough for your slowest stream and for incident recovery (7 days is a sane default), short where regulation demands fast physical erasure. Use `DESCRIBE HISTORY` as your first debugging tool when numbers change unexpectedly: it tells you who wrote, when, with which operation and how many rows.

Remember

- Delta = Parquet files + `_delta_log` of JSON commits (with periodic checkpoints).
- Time travel reads old versions while their files exist; VACUUM deletes unreferenced files older than retention (7 days default).
- Optimistic concurrency per table; conflicts when writers touch the same files; no cross-table transactions.
- Deletion vectors make deletes cheap but not physical until rewrite + VACUUM.
- Skipping needs co-location: liquid clustering is the modern default.

Lab F3 · Read the diary (25 min)

1. `CREATE TABLE learn.bronze.t (id INT, v STRING); INSERT INTO learn.bronze.t VALUES (1,'a'),(2,'b'); UPDATE learn.bronze.t SET v='z' WHERE id=1; DELETE FROM learn.bronze.t WHERE id=2;`
2. `DESCRIBE HISTORY learn.bronze.t` : note each version's operation and `operationMetrics` .
3. `SELECT * FROM learn.bronze.t VERSION AS OF 1` and then `RESTORE TABLE learn.bronze.t TO VERSION AS OF 1` . Check history again: RESTORE is a new version.
4. `SHOW TBLPROPERTIES learn.bronze.t` : is `delta.enableDeletionVectors` on?

Self-check · F3 Delta Lake under the hood

Answer first, check later. Score: _____ / 5 · Date: _____ · Retake date: _____

F3.1 What does `RESTORE TABLE t TO VERSION AS OF 12` do to the transaction log?

(A) It copies every data file of version 12 into new files and marks the old files as removed.

(B) It runs VACUUM on files newer than version 12 and then reopens the table at version 12.

(C) It adds a new commit that re-references version 12's files, so contents equal version 12.

(D) It deletes all commits after version 12 so that version 12 becomes the latest version again.

F3.2 After `VACUUM t RETAIN 168 HOURS`, which query can fail?

A. `DESCRIBE HISTORY`, because history entries older than the retention period are deleted by VACUUM.
B. Reading the current version, because VACUUM temporarily locks the latest files while it scans.
C. Appending new rows, because the table stays read-only until the next OPTIMIZE after a VACUUM.
D. Time travel to an older version whose data files had been unreferenced for more than 7 days.

F3.3 Two jobs append to the same Delta table at the same time. What happens?

A. Both commits are merged into a single version with the combined files of the two jobs.
B. The second job fails with `ConcurrentAppendException` and must be restarted by the scheduler.
C. Both succeed: appends do not conflict, so the second commit retries automatically.
D. Delta queues the second writer until the first finishes, so they run one after the other.

F3.4 A table has deletion vectors enabled. A `DELETE` removes 10 rows out of a 1 GB file. What is written?

A. A tombstone row for each deleted record, appended to the same Parquet file in place.
B. A small deletion-vector file marking those rows; the Parquet file is unchanged.
C. Nothing until the next OPTIMIZE, which applies all pending deletes in a single rewrite.
D. A new 1 GB Parquet file without the 10 rows, and the old file is marked as removed.

F3.5 A query filters `WHERE member_id = 'M123'` on a table clustered by `service_date`. It reads almost every file. Why?

A. `member_id` values are spread across all files, so no file's min/max range can exclude it.
B. Data skipping is disabled on clustered tables because clustering replaces file statistics.
C. String columns never get statistics, so any filter on an ID stored as a string reads all files.
D. Delta collects statistics only for partition columns, and this table has no partition columns.

Self-check · F3 Delta Lake under the hood — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

F3.1 Answer C

RESTORE is itself a version: the log gains a commit that removes current files and re-adds the files of version 12. You can even restore back.

A RESTORE re-references existing files; it does not copy data.

B RESTORE does not vacuum anything.

D History is never truncated by RESTORE; newer versions stay visible.

F3.2 Answer D

VACUUM deletes data files that are unreferenced and older than the retention threshold. Versions needing those files can no longer be read.

A History is governed by log retention, not VACUUM.

B VACUUM never removes files referenced by the current version.

C VACUUM does not block writers.

F3.3 Answer C

Optimistic concurrency checks whether the winning commit touched data the loser read or rewrote. Pure appends read nothing, so they never conflict.

A Each writer gets its own version number.

B Blind appends do not conflict with each other.

D Delta uses optimistic concurrency, not a lock queue.

F3.4 Answer B

Deletion vectors are merge-on-read: the delete is fast, readers filter marked rows, and physical removal waits for a rewrite plus VACUUM.

A Parquet files are immutable; Delta never edits them.

C The delete commits immediately; readers already skip those rows.

D That is copy-on-write, the behaviour without deletion vectors.

F3.5 Answer A

Skipping depends on co-location. Cluster by `member_id` as well (or instead) so files hold narrow ranges of that column.

B Clustering exists precisely to make statistics effective.

C String columns do get (truncated) min/max statistics.

D Delta collects min/max stats for the first 32 columns (or clustering keys).

Notes · F3 Delta Lake under the hood

What clicked, what did not, commands to try, questions to ask a colleague.

Foundations

F4 Structured Streaming from first principles

A stream is a table that keeps growing. Checkpoints, triggers, watermarks and state make that idea reliable.

What you will be able to do

- Explain the micro-batch model and what the checkpoint stores
- Choose a trigger (availableNow, processingTime, continuous pipelines) for a latency and cost target
- Bound state with watermarks and choose an output mode
- Make `foreachBatch` writes idempotent

Matching free course

- Stream Processing and Analysis with Apache Spark (Spark Developer pathway)
- Advanced Techniques with Spark Declarative Pipelines

Study checklist

- [ ] Read F4
- [ ] Lab F4: restart a stream from its checkpoint
- [ ] Self-check questions F4

F4.1 The mental model: an unbounded table

In plain words

Imagine the input as a table where new rows keep arriving at the bottom. Your streaming query is an ordinary query over that table. Every so often (a **micro-batch**), Spark looks at only the new rows, updates the result, writes it out, and writes down how far it got. That note is the **checkpoint**. If the job crashes, it reads the note and continues exactly where it stopped.

```
q = (spark.readStream.table("prod.bronze.hl7_events")          # source: new rows only
       .withWatermark("event_ts", "30 minutes")                 # how late data may be
       .groupBy(F.window("event_ts", "10 minutes"), "facility_id")
       .count()
       .writeStream
       .outputMode("append")                                    # emit only finished windows
       .option("checkpointLocation", "/Volumes/prod/chk/adt_10m")
       .trigger(availableNow=True)                               # process backlog, then stop
       .toTable("prod.gold.adt_counts_10m"))
```

F4.2 The checkpoint

The checkpoint folder holds: the source offsets planned for each batch, which batches committed, and the **state** of stateful operators (aggregations, deduplication, stream joins), stored in RocksDB on Databricks. With a **replayable source** (files, Delta, Kafka) and an **idempotent sink** (Delta), this gives **exactly-once** results.

Rules the exam tests: one checkpoint per query, never shared between queries. Changing the trigger interval or rate limits (`maxFilesPerTrigger`, `maxBytesPerTrigger`) is safe with the same checkpoint. Changing the

source, the sink, or the shape of a stateful operation needs a new checkpoint. Deleting a checkpoint means reprocessing from the beginning (or from a chosen starting point) and losing state.

F4.3 Triggers

Trigger	Behaviour	Use for
default (none)	Next batch starts as soon as the previous finishes	Lowest latency on classic compute
`processingTime="1 minute"`	A batch at a fixed interval	Steady near-real-time, bounded cost
`availableNow=True`	Process everything available (in several batches), then stop	Scheduled incremental jobs: batch cost, streaming correctness
Continuous pipeline / real-time mode	Always-on, very low latency	SLAs in seconds

F4.4 Event time, lateness and watermarks

Events carry their own timestamp (**event time**) that differs from when they arrive. A watermark says: "I will wait this long for late data." Spark tracks the maximum event time seen, subtracts the delay, and treats anything older as final: windows older than the watermark are emitted (in append mode) and their state is dropped. Records arriving later than the watermark are ignored for those windows.

Without a watermark, state for aggregations and `dropDuplicates` grows forever. With `dropDuplicatesWithinWatermark(["event_id"])` you deduplicate within the lateness window while keeping state bounded.

F4.5 Output modes

Mode	Emits	Notes
append	Only rows that will never change	Default for Delta sinks and streaming tables; aggregations need a watermark
update	Rows changed in this batch	Needs an upsert sink, usually `foreachBatch` + `MERGE`
complete	The whole result every batch	Only for aggregations; state never shrinks

F4.6 foreachBatch and idempotency

`foreachBatch` gives you each micro-batch as a normal DataFrame, so you can `MERGE`, write to two tables, or call JDBC. If a batch fails halfway and is retried, your function runs again: that is at-least-once unless you make it idempotent. Use `MERGE` (reapplying gives the same result) or Delta's `txnAppId` + `txnVersion` options so a replayed batch id is skipped.

```
def upsert(batch_df, batch_id):
    latest = (batch_df.withColumn("rn", F.row_number().over(
                Window.partitionBy("patient_id").orderBy(F.col("updated_at").desc())))
              .filter("rn = 1").drop("rn"))                    # MERGE needs one source row per key
    (DeltaTable.forName(spark, "prod.silver.patients").alias("t")
       .merge(latest.alias("s"), "t.patient_id = s.patient_id")
       .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())

stream.writeStream.foreachBatch(upsert).option("checkpointLocation", chk).start()
```

F4.7 Streaming from Delta tables

A Delta table used as a streaming source must be **append-only** by default. If someone runs `UPDATE` or `DELETE` on it, the stream fails. Options: `skipChangeCommits` (ignore commits that changed existing rows), a full refresh of the downstream table, or read the **Change Data Feed** (`readChangeFeed`) to process the changes deliberately.

Streaming distractors

"Use complete mode so no late data is lost" (it keeps all state and does not handle lateness). "Increase shuffle partitions to bound state" (partitions change parallelism, not retention). "A new trigger interval needs a new checkpoint" (it does not). "skipChangeCommits applies the updates" (it skips them).

In production

Treat checkpoint paths as part of the table's identity: put them in a volume with a naming convention (`/Volumes/<catalog>/chk/<target_table>/<version>`), never reuse, and version them when the query changes shape. Prefer Declarative Pipelines for standard ingestion: they manage checkpoints for you. Keep hand-written streams for custom sinks and complex `foreachBatch` logic.

Remember

- Micro-batches over an unbounded table; checkpoint = offsets + commits + state.
- Replayable source + idempotent sink + checkpoint = exactly-once.
- `availableNow` = incremental batch; trigger changes are safe; source/sink/state changes are not.
- Watermark bounds state and finalises windows; append mode needs it for aggregations.
- `foreachBatch` is at-least-once unless the write is idempotent.
- Delta streaming sources must be append-only unless you skip changes or use CDF.

Lab F4 · Crash and resume (30 min)

1. Create `learn.bronze.events (id INT, ts TIMESTAMP)` and insert 5 rows.
2. Stream it with `trigger(availableNow=True)` into `learn.silver.events_copy` with a checkpoint in a volume. Count rows.
3. Insert 3 more rows and run the same cell again: only the new 3 are processed.
4. Run `UPDATE learn.bronze.events SET id = id + 100 WHERE id = 1` and rerun: read the error. Then add `.option("skipChangeCommits", "true")` and rerun.

Self-check · F4 Structured Streaming

Answer first, check later. Score: _____ / 5 · Date: _____ · Retake date: _____

F4.1 A stream is changed from `processingTime='1 minute'` to `processingTime='5 minutes'`. What should the engineer do with the checkpoint?

A. Copy it to a new path, because each trigger interval writes offsets in its own format.
B. Keep it; trigger settings are not part of the checkpoint.
C. Keep it, but run one batch with `availableNow` first so the offsets are rewritten safely.
D. Delete it, then set `startingVersion` to the latest table version to avoid reprocessing.

F4.2 A streaming aggregation counts events per 10-minute window. State grows every day and never shrinks. What is missing?

A. Complete output mode, so that old windows are written out and then cleared from memory.
B. A `foreachBatch` sink that deletes old windows from the target after each micro-batch.
C. A watermark on the event-time column, so finished windows can be emitted and evicted.
D. A larger value for `spark.sql.shuffle.partitions` so state is spread over more tasks.

F4.3 A job must process all new files every hour and then release its compute. Which trigger fits?

A. No trigger, with `maxFilesPerTrigger` set low enough that each run finishes within an hour.
B. `trigger(once=True)` with a fresh checkpoint per run so each hour starts from zero.
C. `trigger(processingTime='1 hour')` running as a continuous job that never stops.
D. `trigger(availableNow=True)`, run hourly by a job

F4.4 A `foreachBatch` function appends each micro-batch to a Delta table. After a failure and retry, some rows appear twice. What is the fix?

A. Make the write idempotent: `MERGE`, or `txnAppId` / `txnVersion` with the batch id.
B. Switch the output mode to complete so every batch rewrites the target from scratch.
C. Disable retries on the job so a failed batch is never executed a second time.
D. Set a longer `checkpointLocation` retention so the failed batch id is remembered on restart.

F4.5 A downstream stream reads `prod.bronze.orders` as a Delta source. An operator runs `DELETE` on old bronze rows for retention. What happens next?

A. The stream ignores the delete silently, since deletes never affect streaming reads.
B. The stream restarts from version 0, because the delete invalidates every stored offset.
C. The stream applies the deletes to the target automatically, because Delta sources are CDC-aware.
D. The stream fails on that commit unless it skips change commits or is fully refreshed.

Self-check · F4 Structured Streaming — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

F4.1 Answer B

Checkpoints store offsets, commits and state. Trigger and rate-limit options can change freely between restarts.

A Offsets do not depend on the trigger interval.

C No such migration step exists or is needed.

D Unnecessary and loses state; trigger changes are safe.

F4.2 Answer C

Without a watermark Spark cannot know when a window is final, so it keeps every window's state. A watermark lets it finalise and evict.

A Complete mode keeps all state forever by design.

B Deleting from the sink does not affect the operator's state store.

D More partitions spread state; they do not expire it.

F4.3 Answer D

`availableNow` processes everything available (in several rate-limited batches) and then stops, giving incremental batch semantics.

A Default trigger runs continuously; rate limits do not make it stop.

B A fresh checkpoint reprocesses everything; `once` is also superseded by availableNow.

C The query stays up and holds compute between batches.

F4.4 Answer A

foreachBatch guarantees each batch runs at least once. Idempotent writes turn that into exactly-once results.

B Complete mode is for aggregations and does not apply to foreachBatch appends.

C That trades duplicates for lost data and manual recovery.

D The checkpoint already remembers; the batch legitimately re-runs after failure.

F4.5 Answer D

Delta streaming sources assume append-only input. Use `skipChangeCommits` for retention deletes, or read the Change Data Feed to process them.

A Non-append commits raise an error by default.

B Offsets remain valid; the problem is the non-append commit.

C Plain Delta streaming reads appends only; it does not propagate deletes.

Notes · F4 Structured Streaming

What clicked, what did not, commands to try, questions to ask a colleague.

Part II

The nine exam domains

Each chapter follows the same path: official objectives, concepts explained plainly, production advice, exam traps, a lab, practice questions with every option explained, and a notes page.

1 · Developing code (23%)

2 · Ingestion and acquisition (12%)

3 · Data manipulation (12%)

4 · Monitoring and alerting (10%)

5 · Cost and performance (15%)

6 · Security and compliance (8%)

7 · Governance (5%)

8 · Debugging and deploying (10%)

9 · Data modeling (5%)

Domain 1

1 Developing Code for Data Processing

Project structure, dependencies, UDFs, Declarative Pipelines, Lakeflow Jobs, serverless, testing and streaming state. The biggest domain: about 14 questions.

23% of the exam · about 14 questions

What you will be able to do

- Structure a Python project so logic is testable and deployable with Declarative Automation Bundles
- Fix dependency problems on serverless, pipelines and classic compute
- Choose between built-ins, SQL UDFs, Pandas UDFs, Python UDFs and UC functions
- Choose streaming table vs materialized view; build SCD1/SCD2 with AUTO CDC
- Design Lakeflow Jobs with control flow, retries and the right compute
- Test transformations and reason about stateful streaming

Matching free course

- Advanced Techniques with Spark Declarative Pipelines
- Automated Deployment with Declarative Automation Bundles
- Build Data Pipelines with Lakeflow SDP (Associate refresher)

Study checklist

- ☐ Read sections 1.1–1.7
- ☐ Lab 1: bronze→silver→gold pipeline with AUTO CDC
- ☐ Practice questions (16)
- ☐ Re-read traps for any question missed

Official objectives (new exam): scalable Python project structure for bundles; troubleshoot dependencies (PyPI, wheels, source archives) across serverless, pipelines and bundles; Pandas/Python/SQL UDFs including Unity Catalog functions; Declarative Pipelines with Auto Loader; Lakeflow Jobs via UI, API and CLI; streaming tables vs materialized views; AUTO CDC for SCD 1/2; Structured Streaming vs Declarative Pipelines; control flow; serverless configuration and performance mode; high-memory tasks; retries; testing; stateful streaming semantics.

1.1 Project structure for Declarative Automation Bundles

In plain words

A bundle is a folder that describes, in YAML, everything a project deploys (jobs, pipelines, schedules, permissions) next to the code those resources run. One command deploys the whole thing to dev, staging or prod. The key design idea: business logic lives in an importable Python package, not in notebooks, so it can be unit-tested on a laptop or in CI without a cluster.

```
my_project/
├── databricks.yml          # bundle name, targets (dev/prod), variables, artifacts
├── resources/              # one YAML per job / pipeline
│   ├── claims_job.yml
│   └── claims_pipeline.yml
├── src/claims/             # importable package → built as a wheel
│   ├── __init__.py
│   ├── transforms.py       # pure functions: DataFrame -> DataFrame
│   └── main.py             # entry point for python_wheel_task
├── notebooks/              # thin entry points that call the package
├── tests/unit/             # pytest, no cluster needed
├── tests/integration/      # run against the dev target
└── pyproject.toml
```

```yaml
bundle: {name: claims}
artifacts:
  claims_whl: {type: whl, path: .}
variables:
  catalog: {default: dev}
targets:
  dev:
    mode: development      # names prefixed "[dev <user>]", schedules paused
    default: true
  prod:
    mode: production       # requires run_as; clean names; schedules active
    workspace: {host: https://adb-123.azuredatabricks.net}
    run_as: {service_principal_name: sp-claims-etl}
    variables: {catalog: prod}
```

What reviewers look for

Logic in `src/`, tests in `tests/`, notebooks under 30 lines, every environment difference expressed as a variable (catalog, schema, cluster size), and prod deployed only by CI as a service principal. If a PR changes a notebook with 400 lines of transformation code, ask for it to move into the package.

1.2 Libraries and dependencies

Mechanism	Scope	When and gotchas
`%pip install pkg==1.2`	Notebook session	Iteration only. Put at the top; it restarts Python.
Task / cluster libraries (PyPI, Maven, wheel in a volume)	Classic compute	Production standard on classic. Store wheels in UC volumes, pin versions.
Serverless environment (`environment_key`, `dependencies`)	Serverless task or notebook	Serverless has **no cluster libraries and no init scripts**. Declare packages, wheel paths or `-r requirements.txt` here. Can also request high memory.
Pipeline environment / dependencies	Declarative Pipelines	Declared in pipeline settings or the bundle.
Init scripts	Classic only	OS-level packages. Not available on serverless.
Source archive / git URL	`%pip`	Needs egress; a pinned wheel is more reproducible.

Troubleshooting patterns: `ModuleNotFoundError` on serverless → add it to the environment; works on driver but not in UDFs → it was installed only on the driver (for example with `!pip`); version conflict with a

runtime-bundled library → pin explicitly and check runtime release notes; works in dev, fails in prod → different runtime or environment version, or a relative path.

1.3 User-defined functions

Type	How it runs	Speed	Use when
Built-in functions	Native, optimised, Photon	Best	Always first choice
SQL UDF (`CREATE FUNCTION ... RETURN expr`)	Inlined into the plan	Near native	Reusable SQL logic; masks, filters, ABAC
Pandas UDF (`@pandas_udf`)	Arrow batches to Python	Good	Vectorisable Python (numpy, ML scoring)
Python UDF (`@udf`)	Row by row in Python	Slowest	Small data, rare logic
UC Python function (`LANGUAGE PYTHON`)	Sandboxed, governed with `EXECUTE`	Moderate	Python logic shared with SQL warehouses

```
CREATE OR REPLACE FUNCTION prod.util.last4(s STRING) RETURNS STRING
  COMMENT 'Keep last 4 characters' RETURN CONCAT('****', RIGHT(s, 4));
GRANT EXECUTE ON FUNCTION prod.util.last4 TO `data_engineers`;
```

UDF order of preference

Built-in → SQL UDF → Pandas UDF → Python UDF. "Enable Photon" does not speed up a Python UDF. Only UC functions (SQL or Python) can be used by row filters, column masks and ABAC policies; a notebook-registered UDF cannot.

1.4 Lakeflow Spark Declarative Pipelines

In plain words

In a declarative pipeline you describe the tables you want and the query that defines each one. Databricks works out the order, runs it, manages checkpoints and retries, records data quality, and draws the lineage graph. You write the "what"; the platform handles the "how".

	Streaming table	Materialized view
Processes	Only new rows since last update	The query result, kept equal to the sources
Sources	Append-only (Auto Loader, Kafka, append-only Delta)	Anything: updates, deletes, joins, aggregates
Mechanism	Checkpoint, exactly-once per flow	Incremental refresh on serverless when possible, else full recompute
Best for	Bronze, append-only silver, high volume, low latency	Dimensions, aggregates, gold, anything needing correctness over changing inputs
Cannot	Reflect upstream updates/deletes (fails unless skipped)	Offer streaming latency

```sql
CREATE OR REFRESH STREAMING TABLE bronze_claims
AS SELECT *, _metadata.file_path AS src_file, current_timestamp() AS loaded_at
FROM STREAM read_files('/Volumes/prod/landing/claims/', format => 'json');

CREATE OR REFRESH STREAMING TABLE silver_claims (
  CONSTRAINT id_present   EXPECT (claim_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT member_known EXPECT (member_id IS NOT NULL) ON VIOLATION FAIL UPDATE,
  CONSTRAINT positive_amt EXPECT (paid_amount >= 0)             -- warn only
) CLUSTER BY (service_date)
AS SELECT claim_id, member_id, CAST(paid_amount AS DECIMAL(12,2)) AS paid_amount,
          to_date(service_date) AS service_date
FROM STREAM bronze_claims;

CREATE OR REFRESH MATERIALIZED VIEW gold_paid_by_month
AS SELECT date_trunc('MONTH', service_date) AS month, SUM(paid_amount) AS paid
FROM silver_claims GROUP BY ALL;
```

``` python from pyspark import pipelines as dp # legacy: import dlt

@dp.table(name="silver_claims", cluster_by=["service_date"])

@dp.expect_or_drop("id_present", "claim_id IS NOT NULL")

@dp.expect_or_fail("member_known", "member_id IS NOT NULL")

def silver_claims(): return spark.readStream.table("bronze_claims").select("claim_id", "member_id", "paid_amount", "service_date")
```

```
**Expectations** have three behaviours: warn (keep row, count it), drop (discard row, count it), fail (stop the update). Choose fail for invariants whose violation means something upstream is broken; drop for known noise; warn while you learn the data. The quarantine pattern (keep bad rows with a reason) is in Chapter 3.

### Flows and multiple sources

A streaming table can have several **flows** writing into it: for example, one Auto Loader flow per regional folder, all appending to one bronze table. This is the multi-flow pattern taught in the Advanced SDP course, and it avoids a `UNION` that would have to be rebuilt whenever a source is added.

```sql
CREATE OR REFRESH STREAMING TABLE bronze_encounters;
CREATE FLOW east AS INSERT INTO bronze_encounters BY NAME SELECT * FROM STREAM
read_files('/Volumes/prod/landing/east/');
CREATE FLOW west AS INSERT INTO bronze_encounters BY NAME SELECT * FROM STREAM
read_files('/Volumes/prod/landing/west/');
```

AUTO CDC (formerly APPLY CHANGES)

> **In plain words**
>
> A change feed is a list of "this row was inserted, updated, deleted" events. AUTO CDC turns such a feed into a correct table: it sorts events by a sequence column (so late or out-of-order events do not overwrite newer data), applies deletes, and keeps either only the current row (SCD Type 1) or the full history with validity dates (SCD Type 2).
```

```
CREATE OR REFRESH STREAMING TABLE dim_member;
CREATE FLOW member_cdc AS AUTO CDC INTO dim_member
FROM STREAM bronze_member_changes
KEYS (member_id)
APPLY AS DELETE WHEN op = 'D'
SEQUENCE BY change_ts
COLUMNS * EXCEPT (op, change_ts)
STORED AS SCD TYPE 2
TRACK HISTORY ON (address, plan_code);      -- only these columns open a new version
```

SCD Type 2 targets get `__START_AT` and `__END_AT` columns. For sources that deliver full snapshots rather than changes, use `AUTO CDC FROM SNAPSHOT`. An AUTO CDC target is not append-only, so downstream consumers read it as a batch source (materialized view) or via Change Data Feed.

Structured Streaming or Declarative Pipelines?

Choose **Declarative Pipelines** for standard medallion ETL, CDC, expectations and minimal operations: this is the default exam answer. Choose **hand-written Structured Streaming** when you need an unsupported sink, custom stateful logic (`transformWithState` , `applyInPandasWithState`), or complex `foreachBatch` behaviour.

1.1 Lakeflow Jobs: orchestration

A job is a DAG of tasks: notebook, Python script, wheel, SQL, pipeline, dbt, JAR, Run job, If/else, For each, SQL alert, dashboard.

Need	Feature
Run a cleanup or notification whatever happens upstream	Task dependency **Run if** = `ALL_DONE` (others: `ALL_SUCCESS` default, `AT_LEAST_ONE_SUCCESS` , `NONE_FAILED` , `AT_LEAST_ONE_FAILED` , `ALL_FAILED`)
Branch on a condition	**If/else** task on a value, e.g. `{{tasks.check.values.rows}} > 0`
Loop over a list with limited parallelism	**For each** task over a JSON array (literal or task value), with concurrency
Pass values between tasks	`dbutils.jobs.taskValues.set(key, value)` → `{{tasks.<task>.values.<key>}}`
Reuse another job	**Run job** task
Start on data arrival	File arrival or table update triggers
Parameters	Job parameters, overridable on run-now and repair; dynamic values like `{{job.start_time.iso_date}}`

Retries re-run the whole task. That is safe only for **idempotent** tasks (overwrite a partition, MERGE, availableNow streams). For tasks with external side effects (sending emails, calling a payment API, blind appends) set retries to 0 and alert instead.

Serverless configuration

- **Performance mode:** *standard* (cost-optimised; may start and scale more slowly) vs *performance optimized* (fast start and scaling, higher cost). Latency-sensitive jobs get performance mode; overnight batch gets standard.
- **High memory** for notebook tasks that OOM on driver-heavy code, set in the environment.
- **Environment version** pins the Python/library baseline, like a runtime version on classic.

Healthcare scheduling reality

Claims files arrive late and irregularly. Prefer file-arrival triggers with `availableNow` over fixed schedules that run on empty folders. Make the "notify" task `ALL_DONE` so on-call hears about failures. Run everything as a service principal owned by the team, never as a person.

1.2 Testing and debugging

Write transformations as pure functions and chain them with `DataFrame.transform`. Unit-test with `pyspark.testing`:

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual

def clean(df):  return df.filter("paid_amount >= 0").withColumn("d", F.to_date("service_date"))

def test_clean(spark):
    src = spark.createDataFrame([(1, 10.0, "2026-01-02"), (2, -1.0, "2026-01-02")], "id INT, paid_amount
DOUBLE, service_date STRING")
    exp = spark.createDataFrame([(1, 10.0, "2026-01-02", date(2026, 1, 2))], "id INT, paid_amount
DOUBLE, service_date STRING, d DATE")
    assertDataFrameEqual(clean(src), exp)     # row order ignored by default; rtol/atol for floats
```

Notebook debugger: breakpoints and variable inspection for driver-side Python. Pipelines: development mode (reuse compute, no retries) and **validate** (check the graph without processing data).

1.3 Stateful streaming semantics

The exam names four mechanisms (Foundations F4 explains each): **checkpoint** (offsets + state; restart gives exactly-once), **watermark** (bounds state, finalises windows, required for append-mode aggregations and `dropDuplicatesWithinWatermark`), **output mode** (append for Delta sinks; update needs foreachBatch/MERGE; complete keeps everything), and **foreachBatch** (custom writes; make them idempotent).

Remember

- Logic in a package, thin notebooks, tests in CI, bundles with dev/prod targets; prod = production mode + service principal.
- Serverless dependencies go in the environment; no init scripts or cluster libraries there.
- Built-ins > SQL UDF > Pandas UDF > Python UDF; only UC functions work in masks/filters/policies.
- Streaming table = append-only incremental; MV = correct over any source, incremental on serverless when possible.
- AUTO CDC: KEYS, SEQUENCE BY, APPLY AS DELETE, SCD TYPE 1/2, TRACK HISTORY ON.
- Run if ALL_DONE for always-run tasks; For each for loops; no retries for non-idempotent side effects.
- Performance mode for latency, standard for cost; high memory for driver OOM on serverless.

Lab 1 · A pipeline with history (60 min)

1. Create a volume and write three small JSON files of member changes: `{member_id, address, op, change_ts}` with one out-of-order update and one delete.
2. Create a pipeline (serverless) with a bronze streaming table from `read_files`, and an AUTO CDC flow into an SCD Type 2 `dim_member`.
3. Add an expectation `EXPECT (member_id IS NOT NULL) ON VIOLATION DROP ROW` and include one bad record. Check the drop count in the pipeline UI.
4. Add a materialized view counting current members by state (`WHERE __END_AT IS NULL`).
5. Add a fourth file and run an update: only the new file is processed. Look at `__START_AT` / `__END_AT`.

Domain 1 practice questions

Answer first, check later. Score: _____ / 16 · Date: _____ · Retake date: _____

Q1 All transformation logic lives in notebooks called directly by job tasks. Unit tests need a running cluster and pull requests cannot be validated in CI. Which change best fixes this while preparing for deployment with bundles?

- A Convert every notebook into a SQL task so that tests can be executed on a serverless SQL warehouse during CI.
- B Use `%run` so notebooks share helper code, and add a final job task that runs the tests in production.
- C Move logic into a `src/` package built as a wheel by the bundle, keep notebooks thin, and run pytest in CI.
- D Keep the notebooks, but store them in DBFS and reference them from the bundle by absolute path.

Q2 A serverless job task fails with `ModuleNotFoundError: No module named 'great_expectations'`. The same code works on an all-purpose cluster where the package is a cluster library. What should the engineer do?

- A Declare the package in the task's serverless environment dependencies or requirements file.
- B Copy the package source into DBFS root and append that folder to `sys.path` in the notebook.
- C Mark the library as "Install on all clusters" so serverless tasks inherit it too.
- D Add an init script that pip-installs the package when the serverless compute starts.

Q3 A Python UDF that extracts a code with a regular expression is the slowest stage of a 2-billion-row job. Which change gives the largest speed-up with the least rewriting?

- A Enable Photon so that the UDF's regular expression runs in the vectorised native engine instead of Python.
- B Use the built-in `regexp_extract`; if that is impossible, a Pandas UDF.
- C Cache the input DataFrame before applying the UDF so each row is read from memory.
- D Increase `spark.sql.shuffle.partitions` so more tasks share the UDF work across the cluster.

Q4 A silver streaming table in a declarative pipeline reads a bronze Delta table. A backfill job now runs `UPDATE` on bronze to fix historical prices. What happens on the next pipeline update?

A. The streaming table applies the updated prices automatically on the next incremental run.

B. The pipeline converts the streaming table to a materialized view and recomputes it from scratch without being asked.

C. The flow fails since the source is not append-only; full-refresh, use `skipChangeCommits`, or use an MV.

D. Only newly inserted rows are processed; the updated rows are ignored without any error.

Q5 Which statement about materialized views in declarative pipelines is correct?

A. They require append-only sources, just like streaming tables that read from Delta.

B. They can only be refreshed by a full recompute, whatever compute type the pipeline uses.

C. They match their sources after refresh, incrementally on serverless when cheaper.

D. They keep a streaming checkpoint that can be reset with a full refresh when the query changes.

Q6 A CDC feed delivers `customer_id`, `op` (I/U/D), `event_ts` and attributes, sometimes out of order. The team needs full address history with validity intervals. Which approach needs the least custom code?

A. AUTO CDC with `KEYS (customer_id)`, `SEQUENCE BY event_ts`, deletes on `op='D'`, `STORED AS SCD TYPE 2`.

B. An append-only streaming table of all events plus a view that picks the latest row per customer.

C. A materialized view with `LEAD()` over `event_ts` to compute each row's end date from the raw feed.

D. A `foreachBatch` MERGE that closes the current row and opens a new one for every change in each micro-batch.

Q7 A job has tasks `extract` → `transform` → `notify_ops`. The ops team wants `notify_ops` to run whether `transform` succeeds or fails. What should be configured?

A. Remove the dependency and run `notify_ops` in parallel with `extract` at the start of the job.

B. Set three retries on `transform` so that it eventually succeeds and the notification runs.

C. Wrap `transform` in try/except that swallows errors so the task always reports success.

D. Set the Run if dependency condition of `notify_ops` to `ALL_DONE` in the job's task settings.

Q8 A nightly job must process a list of 40 regions produced by an upstream task, at most 5 at a time, without defining 40 tasks. Which design fits?

A. A Python task using `ThreadPoolExecutor` in a notebook, because jobs have no iteration construct.

B. A Run job task that triggers a child job with `max_concurrent_runs` set to 5 for the regions.

C. An If/else task that loops back on itself until the list of regions produced upstream is empty.

D. A For each task over `{{tasks.list_regions.values.regions}}` with concurrency 5.

Q9 A task appends daily rows to a Delta table and then calls an external payment API. With retries set to 3, a transient failure caused duplicate payments. What is the correct fix?

A. Set retries to 0 for this non-idempotent task and alert on failure instead.
B. Enable optimized writes on the target table so that repeated appends are deduplicated on commit.
C. Increase retries to 5 and add a retry interval so the API has time to recover between attempts.
D. Move the task to serverless compute, which never retries tasks that have external side effects.

Q10 A notebook task on serverless compute fails with a driver out-of-memory error while building a large pandas DataFrame. The code cannot change this sprint. What is the most direct fix?

A. Set `spark.sql.shuffle.partitions` to 2000 so each partition that reaches the driver is smaller.
B. Select the high-memory option in the task's serverless environment.
C. Switch the task to performance-optimized mode, which gives the driver more memory.
D. Add an init script that raises the JVM heap size of the serverless driver.

Q11 Which assertion compares two DataFrames while ignoring row order and small floating-point differences?

A. `assert actual.subtract(expected).count() == 0`, which ignores order and handles floats.
B. `assertDataFrameEqual(actual, expected)` from `pyspark.testing`, with `rtol` / `atol` if needed.
C. `assertSchemaEqual(actual.schema, expected.schema)` followed by a comparison of row counts.
D. `assert actual.collect() == expected.collect()` after sorting both DataFrames by every column.

Q12 A pipeline reads Kafka, applies validation rules with metrics, deduplicates late events and writes Delta. Analysts with limited Spark experience will maintain it. What do you recommend?

A. Structured Streaming with `foreachBatch`, because declarative pipelines cannot read from Kafka.
B. Structured Streaming, because expectations can only be attached to batch materialized views.
C. A scheduled batch job using COPY INTO, since streaming is too complex for this team to operate.
D. Declarative Pipelines: managed checkpoints and retries, expectation metrics, watermark dedup.

Q13 A bundle target `dev` uses `mode: development`. What should the engineer expect after `databricks bundle deploy -t dev`?

A. Resources are deployed with the same names as production, overwriting the prod versions.
B. Resource names get a `[dev <username>]` prefix, and job schedules and triggers are paused.
C. The deployment is rejected unless the local Git branch is `main` and has no uncommitted changes.
D. Jobs run as the configured service principal and schedules are activated immediately.

Q14 A streaming query counts events in 5-minute windows. Very late events make state grow without bound, and after a driver restart some windows were double-counted. Which configuration fixes both?

A. Add a watermark on event time and keep the same checkpoint location across restarts.
B. Use complete output mode without a watermark so every window is rewritten in full each batch.
C. Delete the checkpoint before each restart and increase shuffle partitions to spread the state.
D. Wrap the aggregation in `foreachBatch` and commit Kafka offsets manually to a Delta table.

Q15 A "keep only the last 4 characters" rule must be reused by SQL warehouse users, pipeline code and an ABAC column-mask policy, with EXECUTE governed centrally. What should be created?

A. A Pandas UDF registered in each notebook session, documented in the team wiki for reuse.
B. A shared notebook with the Python logic, imported by other notebooks through `%run`.
C. A Unity Catalog SQL function with EXECUTE granted to the relevant groups.
D. A Scala UDF packaged in a JAR and installed as a library on every cluster in the workspace.

Q16 One serverless job feeds an operational dashboard and must start within seconds; another runs an overnight backfill where a slower start is fine. Which configuration matches each?

A. High-memory environment for the dashboard job and performance-optimized mode for the backfill.
B. Classic job clusters for both, because serverless compute cannot be tuned for latency.
C. Standard mode for both, since performance mode only changes whether Photon is enabled.
D. Performance-optimized mode for the dashboard job; standard mode for the backfill.

Domain 1 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Answer C

A package with pure functions is importable by pytest locally and in CI; the bundle builds it as a wheel artifact for tasks to use.

A Changing the language does not make logic importable or testable without the platform.

B `%run` code is not importable by pytest, and tests belong before deployment, not in prod.

D Storage location does not change testability; DBFS paths are legacy.

Q2 Answer A

Serverless compute takes dependencies from the task's environment specification (package names, wheel paths or a requirements file).

B A fragile hack that bypasses governance and may not reach all processes.

C Cluster libraries apply only to classic clusters.

D Serverless does not support init scripts.

Q3 Answer B

Native functions run in the JVM/Photon and are visible to the optimizer; Pandas UDFs vectorise via Arrow when Python is unavoidable.

A Photon cannot execute Python UDFs.

C Caching does not reduce the Python serialization per row.

D The cost is per-row Python execution, not the shuffle partition count.

Q4 Answer C

Streaming tables require append-only sources. Detecting updated or deleted rows raises an error; you choose how to handle it.

A Streaming tables cannot consume updates from their source.

B Pipelines never change a dataset's type on their own.

D Non-append commits raise an error unless explicitly skipped.

Q5 Answer C

MVs guarantee results consistent with the sources; serverless uses cost-based incremental refresh where possible.

A MVs accept any source, including updates and deletes.

B Incremental refresh exists on serverless pipelines.

D MVs have no streaming checkpoint.

Q6 Answer A

AUTO CDC handles ordering, deduplication, deletes and SCD Type 2 validity columns declaratively.

B Gives current state, not validity intervals, and ignores deletes.

C Does not handle deletes or dedup robustly and recomputes history.

D It works, but it is exactly the custom code AUTO CDC removes.

Q7 Answer D

`ALL_DONE` runs a task after all dependencies finish, regardless of their result.

A The notification would run before the outcome is known.

B Retries do not guarantee success, so the notification can still be skipped.

C Hides real failures from monitoring.

Q8 For each iterates over a JSON array from a literal or a task value and runs a nested task per element with a concurrency limit.

Answer D

A Jobs do have one: the For each task.

B Limits concurrent runs but does not iterate over the list.

C If/else branches; it does not loop.

Q9 Retries re-run the whole task. Tasks with external side effects must not be retried automatically.

Answer A

B Optimized writes change file sizes, not duplicates.

C More retries mean more duplicate side effects.

D Serverless has no such behaviour.

Q10 Serverless notebook tasks can request high memory through the environment settings.

Answer B

A Shuffle partitions do not change driver memory.

C Performance mode affects start-up and scaling speed, not memory.

D Serverless has no init scripts.

Q11 assertDataFrameEqual checks schema and data, ignores row order by default, and supports tolerances.

Answer B

A subtract ignores duplicates and one direction of differences.

C Schema plus count does not compare values.

D Exact equality fails on tiny float differences.

Q12 Declarative Pipelines give the lowest operational burden for standard streaming ETL with quality rules.

Answer D

A Pipelines read Kafka natively.

B Expectations work on streaming tables too.

C COPY INTO reads files, not Kafka, and is not streaming.

Q13 Development mode isolates each developer's copy ([dev user] prefix), pauses schedules and triggers, and allows fast iteration.

Answer B

A The prefix exists precisely to avoid collisions.

C Development mode imposes no branch rule.

D That is production-mode behaviour.

Q14 The watermark bounds state; the checkpoint restores offsets and state so recovery is exactly-once.

Answer A

B Complete mode keeps all state and does not fix recovery.

C Deleting checkpoints causes reprocessing; partitions do not bound state.

D Reinvents the checkpoint and still lacks a watermark.

Q15 UC functions are governed objects usable from any engine and are the building blocks of masks, filters and ABAC policies.

Answer C

A Session UDFs are not governed and cannot be used by policies.

B Not callable from SQL warehouses or policies.

D Not available on warehouses and not governed by UC.

Q16 Performance mode trades cost for faster start and scaling; standard mode is the cost-optimised default.

Answer D

A High memory addresses OOM, not latency, and the backfill does not need speed.

B Serverless offers exactly this knob.

C Performance mode governs start-up and scaling speed.

Notes · Domain 1

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 2

2 Data Ingestion and Acquisition

Getting data in: files, message buses, database CDC, and data that stays elsewhere (sharing, clean rooms, federation).

12% of the exam · about 7 questions

What you will be able to do

- Read every exam-relevant format correctly, including Iceberg, XML and binary
- Configure Auto Loader for scale, schema evolution and exactly-once
- Choose COPY INTO, Auto Loader or batch reads
- Ingest Kafka-style buses into replayable bronze
- Set up Lakeflow Connect database CDC and know its failure modes
- Choose between OpenSharing, Clean Rooms and Lakehouse Federation

Matching free course

- Advanced Techniques with Spark Declarative Pipelines
- Data Ingestion with Lakeflow Connect (Associate refresher)

Study checklist

- ☐ Read 2.1–2.7
- ☐ Lab 2: Auto Loader schema evolution
- ☐ Practice questions (20)

Official objectives: ingestion from message buses (Kafka, Kinesis, Pub/Sub) and cloud storage for Delta, Parquet, JSON, Iceberg, CSV and binary; incremental CDC pipelines with Delta or Iceberg targets; Lakeflow Connect CDC from SQL Server, MySQL and PostgreSQL; OpenSharing (D2D and D2O) and Clean Rooms; Lakehouse Federation with UC permissions and connection credentials.

2.1 File formats in one table

Format	Read with	Remember
Delta	`spark.read.table`, `readStream.table`	Streaming options: `skipChangeCommits`, `startingVersion`, `readChangeFeed`, `maxFilesPerTrigger`
Iceberg	UC managed Iceberg (`USING iceberg`), foreign Iceberg catalogs, or UniForm on Delta	External engines read and write managed Iceberg through the Iceberg REST catalog
Parquet / ORC	`format("parquet")`	`mergeSchema` if files differ; `CONVERT TO DELTA` upgrades in place without rewriting
Avro	`format("avro")`, `from_avro`	Common for Kafka payloads with a schema registry
JSON	`format("json")`	`multiLine` for pretty-printed files; prefer VARIANT for changing shapes
CSV	`format("csv")`	`header`, `sep`, `quote`, `escape`, `mode` = PERMISSIVE (default, with corrupt-record column) / DROPMALFORMED / FAILFAST; `badRecordsPath`
XML	`format("xml")`	`rowTag` is required
Binary	`format("binaryFile")`	One row per file with path, length, content: PDFs, images, DICOM headers

`read_files()` is the SQL table-valued function for all of these and becomes Auto Loader when used with `STREAM`.

2.2 Auto Loader

> **In plain words**
> Auto Loader watches a folder and processes each new file exactly once. It remembers which files it has seen in the checkpoint, can infer and evolve the schema, and scales from a handful to billions of files.

```
(spark.readStream.format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", "/Volumes/prod/chk/claims_schema")
   .option("cloudFiles.inferColumnTypes", "true")
   .option("cloudFiles.schemaHints", "paid_amount DECIMAL(12,2)")
   .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
   .load("/Volumes/prod/landing/claims/")
 .writeStream.option("checkpointLocation", "/Volumes/prod/chk/bronze_claims")
   .option("mergeSchema", "true").trigger(availableNow=True).toTable("prod.bronze.claims"))
```

Schema evolution mode	On a new column
`addNewColumns` (default)	Stream fails once, schema location updated; succeeds on restart (so configure job retries)
`rescue`	Schema unchanged; unexpected data goes to `_rescued_data`
`failOnNewColumns`	Fails until you change the schema yourself
`none`	Ignores new columns (data lost unless rescued)

`_rescued_data` holds, as JSON, anything that did not fit the schema (new fields, type mismatches, case mismatches). It is your quarantine signal.

File discovery: directory listing (default, incremental) vs **file notification / managed file events** on UC external locations for huge or busy directories. `includeExistingFiles` affects only the first run; `maxFilesPerTrigger` sets batch size, not discovery speed; `allowOverwrites` reprocesses changed files.

COPY INTO vs Auto Loader vs batch read

	COPY INTO	Auto Loader	`spark.read` + append
Nature	Idempotent SQL batch command	Incremental streaming source	Full re-read
Scale	Thousands of files	Millions to billions	Small, one-off
Best for	SQL-only teams, warehouse loads	Default ingestion	Ad hoc

2.3 Message buses

Keep bronze **raw and replayable**: store Kafka's `key`, `value` (binary), `topic`, `partition`, `offset` and `timestamp`, and decode (`from_avro`, `from_json`, `parse_json`) in silver. If decoding logic changes, you replay from bronze instead of from Kafka, whose retention is short. Kinesis and Pub/Sub have equivalent connectors; Event Hubs speaks the Kafka protocol.

```
raw = (spark.readStream.format("kafka")
       .option("kafka.bootstrap.servers", brokers).option("subscribe", "adt-events")
       .option("startingOffsets", "earliest").load()
       .select("key", "value", "topic", "partition", "offset", "timestamp"))
```

2.4 Lakeflow Connect: managed database CDC

Component	Role
Connection	UC object with host and credentials (from secrets)
Ingestion gateway	Runs continuously; captures snapshots and the change log (SQL Server change tracking/CDC, MySQL binlog, PostgreSQL logical replication) into staging in a UC volume
Ingestion pipeline	Serverless pipeline that applies staged changes to destination streaming tables (SCD 1, SCD 2 or append)
Destination tables	One streaming table per source table, governed by UC

Source prerequisites: change tracking/CDC enabled (SQL Server), `binlog_format=ROW` (MySQL), `wal_level=logical` with a replication slot (PostgreSQL), primary keys, network path, a least-privilege database user.

> **The gateway must keep running**
>
> If the gateway is stopped long enough for the source to recycle its log (or a replication slot is dropped), the changes in between are gone. Incremental recovery is impossible; the fix is a **full refresh** of affected tables. Staging holds only what the gateway already captured.

For other change feeds (Kafka, files, upstream CDF) the same **AUTO CDC** construct applies changes. Targets can be exposed to Iceberg clients with UniForm (`delta.universalFormat.enabledFormats = 'iceberg'`), or created as managed Iceberg tables when external engines must write.

2.5 Data that stays where it is

Need	Feature	Key facts
Give another organisation live read access, no copies	**OpenSharing** (Delta Sharing)	Share = collection of tables, views, volumes, models, notebooks. **D2D**: recipient has Unity Catalog, identified by metastore ID. **D2O** (open): bearer token + activation link, read with pandas/Spark/BI. Streaming or CDF on shared tables needs `WITH HISTORY`. Rotate leaked tokens with `ALTER RECIPIENT ... ROTATE TOKEN`. Per-recipient rows with `current_recipient()` in a dynamic view.
Joint analysis where nobody sees the other's rows	**Clean Rooms**	Parties share data into a Databricks-hosted room; only approved notebooks run; only outputs leave.
Query an external database in place	**Lakehouse Federation**	`CREATE CONNECTION` + `CREATE FOREIGN CATALOG`; UC grants apply; predicates pushed down where the connector supports them.

Need	Feature	Key facts
Replicate an operational database continuously	**Lakeflow Connect**	Not federation: federation queries live, it does not copy.
Teams in the same metastore	**Plain GRANTs**	Sharing inside one metastore is unnecessary.

```
CREATE SHARE research_share;
ALTER SHARE research_share ADD TABLE prod.gold.deid_encounters WITH HISTORY;
CREATE RECIPIENT univ_lab;                                  -- open recipient: activation link + token
GRANT SELECT ON SHARE research_share TO RECIPIENT univ_lab;

CREATE CONNECTION pg_crm TYPE postgresql OPTIONS (host 'crm.internal', port '5432',
  user secret('crm','user'), password secret('crm','pwd'));
CREATE FOREIGN CATALOG crm USING CONNECTION pg_crm OPTIONS (database 'crm_prod');
GRANT USE CATALOG, USE SCHEMA, SELECT ON CATALOG crm TO `analysts`;
```

Healthcare lens

Research partners usually get a **de-identified** view through OpenSharing, never raw tables; hospitals comparing patient overlap use **Clean Rooms**; the EHR's operational database is replicated with **Lakeflow Connect** so analytics never loads the clinical system during the day.

Remember

- Auto Loader = default ingestion; `addNewColumns` fails once then succeeds; `rescue` → `_rescued_data`.
- Notifications/file events fix discovery latency; `maxFilesPerTrigger` does not.
- COPY INTO = idempotent SQL batch for thousands of files; `CONVERT TO DELTA` = in-place upgrade.
- Bronze keeps raw bus payloads + offsets; decode in silver.
- Lakeflow Connect: connection + always-on gateway + pipeline; a log gap needs a full refresh.
- Sharing = another org, live, read-only (`WITH HISTORY` for streaming); Clean Rooms = no row exposure; Federation = query in place; same metastore = GRANT.

Lab 2 · Schema drift on purpose (40 min)

1. Put `a.json` with fields `{id, name}` in a volume. Start an Auto Loader stream (`availableNow`) into a bronze table.
2. Add `b.json` with an extra field `phone` and a string in `id`. Rerun: observe the failure with `addNewColumns`, rerun again and see the new column.
3. Switch a fresh stream to `rescue` mode and find the bad `id` value inside `_rescued_data`.
4. Load the same files with `COPY INTO` twice and confirm the second run loads zero files.

Domain 2 practice questions

Answer first, check later. Score: _____ / 20 · Date: _____ · Retake date: _____

Q1 A vendor drops about 200,000 small JSON files per hour into one folder. Directory listing has become the bottleneck of the Auto Loader stream. What most directly fixes discovery latency?

A. Switch to a scheduled COPY INTO so that files are loaded in large idempotent batches.
B. Raise `cloudFiles.maxFilesPerTrigger` so each micro-batch picks up more of the new files.
C. Switch to file notification mode, or managed file events on the UC external location.
D. Set `cloudFiles.includeExistingFiles` to false so the stream skips the files already present.

Q2 An Auto Loader stream in the default schema evolution mode stops with `UnknownFieldException` after the producer adds a column. What is expected and what should the operator do?

A. Expected: the schema was already updated, so restart the stream.
B. The stream is permanently broken; delete the checkpoint and restart from the beginning.
C. Ask the producer to write Parquet instead, since JSON schemas cannot evolve in Auto Loader.
D. Switch `schemaEvolutionMode` to `none` so that the new column is added silently next time.

Q3 A SQL-only team loads a few hundred new CSV files each night from a SQL warehouse and must be able to re-run safely without duplicates. What fits best?

A. `COPY INTO` with `FILEFORMAT = CSV`, which skips files it has already loaded.
B. A Structured Streaming job with `foreachBatch`, triggered from the warehouse with a SQL alert.
C. `INSERT INTO ... SELECT * FROM read_files(...)` scheduled every night on the same folder.
D. `CREATE OR REPLACE TABLE ... AS SELECT` from the whole folder every night.

Q4 Kafka messages are Avro with a Confluent Schema Registry. Bronze must allow parsing errors to be replayed later. Which design is correct?

A. Decode with `from_avro` in bronze and drop the binary value, since the registry keeps every schema version.
B. Bronze stores key, binary value, topic, partition, offset and timestamp; silver decodes with `from_avro`.
C. Use COPY INTO with `FILEFORMAT = AVRO` pointed at the broker's bootstrap servers.
D. Write Kafka messages to CSV files with a connector first and ingest them with Auto Loader.

Q5 Which read configuration ingests XML files where each `<patient>` element is one row?

A. `spark.read.format("json").option("multiLine", "true").load(path)`
B. `spark.read.format("xml").load(path)` with `rowTag` inferred from the root element
C. `spark.read.format("text").option("wholetext", "true").load(path)` then regex per field
D. `spark.read.format("xml").option("rowTag", "patient").load(path)`

Q6 A pipeline runs hourly on a job cluster, must process every file that arrived since the last run exactly once, then stop. Which trigger?

A `trigger(processingTime="1 hour")`
B `trigger(availableNow=True)`
C `trigger(continuous="1 second")`
D No trigger, relying on the default micro-batch schedule

Q7 A CSV feed sometimes contains malformed lines. The team wants good rows loaded and bad lines kept for inspection, without failing the read. Which option set?

A `mode="DROPMALFORMED"` and a count of rows before and after the read.
B `mode="FAILFAST"` with a retry policy on the job task that reads the file.
C `inferSchema="false"` so every column is a string and nothing can be malformed.
D `mode="PERMISSIVE"` with a corrupt-record column (or `badRecordsPath`).

Q8 A 50 TB Parquet directory must become a Delta table with minimal data movement. What is the most efficient command?

A `CONVERT TO DELTA parquet.`/path``, writing a log over the existing files
B `CREATE TABLE t USING DELTA AS SELECT * FROM parquet.`/path``
C `COPY INTO t FROM '/path' FILEFORMAT = PARQUET` into an empty Delta table
D An Auto Loader stream in availableNow mode writing into a new Delta table

Q9 A streaming table reads bronze. An operator ran `DELETE FROM bronze WHERE ingest_date < '2024-01-01'` for retention and the stream now fails. Which option lets it continue while ignoring these deletes?

A Set `skipChangeCommits` on the streaming read (or full-refresh the streaming table).
B Set `startingOffsets` to latest so the stream resumes after the delete commit.
C Delete the checkpoint directory so the stream rebuilds its state from the current table.
D Set `mergeSchema` on the write so the target accepts the new shape of the source.

Q10 A hospital must give a research partner that does not use Databricks read access to a de-identified gold table, refreshed continuously, without copying files. What is correct?

A Create a Databricks-to-Databricks recipient using the partner's email address as identity.
B Export the table nightly to CSV in a bucket the partner can read with a signed URL.
C Grant SELECT on the table to an account group that contains the partner's users.
D Add the table to a share, create an open-sharing recipient, and send the activation link.

Q11 A recipient in another Databricks account wants a streaming table on top of a shared Delta table. The stream fails immediately. What must the provider change?

A. Convert the recipient to open sharing so that it can read change files directly.
B. Grant MODIFY on the share so that the recipient can create a checkpoint on it.
C. Add the table to the share `WITH HISTORY` so its versions are visible to them.
D. Enable deletion vectors on the shared table so that row-level changes are visible.

Q12 Two business units in the same Unity Catalog metastore need to read each other's gold tables. What is the recommended mechanism?

A. Lakehouse Federation from one workspace to the other using a JDBC connection.
B. Ordinary Unity Catalog grants on the catalogs, schemas or tables.
C. Nightly DEEP CLONE of each gold table into the other unit's catalog.
D. Delta Sharing between the two workspaces, so each side controls what the other sees.

Q13 A provider wants each partner to see only its own country's rows in a single shared view, without creating a view per partner. Which feature enables this?

A. A row filter defined on the recipient side after each partner mounts the shared catalog.
B. Sharing the table `WITH HISTORY` and partition pruning by the country column.
C. A dynamic view filtering on `current_recipient('country')`.
D. One metastore per partner, each receiving a copy of the view filtered to its country.

Q14 Analysts need to join a 2 GB customer table in an on-prem PostgreSQL database with lakehouse orders a few times a day, with UC permissions enforced. What is the lowest-effort approach?

A. A declarative pipeline with a JDBC source that ingests the table into bronze every hour.
B. Share the PostgreSQL password with analysts and let them call `spark.read.jdbc` in notebooks.
C. Export the table to Parquet weekly and load it with Auto Loader into a managed table.
D. A UC connection to PostgreSQL plus a foreign catalog, with SELECT granted to analysts.

Q15 A federated query to Snowflake pulls all 500 million rows before filtering in Databricks. What is the most likely cause?

A. Photon is disabled on the warehouse, which prevents filters from being sent to the source.
B. Lakehouse Federation never pushes predicates down, so every federated query reads full tables.
C. The filter uses an expression that the Snowflake connector cannot push down to the source.
D. The foreign catalog was created without `WITH HISTORY`, which disables pushdown on reads.

Q16 An open-sharing recipient's token has leaked. Security wants the credential invalidated immediately. Which action?

A. Revoke SELECT on the underlying tables from the provider's own analyst groups.
B. Drop the share so that no recipient can access the tables until a new share is created.
C. `ALTER RECIPIENT ... ROTATE TOKEN`, expiring the old token immediately (or drop it).
D. Rotate the cloud storage account keys used by the metastore's managed location.

Q17 A hospital must continuously replicate inserts, updates and deletes from an operational SQL Server database into the lakehouse with minimal custom code. Which approach fits?

A. A Lakeflow Connect SQL Server connector: connection, ingestion gateway, ingestion pipeline.
B. A Lakehouse Federation foreign catalog over SQL Server that downstream jobs query directly.
C. Hourly CSV extracts written by the DBA to a volume and ingested with Auto Loader.
D. A nightly JDBC read of each full table that overwrites the target in the lakehouse.

Q18 A Lakeflow Connect PostgreSQL gateway was stopped for two weeks; the replication slot was dropped and WAL recycled. What should the team expect when it restarts?

A. Converting the destination tables to Iceberg recovers the missing changes from snapshots.
B. A change gap that cannot be recovered incrementally: full-refresh the affected tables.
C. The pipeline resumes where it stopped, because staging keeps all changes for thirty days.
D. Only deletes are lost; inserts and updates are recovered from the table snapshot.

Q19 A gold table must be written by Databricks pipelines and updated by an external Flink job, with Unity Catalog governing it and running compaction. Which table type?

A. A Delta Sharing recipient configured for the Flink cluster with write permission.
B. A managed Delta table with deletion vectors, with Flink writing Delta files directly.
C. An external Parquet directory both engines write to, registered in UC as an external table.
D. A Unity Catalog managed Iceberg table, accessed by Flink through the Iceberg REST catalog.

Q20 Two health systems want to measure patient overlap and joint outcomes, but neither may expose patient-level records to the other. Which capability is designed for this?

A. A shallow clone of each table exchanged through a shared cloud storage bucket.
B. A Clean Room where only approved notebooks run and only aggregate outputs leave.
C. Lakehouse Federation between the two workspaces with row filters on each side.
D. OpenSharing both patient tables with each other using D2D recipients.

Domain 2 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Answer C

Notification and file-event modes replace listing with event-driven discovery.

A COPY INTO also lists the directory each run.

B That sets batch size, not how files are discovered.

D Affects only the first run.

Q2 Answer A

`addNewColumns` fails once by design, updates the schema location, and succeeds on restart.

B Unnecessary and causes full reprocessing.

C JSON evolution is fully supported.

D `none` ignores new columns rather than adding them.

Q3 Answer A

COPY INTO tracks which files were loaded and skips them on later runs: idempotent SQL batch loading.

B Overkill and not SQL-only.

C Re-reads all files and duplicates rows on re-run.

D Rewrites everything each night; history of loads is lost.

Q4 Answer B

Raw bytes plus Kafka metadata in bronze make silver parsing re-runnable long after Kafka retention expires.

A The raw payload is gone, so fixes cannot be replayed.

C COPY INTO reads files, not brokers.

D An unnecessary, lossy hop.

Q5 Answer D

The native XML reader requires `rowTag` to know which element becomes a row.

A JSON readers cannot parse XML.

B `rowTag` must be specified.

C Gives one string per file, not rows.

Q6 Answer B

availableNow processes everything available with checkpointing, then terminates.

A Keeps the query and cluster running between batches.

C Continuous processing never stops and is not for this.

D The default runs forever.

Q7 Answer D

PERMISSIVE keeps malformed rows in a corrupt-record column (or writes them to badRecordsPath) while loading the rest.

A Silently discards the bad lines.

B Aborts on the first bad line.

C Structural problems (quotes, column counts) still occur.

Q8 CONVERT TO DELTA writes a transaction log over the existing files without rewriting them.

Answer A

B Rewrites all 50 TB.

C Copies every file.

D Also rewrites all data.

Q9 skipChangeCommits tells the Delta source to ignore commits that changed existing rows.

Answer A

B `startingOffsets` is a Kafka option and applies only to new queries.

C Reprocesses everything and loses progress.

D The problem is a non-append commit, not schema.

Q10 The open protocol serves non-Databricks consumers with live access and no copies.

Answer D

A D2D needs the recipient's metastore ID; the partner has no Databricks.

B A copy, and not continuously fresh.

C UC grants do not extend to another organisation.

Q11 By default only the latest snapshot is shared; streaming and CDF need history.

Answer C

A Open sharing does not add streaming ability.

B Shares are read-only; MODIFY is not a share privilege for recipients.

D Unrelated to streaming from a share.

Q12 Within one metastore, GRANTs are the mechanism; sharing adds overhead for nothing.

Answer B

A Federation is for external systems.

C Creates copies and staleness.

D Unnecessary inside one metastore.

Q13 Dynamic views can read recipient properties and filter rows per recipient.

Answer C

A The provider must enforce it; recipients control their own side.

B History is about versions, not per-recipient filtering.

D Heavy duplication for something one view can do.

Q14 Federation exposes the table as a governed foreign catalog, with predicate pushdown and no ingestion.

Answer D

A More moving parts than needed for occasional queries.

B Credential sprawl and no governance.

C Stale and more work.

Q15 Pushdown depends on connector support for the expression; rewrite the predicate or materialise the data.

Answer C

A Pushdown does not depend on Photon.

B Supported predicates are pushed down.

D `WITH HISTORY` is a sharing concept.

Q16 Rotating with immediate expiry invalidates only the compromised credential.

Answer C

A Recipient access does not depend on those grants.

B Breaks every recipient, not only the leaked one.

D Recipients authenticate with tokens, not storage keys.

Q17 Managed database connectors capture changes (including deletes) and apply them to streaming tables with almost no code.

Answer A

B Federation queries live; it does not replicate.

C No deletes, extra operations work.

D Not continuous; heavy on the source.

Q18 The gateway must run continuously; once the source log is gone, only a full refresh restores correctness.

Answer B

A Table format cannot recreate lost source changes.

C Staging only holds what the gateway captured.

D All changes in the gap are lost.

Q19 Managed Iceberg lets external engines read and write through the REST catalog while UC governs and maintains it.

Answer D

A Sharing is read-only.

B External Iceberg engines cannot write it through UC.

C Parquet has no transactions; concurrent writers corrupt it.

Q20 Clean Rooms run approved code on combined data without either party seeing the other's raw rows.

Answer B

A Copies raw data to the other party.

C Still exposes records to queries.

D Exposes rows to the other party.

Notes · Domain 2

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 3

3 Data Manipulation

Windows, joins and aggregations at scale; semi-structured data with VARIANT; AI functions inside pipelines; quarantining bad data.

12% of the exam · about 7 questions

What you will be able to do

- Write window functions that are correct and do not collapse onto one partition
- Explain which join strategy Spark picks and how to change it
- Deduplicate correctly in batch, MERGE and streaming
- Model changing JSON with VARIANT and query it safely
- Use AI functions in a pipeline without re-scoring or failing everything
- Build a zero-data-loss quarantine with expectations

Matching free course

- Advanced Techniques with Spark Declarative Pipelines (quarantine, expectations)
- Databricks Performance Optimization (joins, skew)

Study checklist

- [ ] Read 3.1–3.6
- [ ] Lab 3: VARIANT + quarantine
- [ ] Practice questions (14)

Official objectives: advanced transformations (window functions, joins, aggregations) in Spark SQL and PySpark on large data; model and query semi-structured data with VARIANT (`parse_json` , `variant_get`); AI functions including `ai_query` for enrichment and classification in pipelines; expectations to quarantine, drop or fail bad records.

3.1 Window functions

In plain words

A window function computes something for each row by looking at a group of related rows (the window), without collapsing the rows the way `GROUP BY` does. "Rank this claim within its member", "running total per day", "previous visit date" are all windows.

```
SELECT member_id, claim_id, service_date, paid_amount,
  ROW_NUMBER() OVER (PARTITION BY member_id ORDER BY service_date DESC, claim_id DESC) AS rn,
  SUM(paid_amount) OVER (PARTITION BY member_id ORDER BY service_date
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)             AS running_paid,
  LAG(service_date) OVER (PARTITION BY member_id ORDER BY service_date)                 AS prev_visit,
  AVG(paid_amount) OVER (PARTITION BY member_id ORDER BY service_date
                         RANGE BETWEEN INTERVAL 30 DAYS PRECEDING AND CURRENT ROW)     AS avg_30d
FROM prod.silver.claims;
```

Choice	Meaning
`ROW_NUMBER`	Always unique: use for "exactly one latest row" (add a tie-breaker to ORDER BY)
`RANK` / `DENSE_RANK`	Ties share a rank (RANK leaves gaps): can return several "first" rows

Choice	Meaning
`ROWS BETWEEN`	Counts physical rows (last 7 rows)
`RANGE BETWEEN`	Uses values of the ORDER BY column (last 7 days), correct for irregular timestamps

> **The window without PARTITION BY**
> `SUM(x) OVER (ORDER BY ts)` over billions of rows moves every row into **one partition** on one task. Spark even logs a warning. Always partition by a key where the logic allows it.

3.2 Joins at scale

Revisit F2 for strategies. The exam's recurring cases:

- **Small dimension, huge fact, sort-merge join chosen:** statistics missing or threshold too low. Broadcast hint (`/*+ BROADCAST(d) */`) or run `ANALYZE TABLE ... COMPUTE STATISTICS` ; AQE may also switch at runtime.
- **One task processes 40 GB:** skew. AQE skew join (`spark.sql.adaptive.skewJoin.enabled`), salting the hot key, or broadcasting the other side.
- **`OR` in the join condition** or inequality joins: broadcast nested loop. Rewrite `OR` as two equi-joins with `UNION` ; for interval joins (`ts BETWEEN start AND end`) use the **range join** hint with a bin size, or add a coarse bucket equi-key.
- **Filter before joining**, aggregate before joining, and select only needed columns.

3.3 Aggregations and deduplication

Situation	Correct tool
Exact duplicates in a batch	`dropDuplicates()` / `SELECT DISTINCT`
Latest version per key	`ROW_NUMBER() ... = 1` with deterministic tie-breaker, or `max_by(struct, ts)`
MERGE fails: "multiple source rows matched"	Deduplicate the source to one row per key **before** MERGE
Streaming duplicates with bounded state	`withWatermark` + `dropDuplicatesWithinWatermark(["event_id"])`
Arrays and maps	Higher-order functions: `transform` , `filter` , `exists` , `aggregate` ; `explode` only when you need rows

```
MERGE INTO prod.silver.members t
USING (SELECT * EXCEPT (rn) FROM (
        SELECT *, ROW_NUMBER() OVER (PARTITION BY member_id ORDER BY updated_at DESC) rn
        FROM prod.bronze.member_updates) WHERE rn = 1) s
ON t.member_id = s.member_id
WHEN MATCHED AND s.updated_at > t.updated_at THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE AND t.is_active THEN UPDATE SET is_active = false;
```

3.4 Semi-structured data with VARIANT

> **In plain words**
>
> Device and EHR payloads change shape from version to version. Parsing them into a fixed STRUCT breaks or silently drops new fields; keeping them as strings makes every query re-parse text. **VARIANT** stores JSON in an efficient binary form that keeps any shape, and lets you reach into it quickly with path syntax.

```
CREATE TABLE prod.bronze.vitals (device_id STRING, payload VARIANT, loaded_at TIMESTAMP);
INSERT INTO prod.bronze.vitals SELECT device_id, parse_json(raw_json), now() FROM staging;

SELECT payload:patient.mrn::STRING                       AS mrn,
       variant_get(payload, '$.vitals.hr', 'INT')         AS hr,        -- error on bad cast
       try_variant_get(payload, '$.vitals.spo2', 'DOUBLE') AS spo2,      -- NULL on bad cast
       schema_of_variant(payload)                         AS shape
FROM prod.bronze.vitals;

SELECT v.key, v.value FROM prod.bronze.vitals, LATERAL variant_explode(payload:labs) AS v;
```

Function	Use
`parse_json` / `try_parse_json`	String to VARIANT (try_ returns NULL on invalid JSON)
`col:path::TYPE`	Path access with cast
`variant_get` / `try_variant_get`	Typed extraction by JSON path; try_ returns NULL instead of error
`schema_of_variant`, `schema_of_variant_agg`	Discover shapes
`variant_explode`	One row per array element or object field

Design pattern: keep the VARIANT in bronze/silver for flexibility, and **shred hot, stable fields into typed columns** in silver so they get statistics, clustering and constraints.

3.5 AI functions in pipelines

Function	Does
`ai_query(endpoint, prompt, ...)`	Calls any model serving endpoint (foundation or custom); `returnType`, `failOnError => false` returns errors in a struct instead of failing
`ai_classify(text, ARRAY(labels))`	Pick one label
`ai_extract(text, ARRAY(fields))`	Pull named entities
`ai_mask(text, ARRAY('person','address'))`	Redact entity types
`ai_summarize`, `ai_translate`, `ai_analyze_sentiment`, `ai_parse_document`	Task-specific helpers

Cost and correctness with LLM calls

Put AI functions in a **streaming table** so each row is scored once, not in a materialized view that might recompute and re-score everything. Use `failOnError => false` so one bad response does not fail the update, validate outputs with an expectation (for example, the label is in the allowed list), and log model name/version as columns. For PHI, use endpoints governed in your workspace, never external APIs from UDFs.

3.6 Quarantine: keep bad rows, with a reason

Dropping rows loses evidence; failing stops the pipeline. The **quarantine** pattern routes invalid rows to their own table so nothing is lost and the valid path keeps flowing.

```
CREATE OR REFRESH STREAMING TABLE silver_claims (
  CONSTRAINT valid EXPECT (claim_id IS NOT NULL AND paid_amount >= 0) ON VIOLATION DROP ROW)
AS SELECT * FROM STREAM bronze_claims;

CREATE OR REFRESH STREAMING TABLE quarantine_claims (
  CONSTRAINT invalid EXPECT (NOT (claim_id IS NOT NULL AND paid_amount >= 0)) ON VIOLATION DROP ROW)
AS SELECT *, 'claim_id null or negative amount' AS failed_rule, current_timestamp() AS quarantined_at
FROM STREAM bronze_claims;
```

Alternatives: one table with an `is_valid` flag column computed from the rules (then split downstream), or using `_rescued_data IS NOT NULL` as a quarantine condition for schema problems. The event log keeps counts, never the rows themselves.

Remember

- ROW_NUMBER + tie-breaker for one row per key; RANGE frames for time-based windows; always PARTITION BY.
- Broadcast small sides; fix skew with AQE/salting; rewrite OR joins; range-join hint for intervals.
- Deduplicate the source before MERGE; watermark + `dropDuplicatesWithinWatermark` in streams.
- VARIANT for changing JSON; `try_variant_get` returns NULL on bad casts; shred hot fields.
- AI functions in streaming tables, `failOnError => false`, validate with expectations; `ai_mask` for redaction.
- Quarantine = inverse expectation into a second table; the event log has counts, not rows.

Lab 3 · Messy JSON, nothing lost (45 min)

1. Create a table with a VARIANT column from 5 JSON strings of different shapes, including one where `hr` is `"n/a"`.
2. Compare `payload:vitals.hr::INT`, `variant_get` and `try_variant_get` on the bad row.
3. Build the two-table quarantine pattern in a pipeline and confirm the bad rows appear with `failed_rule`.
4. Run `ai_mask('Patient John Smith lives at 12 Oak St', array('person','address'))` if AI functions are available in your workspace.

Domain 3 practice questions

Answer first, check later. Score: _____ / 14 · Date: _____ · Retake date: _____

Q1 A query computes `SUM(amount) OVER (ORDER BY order_ts)` over 3 billion rows and runs on a single task for hours. What is the root cause?

A. SUM is not optimised in window functions, so Spark falls back to a row-at-a-time implementation.
B. Photon is disabled, and Photon is required to run window functions in parallel.
C. The window has no `PARTITION BY`, so every row goes to one partition.
D. The frame is missing: without `ROWS BETWEEN` Spark cannot split the work across tasks.

Q2 Which expression returns exactly one row per `patient_id`, the most recent by `updated_at`, even when two rows share the same timestamp?

A. `RANK() OVER (PARTITION BY patient_id ORDER BY updated_at DESC) = 1`
B. `MAX(updated_at) OVER (PARTITION BY patient_id)` compared with `updated_at`
C. `ROW_NUMBER() OVER (PARTITION BY patient_id ORDER BY updated_at DESC, ingest_id DESC) = 1`
D. `DENSE_RANK() OVER (PARTITION BY patient_id ORDER BY updated_at DESC, ingest_id) = 1` without a filter on ties

Q3 A join between a 5 TB fact table and a 4 MB reference table uses a sort-merge join and shuffles the fact table. What is the most likely reason and fix?

A. Missing statistics or a lowered broadcast threshold; add a BROADCAST hint or compute statistics.
B. Too few shuffle partitions; raise `spark.sql.shuffle.partitions` so the fact side is split more finely.
C. Delta tables always use sort-merge joins, so the only option is a larger cluster.
D. The reference table is stored as Delta; convert it to CSV so Spark can estimate its size exactly.

Q4 In the Spark UI, one task in a join stage processes 40 GB while the median task processes 200 MB. Which combined mitigation fits?

A. Disable AQE so the partitioning stays stable, and give executors more memory to absorb the big task.
B. Call `coalesce(1)` before the join so the uneven partitions are combined into a single one.
C. Cache both input tables so the hot key is read from memory during the join.
D. Keep AQE skew-join handling on; salt the hot key or broadcast the smaller side.

Q5 A streaming deduplication uses `dropDuplicates(["event_id"])` and its state grows without bound. Which change bounds state while still removing duplicates that arrive within an hour?

- A Replace it with `distinct()`, which forgets keys once each micro-batch is committed.
- B Switch to complete output mode so older keys are flushed to the sink and removed.
- C `withWatermark("event_ts", "1 hour")` plus `dropDuplicatesWithinWatermark(["event_id"])`.
- D Increase driver memory, since streaming deduplication state is held on the driver.

Q6 An SDP silver table drops rows failing `amount > 0`. Auditors require dropped rows to be retained with the reason. What is the simplest compliant design?

- A Change the expectation to FAIL UPDATE so that invalid rows can never be lost.
- B A second streaming table from the same source with the inverse rule and a `failed_rule` column.
- C Reconstruct the dropped rows later from the pipeline event log, which stores every failed record.
- D Remove the expectation from silver and clean the data at the gold layer for each report.

Q7 Which statement about the `_rescued_data` column from Auto Loader is true?

- A It holds, as JSON, values that did not fit the schema, e.g. new fields or type mismatches.
- B It is written automatically to `badRecordsPath`, so it is empty in the target table.
- C It contains the rows that failed an expectation in a declarative pipeline.
- D It exists only for CSV sources and is populated when a line has the wrong number of columns.

Q8 `MERGE INTO dim_customer USING updates ON dim.id = updates.id ...` fails because multiple source rows matched one target row. What is the fix?

- A Enable deletion vectors on the target so several updates to one row can be applied in order.
- B Add `WHEN NOT MATCHED BY SOURCE THEN DELETE` so extra matches are removed during the merge.
- C Deduplicate `updates` to one row per `id` (latest by timestamp) before the MERGE.
- D Change the condition to `ON dim.id = updates.id AND dim.updated_at < updates.updated_at`.

Q9 A 7-day moving average per device is needed; readings arrive at irregular intervals. Which frame is correct?

- A `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` with a filter on the last week
- B No explicit frame, since the default frame of an ordered window is the previous seven days
- C `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`, ordered by the timestamp
- D `RANGE BETWEEN INTERVAL 7 DAYS PRECEDING AND CURRENT ROW`, ordered by the timestamp

Q10 A non-equi join `ON e.ts BETWEEN s.start_ts AND s.end_ts` between two large tables takes hours. Which technique is most effective?

- A Raise the broadcast threshold to 10 GB so that one side is always broadcast to executors.
- B The range join hint with a suitable bin size, or an equi-join on a coarse time bucket.
- C Implement the interval check in a Python UDF and apply it after a cross join.
- D Replace the join with `UNION ALL` and a window function over the combined rows.

Q11 Bronze stores device telemetry as a JSON string whose keys change between firmware versions. `from_json` with a fixed schema drops new fields and regex queries are slow. Which redesign fits best?

- A Store the payload as VARIANT and shred hot fields into typed silver columns.
- B Keep the string column and cluster the table by it so that regex queries skip more files.
- C Convert the payloads to XML so that the schema can be declared with an XSD per firmware version.
- D Re-infer a STRUCT schema on every batch and overwrite the table schema to include new keys.

Q12 Which expression returns NULL rather than raising an error when `payload:vitals.hr` contains a non-numeric value?

- A `payload:vitals.hr::INT`, because path casts are always lenient
- B `try_variant_get(payload, '$.vitals.hr', 'INT')`
- C `variant_get(payload, '$.vitals.hr', 'INT')`
- D `schema_of_variant(payload:vitals.hr)` filtered to numeric types

Q13 A pipeline must tag each new support ticket with a category using a hosted foundation model, scoring each ticket once and never failing the update because of one bad response. Which design is best?

- A A materialized view that calls `ai_query` so the categories always reflect the latest model version.
- B A Python UDF that calls the model endpoint with `requests` from every executor in parallel.
- C A nightly notebook that exports new tickets to CSV so a team can label them before loading.
- D A streaming table with `ai_classify` or `ai_query(..., failOnError => false)` and a label expectation.

Q14 Clinical notes must be stored in silver with names and addresses redacted before analysts can query them. Which built-in function does this in SQL?

- A `sha2(note_text, 256)`, which irreversibly hides every identifier in the note
- B `ai_summarize(note_text, 50)`, since summaries omit personal details
- C `regexp_replace(note_text, '[A-Z][a-z]+', '***')` applied to capitalised words
- D `ai_mask(note_text, ARRAY('person', 'address'))` in the silver transformation

Domain 3 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Answer C

Without PARTITION BY, all rows must be ordered together on one task. Partition by a key, or compute cumulative values per key.

A SUM is fully supported in windows; the problem is partitioning.

B Parallelism depends on partitioning, not Photon.

D The default frame does not change parallelism.

Q2 Answer C

ROW_NUMBER never ties; a deterministic tie-breaker makes the choice stable.

A Ties both get rank 1.

B Returns every row sharing the max timestamp.

D DENSE_RANK can still tie when the full ordering is not unique; ROW_NUMBER is the guarantee.

Q3 Answer A

A 4 MB side should be broadcast so the large side is not shuffled at all.

B More partitions still shuffle 5 TB.

C Delta tables broadcast like any other.

D Format change is irrelevant and worse.

Q4 Answer D

Skew is fixed by splitting or avoiding the hot partition: AQE splits it; salting spreads it; broadcasting removes the shuffle.

A Disabling AQE removes the skew handling.

B Puts all data on one task: far worse.

C Caching does not rebalance keys.

Q5 Answer C

The watermark lets Spark evict keys older than the lateness window.

A distinct is equally unbounded in a stream.

B Complete mode keeps all state.

D State lives in executors' state stores; memory does not bound it.

Q6 Answer B

The inverse-expectation quarantine keeps every bad row with context while silver stays clean.

A Stops the pipeline instead of retaining rows.

C The event log stores counts, not rows.

D Pushes bad data downstream and loses auditability.

Q7 Answer A

The rescued-data column captures schema-mismatched data per row, which makes it a natural quarantine filter.

B It is a column in the table; badRecordsPath is a different option.

C Expectations are a separate mechanism.

D Available for JSON, CSV, Avro, Parquet and more.

Q8 MERGE requires at most one source row per target row; resolve duplicates first.

Answer C

A DVs do not change MERGE's one-match rule.

B That handles target rows missing from the source, not duplicates.

D Several source rows can still match the same target row.

Q9 RANGE frames use the ORDER BY values, so irregular timing is handled correctly.

Answer D

A A running total from the start, not a moving window.

B The default frame is unbounded preceding to current row.

C Counts 7 readings, not 7 days.

Q10 Range joins bin both sides so only overlapping bins are compared, avoiding the nested loop.

Answer B

A A large table cannot be broadcast safely.

C Even slower than the nested loop.

D Changes semantics and does not solve the cost.

Q11 VARIANT keeps any shape efficiently with fast path access; typed columns restore statistics for common filters.

Answer A

B Clustering on a JSON blob does nothing for field access.

C Adds complexity and does not solve flexibility.

D Constant schema rewrites break readers.

Q12 try_ variants return NULL on cast or path errors.

Answer B

A The `::` cast raises on invalid values.

C Raises an error on an invalid cast.

D Returns a type description, not the value.

Q13 Streaming tables score only new rows; non-failing calls plus expectations isolate bad responses.

Answer D

A May re-score every row on refresh, multiplying cost.

B Bypasses governance, rate limiting and error handling.

C Manual and not model inference.

Q14 ai_mask redacts the requested entity types while keeping the rest of the text.

Answer D

A Destroys the whole note, not just the identifiers.

B Summaries can still contain names.

C Crude; misses and over-masks.

Notes · Domain 3

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 4

4 Monitoring and Alerting

System tables for history, event logs for pipelines, APIs and the SDK for automation, Lakehouse alerts for signals.

10% of the exam · about 6 questions

What you will be able to do

- Answer cost, audit and job-health questions with system tables
- Automate monitoring and repair with REST, CLI and the Databricks SDK
- Read the pipeline event log for errors, data quality and refresh decisions
- Build alerts for freshness, quality, cost, warehouse health, audit events and job branching

Matching free course

- Deploy Workloads with Lakeflow Jobs (Associate refresher)
- Monitoring and Optimizing Apache Spark Workloads (Spark Developer pathway)

Study checklist

- [ ] Read 4.1–4.5
- [ ] Lab 4: cost-per-job query + freshness alert
- [ ] Practice questions (14)

Official objectives: system tables (billing, compute, access, lakeflow) for cost, auditing and workload monitoring; REST/CLI and SDK for jobs and pipelines; pipeline event logs for health and quality; Lakehouse alerts for business metrics, data quality and anomalies, usage and cost, warehouse/query health, audit and security, AI agent quality and job branching; Jobs UI and API.

In plain words

Monitoring answers three questions: what happened (history), what is happening (current state), and who should be told (alerts). On Databricks, history is in **system tables**, current detail is in the **UIs, event logs and APIs**, and telling people is done with **alerts, job notifications and health rules**.

4.1 System tables

Account-wide Delta tables in the `system` catalog, governed by Unity Catalog (an admin grants `USE SCHEMA` and `SELECT`; some schemas must be enabled). Retention is finite, usually about a year.

Table	Answers
`system.billing.usage` + `system.billing.list_prices`	DBUs and dollars per workspace, SKU, job, pipeline, warehouse, custom tag
`system.access.audit`	Who did what: logins, grants, table access, sharing events, policy changes
`system.access.table_lineage`, `column_lineage`	Which workloads read and write which tables

Table	Answers
`system.compute.clusters`, `node_timeline`, `warehouse_events`	Cluster configs over time, per-node utilisation, warehouse scaling
`system.lakeflow.jobs`, `job_tasks`, `job_run_timeline`, `job_task_run_timeline`, `pipelines`	Every job/task run with result, duration, trigger
`system.query.history`	SQL warehouse queries: text, duration, user, rows
`system.storage.predictive_optimization_operations_history`	What automatic maintenance ran and cost
`system.information_schema`	Metadata, grants and tags across catalogs

```sql
-- Dollar cost per job, last 30 days, by cost-center tag
SELECT u.usage_metadata.job_id, u.custom_tags['cost_center'] AS cost_center,
       SUM(u.usage_quantity * p.pricing.default) AS usd
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON u.sku_name = p.sku_name AND u.cloud = p.cloud
 AND u.usage_start_time >= p.price_start_time
 AND (p.price_end_time IS NULL OR u.usage_start_time < p.price_end_time)
WHERE u.usage_date >= current_date() - 30 AND u.usage_metadata.job_id IS NOT NULL
GROUP BY ALL ORDER BY usd DESC;

-- Who read the patients table in the last 90 days
SELECT event_time, user_identity.email, action_name
FROM system.access.audit
WHERE request_params.full_name_arg = 'prod.clinical.patients' AND event_date >= current_date() - 90;
```

4.2 UIs: Spark UI, Query Profile, Jobs UI

- **Query Profile** (warehouses, serverless): operator tree with time, rows, bytes, spill, **files read vs pruned**. Low pruning → layout problem (Ch. 5).
- **Spark UI** (classic): stages and task distributions (skew: max ≫ median), shuffle read/write, spill, GC time, SQL tab with physical plan.
- **Jobs UI**: run matrix, task durations over time, repair, lineage of task values.

4.3 REST API, CLI and the SDK

```
GET  /api/2.2/jobs/runs/list?job_id=123&active_only=true     runs (paginated)
GET  /api/2.2/jobs/runs/get?run_id=456                        state + tasks[] (each with its own run_id)
GET  /api/2.2/jobs/runs/get-output?run_id=<task run id>       task error / notebook exit value
POST /api/2.2/jobs/runs/repair  {"run_id":456,"rerun_all_failed_tasks":true,"job_parameters":{...}}
GET  /api/2.0/pipelines/{id}/events                           pipeline event log
```

`get-output` works on **task** run ids, not the parent run of a multi-task job. Run states: `life_cycle_state` (PENDING, RUNNING, TERMINATED, SKIPPED, INTERNAL_ERROR) and `result_state` (SUCCESS, FAILED, TIMEDOUT, CANCELED).

The **Databricks SDK** wraps the same API with typed objects, automatic pagination and unified authentication. It is the expected tool for monitoring code; authenticate as a **service principal with OAuth**

(M2M), not a personal token.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import RunResultState
w = WorkspaceClient()                                  # env vars or a profile; OAuth M2M in CI
for run in w.jobs.list_runs(job_id=123, expand_tasks=True):      # paginates for you
    if run.state and run.state.result_state == RunResultState.FAILED:
        for t in run.tasks or []:
            if t.state.result_state == RunResultState.FAILED:
                print(run.run_id, t.task_key, w.jobs.get_run_output(run_id=t.run_id).error)
        w.jobs.repair_run(run_id=run.run_id, rerun_all_failed_tasks=True,
                          job_parameters={"run_date": "2026-10-01"})
```

4.4 The pipeline event log

Every declarative pipeline writes a structured event log (a Delta table), readable with `event_log(TABLE(...))` or `event_log('<pipeline_id>')`, or published to Unity Catalog in pipeline settings.

Event type	Contains
`flow_progress`	Rows processed, backlog, **expectation pass/fail counts** (`details:flow_progress:data_quality:expectations`)
`planning_information`	Whether an MV refresh was incremental or complete, **and why**
`flow_definition`	Schemas, lineage
`user_action` , `update_progress`	Who started what; update state
`cluster_resources`	Autoscaling decisions

```
SELECT timestamp, message, details:error FROM event_log(TABLE(prod.silver.claims))
WHERE level = 'ERROR' ORDER BY timestamp DESC;
```

4.5 Lakehouse alerts

A Databricks SQL **alert** = a saved query + a condition on a result + a schedule + notification destinations (email, Slack, Teams, PagerDuty, webhook). Most exam questions reduce to "which table do I query?".

Signal	Query or feature
Business metric	A metric view: `SELECT MEASURE(net_revenue) ...`
Freshness / volume	`SELECT max(updated_at) < now() - INTERVAL 2 HOURS AS stale FROM gold.kpi`
Data quality and anomalies	**Data quality monitoring**: anomaly detection on a whole schema (freshness, completeness), profiling metrics; expectation counts from the event log
Cost	`system.billing.usage` × `list_prices` ; budget policies
Warehouse / query health	`system.query.history` , `system.compute.warehouse_events`
Audit and security	`system.access.audit` (grants to broad groups, policy deletions, failed logins)

Signal	Query or feature
AI agent quality	MLflow evaluation and inference tables
Job branching	The **SQL alert task** in a job evaluates an alert; an **If/else** task branches on its result

Job-level signals: **notifications** on start, success, failure, duration warning (with `no_alert_for_skipped_runs` / `no_alert_for_canceled_runs` to cut noise), and **health rules**: `RUN_DURATION_SECONDS` and, for streaming, `STREAMING_BACKLOG_SECONDS` / `_BYTES` / `_RECORDS` / `_FILES`, which fire while the run is still active.

A job that succeeds is not a healthy dataset

Job failure notifications cannot detect an upstream that silently delivers zero rows, or a stream falling behind. Freshness and volume are **data conditions**: they need SQL alerts or data quality monitoring; backlog needs streaming health rules.

What to set up on day one of a new pipeline

A failure notification to the team channel, a backlog or duration health rule, a freshness alert on the gold table the business reads, expectation counts in a dashboard from the event log, and a weekly cost-per-job query from system tables. Five objects, and most 2 a.m. surprises are caught by 9 a.m.

Remember

- Cost: billing.usage × list_prices. Who accessed: access.audit. Runs: lakeflow.*_timeline. Queries: query.history.
- get-output needs task run ids; repair re-runs failed tasks with parameter overrides.
- SDK = typed, paginated, unified auth; service principal + OAuth in automation.
- Event log: flow_progress for expectations, planning_information for MV refresh choices.
- Alerts are queries with conditions; SQL alert task + If/else = branching inside a job.
- Streaming backlog health rules warn during a run; freshness needs a data-level alert.

Lab 4 · Know what you spent and what is stale (40 min)

1. Run the cost-per-job query (Free Edition may limit system tables; if so, run it on `system.lakeflow.job_run_timeline` for durations instead).
2. Create a SQL alert on `SELECT max(loaded_at) < now() - INTERVAL 1 HOUR AS stale FROM learn.bronze.sample` with an email destination.
3. Build a job: SQL alert task → If/else on its result → one notebook task per branch.
4. Use the SDK in a notebook to list the last 5 runs of that job and print each task's state.

Domain 4 practice questions

Answer first, check later. Score: _____ / 14 · Date: _____ · Retake date: _____

Q1 Finance asks for the monthly dollar cost of each job across all workspaces, attributed by a `cost_center` tag. Which source answers this with the least effort?

A. The Spark UI executors tab, which shows task time per job for each cluster.
B. The cloud provider invoice, split by the resource tags that Databricks forwards to VMs.
C. The Jobs UI, exporting each run's duration and multiplying by the cluster's node count.
D. `system.billing.usage` (job id, custom tags) joined to `system.billing.list_prices`.

Q2 A security reviewer needs to know which users queried the `patients` table in the last 90 days. Which table should be queried?

A. `system.lakeflow.jobs`, listing jobs whose tasks mention the table in their parameters.
B. `system.billing.usage`, filtered to SQL warehouse SKUs and the users who ran queries.
C. `system.compute.clusters`, joined to the owners of clusters that mounted the table.
D. `system.access.audit`, optionally with `system.access.table_lineage`.

Q3 In a Query Profile the scan shows "files read: 12,000 / files pruned: 30" for a query filtering on `event_date`. What does it indicate and what is the remedy?

A. Skipping is ineffective for `event_date`; cluster the table by it and run OPTIMIZE.
B. The query needs a broadcast hint so the date filter is applied before the scan.
C. The warehouse is undersized, so it cannot prune files in parallel; move to a larger size.
D. Deletion vectors are preventing file statistics from being used and should be disabled.

Q4 A stage shows one task running 25 minutes (median 40 seconds) with large disk spill. What is the most likely diagnosis?

A. Too many small files in the source, which makes one task open thousands of files.
B. Skew on the join or aggregation key, so one partition exceeds task memory.
C. A missing checkpoint that forces the task to recompute its lineage from the source.
D. Network throttling between executors while the shuffle files are fetched.

Q5 A script must page on-call when any task of a multi-task job fails, including that task's error message. Which API sequence is correct?

A. `runs/list` → `runs/get` to read `tasks[]` → `runs/get-output` for each failed task's run id.
B. A query on `system.query.history` for statements that failed during the job's run window.
C. `jobs/get` for the job definition, which includes the error of the most recent failed task.
D. `runs/get-output` with the parent run id, which returns the output of every task in the run.

Q6 A data steward wants a daily report of how many rows each pipeline expectation dropped. Where is this information?

A. The `_rescued_data` column of the target table, which records rows removed by rules.
B. `system.billing.usage` for the pipeline, which reports rows processed per update.
C. The driver log of the pipeline's cluster, which prints a summary after each flow.
D. `flow_progress` events in the pipeline event log, queried with the `event_log()` function

Q7 A streaming job usually keeps up, but some days its backlog grows for hours before anyone notices. Which configuration gives the earliest automatic warning?

A. A SQL alert on `system.billing.usage` that fires when the job's daily DBUs rise.
B. A success notification on the job so the team notices when it stops reporting success.
C. A job health rule on `STREAMING_BACKLOG_SECONDS` with a notification destination.
D. A longer trigger interval so each micro-batch has more time to catch up on backlog.

Q8 What is the best way to alert when a gold table has not been updated in the last 2 hours?

A. A weekly VACUUM schedule with notifications, since VACUUM fails on stale tables.
B. A failure notification on the job that loads the table, sent to the team's channel.
C. A SQL alert on `max(updated_at) < now() - INTERVAL 2 HOURS`, run every 15 min.
D. A duration health rule on the loading job with a two-hour threshold.

Q9 Which statement about system tables is correct?

A. They keep data indefinitely, so they can replace your own long-term audit archive.
B. Account-wide Delta tables, governed by UC; access must be granted by an admin.
C. They are per-workspace views visible only to workspace admins of that workspace.
D. They are reachable only through the REST API because they live outside the metastore.

Q10 A materialized view took much longer than usual to refresh. Which event log detail explains whether it was fully recomputed, and why?

A. `system.access.audit` entries for the MV, which record each refresh operation type.
B. `cluster_resources` events, which show how many workers were added during the refresh.
C. `planning_information` events, which show the refresh technique and the reason.
D. `user_action` events, which show who started the update and with which settings.

Q11 The team wants failure notifications for a job but not for runs skipped because a previous run was still active. Which setting applies?

A. `max_retries = 0` on every task so that skipped runs are not retried and reported.
B. `notification_settings.no_alert_for_skipped_runs = true`
C. `max_concurrent_runs = 10` so that new runs are never skipped in the first place.
D. Disabling queueing so that overlapping triggers are dropped without creating a run.

Q12 A daily job must publish a gold table only if a freshness check passes, and page on-call otherwise, all inside the job's DAG. Which design is native?

A. A duration health rule on the publish task that cancels it when the data is stale.
B. A SQL alert task, then an If/else task branching on its result to publish or notify.
C. Two separate jobs on the same schedule: one publishes, one checks and pages.
D. A notebook task that polls the alert's REST endpoint in a loop until it reports OK.

Q13 A steward wants every table in `prod.silver` watched for late or missing loads, without writing a query per table, and alerts when one becomes unhealthy. Which feature is designed for this?

A. Anomaly detection (data quality monitoring) on the schema, with alerts on its results.
B. An EXPECT constraint on every table that fails when the latest load is older than a day.
C. Higher retry counts on all loading jobs so late loads recover before anyone notices.
D. A nightly notebook that reads the Spark UI of each job and emails a summary.

Q14 A Python script must list failed runs of several jobs across pages, fetch each failed task's error, and repair runs with a corrected parameter, authenticating as a service principal. Which approach is recommended?

A. `DESCRIBE HISTORY` on each output table to infer which runs failed and rerun them.
B. Hand-written `requests` calls with a personal access token stored in the script's config file.
C. The Databricks SDK (`WorkspaceClient`) with OAuth machine-to-machine credentials.
D. Screen-scraping the Jobs UI with a headless browser logged in as the service principal.

Domain 4 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Answer D

Billing system tables carry job ids and custom tags per usage record; list prices convert DBUs to currency.

A No cost data.

B Misses serverless and DBU pricing detail.

C Manual and not in dollars.

Q2 Answer D

Audit logs record table access events; lineage shows which workloads read the table.

A Covers jobs only, not user queries.

B Billing has no table-access events.

C No access data at all.

Q3 Answer A

Almost nothing was pruned because the files do not hold narrow date ranges; physical layout fixes it.

B Hints affect joins, not scan pruning.

C Pruning is about layout, not warehouse size.

D DVs do not disable statistics.

Q4 Answer B

One long task with spill is the classic skew signature.

A Small files cause many short tasks.

C Checkpoints are a streaming concept here.

D Would slow many tasks, not one.

Q5 Answer A

The parent run's tasks array supplies task run ids; get-output returns each task's error.

B Only covers SQL warehouse queries.

C Job definitions contain no run errors.

D get-output needs task run ids for multi-task jobs.

Q6 Answer D

Expectation pass/fail counts are emitted per flow update in the event log.

A Rescued data is about schema mismatches.

B Billing has no quality metrics.

C Not a structured, queryable source.

Q7 Answer C

Backlog health rules evaluate while the run is active and notify as soon as the threshold is crossed.

A Indirect and delayed.

B A healthy-but-lagging stream never finishes, so it never fires.

D Changes behaviour but does not warn anyone.

Q8 Answer C

Freshness is a data condition, so it needs a query-based alert.

A Unrelated to freshness.

B An upstream that silently produces nothing still succeeds.

D Measures run length, not data freshness.

Q9 System tables are UC-governed, SQL-accessible, account-wide and have finite retention.

Answer B

A Retention is finite, typically about a year.

C They are account-wide and UC-governed.

D They are queried with SQL.

Q10 Planning events say whether a refresh was incremental or complete and which operator forced a full recompute.

Answer C

A Audit records actions, not planning decisions.

B Explains scaling, not the refresh strategy.

D Does not state incremental vs complete.

Q11 Notification settings can mute skipped and cancelled runs.

Answer B

A Retries do not control notifications.

C Changes execution; may cause overlapping runs.

D Changes execution, not alerting.

Q12 The SQL alert task exposes its evaluation as a value that an If/else task can branch on.

Answer B

A Health rules watch run time, not data.

C No dependency between check and publish.

D Custom polling instead of the native task.

Q13 Schema-level anomaly detection checks freshness and completeness for every table automatically.

Answer A

B Per table, and expectations check rows, not freshness.

C Does not detect anything.

D Manual and fragile.

Q14 The SDK paginates, types responses and unifies auth; OAuth service principals are the automation standard.

Answer C

A Table history does not expose task errors or repair.

B Manual pagination and a personal credential.

D Fragile and unsupported.

Notes · Domain 4

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 5

5 Cost and Performance Optimization

Managed tables and Predictive Optimization, liquid clustering, deletion vectors, caching, Change Data Feed and reading query profiles.

15% of the exam · about 9 questions

What you will be able to do

- Explain why managed tables reduce cost and operations
- Choose between partitioning, Z-order and liquid clustering (including CLUSTER BY AUTO)
- Fix small files, skew, spill and poor pruning from a query profile
- Use the disk cache and Change Data Feed to avoid repeated work
- Pick compute that matches the workload's cost profile

Matching free course

- Databricks Performance Optimization
- Monitoring and Optimizing Apache Spark Workloads (Spark Developer pathway)

Study checklist

- [ ] Read 5.1–5.6
- [ ] Lab 5: liquid clustering before/after
- [ ] Practice questions (14)

Official objectives: UC managed tables and Predictive Optimization; liquid clustering, deletion vectors, `CLUSTER BY AUTO`; Delta cache; Change Data Feed for incremental downstream processing; query profile bottlenecks; liquid clustering vs partitioning/Z-order by table size and query pattern.

> **In plain words**
>
> Almost every performance problem is one of four things: reading more data than needed, moving data between machines, doing work in a slow place (Python, the driver), or doing the same work twice. Cost follows performance, plus one more thing: paying for compute that sits idle.

5.1 Managed tables and Predictive Optimization

Managed Unity Catalog tables let Databricks own the physical layer. With **Predictive Optimization** enabled (the default for new accounts), the platform decides when to run `OPTIMIZE` (compaction and clustering), `VACUUM` and `ANALYZE` (statistics), based on how each table is used, and bills it as serverless. It also enables **automatic liquid clustering** (`CLUSTER BY AUTO`), which picks and adapts clustering keys from real query predicates. The exam's framing: managed tables reduce operational overhead and improve performance; nightly maintenance jobs are the old way.

5.2 Physical layout

	Partitioning	Z-order	Liquid clustering
How	Folder per value	`OPTIMIZE ... ZORDER BY` rewrite	`CLUSTER BY (cols)` / `AUTO`
Good for	Very large tables with a low-cardinality lifecycle key (month)	Legacy tables	Default for new tables, any size

	Partitioning	Z-order	Liquid clustering
Cardinality	Low only; high → millions of tiny files	Any	Any
Change keys	Rewrite the table	Re-run full rewrite	`ALTER TABLE t CLUSTER BY (...)` (metadata); later OPTIMIZE applies
Maintenance	None but small files	Full rewrite each time	**Incremental**: only unclustered data is rewritten
Concurrency	Partition-level	File-level	Row-level with deletion vectors

```
CREATE TABLE prod.silver.claims (...) CLUSTER BY (member_id, service_date);
ALTER TABLE prod.silver.claims CLUSTER BY (service_date, plan_code);   -- change keys, no rewrite now
OPTIMIZE prod.silver.claims;           -- incremental clustering of new data
OPTIMIZE prod.silver.claims FULL;      -- recluster everything (after changing keys)
CREATE TABLE prod.gold.events CLUSTER BY AUTO;                          -- let Databricks choose
```

Choosing clustering keys: the columns most often in `WHERE` filters and join keys, up to four, ordered by importance. Z-order and liquid clustering cannot be combined on one table; partitioning and liquid clustering cannot be combined either.

Small files

Frequent small writes (streaming micro-batches, many tiny appends) create thousands of small files and slow every read. Remedies: **optimized writes** (bigger files at write time) and **auto compaction** (merge small files after writes), both automatic on managed tables with Predictive Optimization; `OPTIMIZE` on a schedule otherwise. A shorter trigger interval makes this worse, not better.

Deletion vectors

Merge-on-read deletes/updates (F3): much faster DML and row-level concurrency, at the cost of reads skipping marked rows until files are rewritten. Physical removal requires a rewrite (`OPTIMIZE`, `REORG TABLE ... APPLY (PURGE)`) plus `VACUUM` after retention.

5.3 Compute-side levers

Lever	Use
Job or serverless compute instead of all-purpose	Largest single saving for scheduled work
Serverless standard vs performance mode	Cost vs latency
Photon	SQL/DataFrame-heavy workloads; not Python UDFs
Autoscaling, spot workers with on-demand driver	Variable load, cost
Instance families	Memory-optimised for big joins/aggregations; storage-optimised (local SSD) for disk cache; compute-optimised for CPU-heavy parsing
`spark.sql.shuffle.partitions` / AQE	Right-size shuffle partitions (AQE usually handles it)
`spark.sql.files.maxPartitionBytes`	Input split size (and output file size for narrow pipelines)

5.4 Caching

Cache	What	When
Disk cache (Delta cache)	Copies of remote Parquet files on workers' local SSDs, automatic on supported instance types and warehouses	Repeated reads of the same data (dashboards)
Spark cache (`df.cache()`)	Deserialised DataFrame in executor memory	Reusing an intermediate result within one job
Query result cache (warehouses)	Results of identical queries on unchanged data	BI refreshes

`CACHE SELECT * FROM t WHERE ...` pre-warms the disk cache. The disk cache is not shared across clusters and is invalidated automatically when files change.

5.5 Change Data Feed: do only the changed work

When enabled (`delta.enableChangeDataFeed = true`), each commit also records row-level changes. Downstream consumers read only what changed instead of recomputing from full tables.

```
ALTER TABLE prod.silver.members SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
SELECT * FROM table_changes('prod.silver.members', 1520);  -- _change_type, _commit_version,
_commit_timestamp
```

`_change_type` values: `insert`, `delete`, `update_preimage`, `update_postimage`. Streaming: `readStream.option("readChangeFeed", "true").table(...)`. Typical use: a gold aggregate over a large, frequently updated silver table becomes an incremental AUTO CDC or MERGE flow driven by CDF, instead of a full recompute every run.

5.6 Reading a query profile like an examiner

Observation	Likely cause	Fix
Files read ≫ files pruned is tiny	Layout does not match the filter	Cluster by the filter column
Many tiny files, many short tasks	Small files	OPTIMIZE / auto compaction
Broadcast nested loop join	`OR` or non-equi join	Rewrite as equi-joins / range join
Big shuffle on a small-dimension join	Sort-merge where broadcast fits	Broadcast hint, statistics
Spill, one long task	Skew / memory	AQE skew, salting, bigger memory
Time dominated by cloud reads on repeated queries	No local caching	Disk cache on SSD instances
Python UDF / BatchEvalPython node	Python bottleneck	Native functions / Pandas UDF

The cost review a senior engineer runs every month

Top 20 jobs by dollars from system tables; for each, ask: right compute type? idle time? does it recompute everything when 1% changed (then CDF/streaming)? is the layout matching the filters (pruning ratio)? are there maintenance jobs that Predictive Optimization could replace? Write down savings in dollars; that is how platform work gets prioritised.

Remember

- Managed tables + Predictive Optimization = automatic OPTIMIZE/VACUUM/ANALYZE and auto clustering.
- Liquid clustering: any cardinality, keys changeable, incremental OPTIMIZE; FULL to recluster; AUTO to let Databricks choose.
- Partition only huge tables by a low-cardinality lifecycle key; never by IDs.
- Small files: optimized writes + auto compaction; shorter triggers make it worse.
- Disk cache = local SSD copies for repeated reads; CDF = process only changes.
- VACUUM retention must exceed the slowest reader or stream.

Lab 5 · Measure pruning (45 min)

1. Create `learn.silver.big` with 20 million rows: `id`, `member_id` (100k distinct), `d` (dates over 2 years), unclustered, written in 200 small files (`repartition(200)`).
2. Query `WHERE member_id = 'M123'` and note files read vs pruned in the query profile.
3. `ALTER TABLE learn.silver.big CLUSTER BY (member_id); OPTIMIZE learn.silver.big FULL;` then run the query again and compare.
4. Enable CDF, update 10 rows, and read them back with `table_changes` .

Domain 5 practice questions

Answer first, check later. Score: _____ / 14 · Date: _____ · Retake date: _____

Q1 What is the strongest reason to prefer Unity Catalog managed tables over external tables for performance and cost?

A. Managed tables disable time travel, which reduces the storage cost of old file versions.

B. Predictive Optimization maintains them automatically: compaction, clustering, VACUUM and statistics.

C. External tables cannot use deletion vectors, so their DML is always copy-on-write.

D. Managed tables can only be read by Databricks engines, which lets the optimizer assume a fixed format.

Q2 A 20 TB claims table is filtered mostly by `member_id` (high cardinality) and `service_date`. It is partitioned by `member_id` and has millions of tiny files. What is the best redesign?

A. Keep the partitioning and run OPTIMIZE every hour to compact the files inside each partition.
B. Partition by both `member_id` and `service_date` to prune on either filter.
C. Keep `member_id` partitioning and add Z-order by `service_date` within each partition.
D. Recreate it unpartitioned with `CLUSTER BY (member_id, service_date)`.

Q3 After enabling deletion vectors, a nightly erasure of a few hundred rows takes seconds instead of an hour. Compliance asks whether the rows are physically gone. What is accurate?

A. No, and they can never be physically removed while deletion vectors are enabled on the table.
B. Yes, because deletion vectors rewrite the affected files immediately in the same commit.
C. No: rows are masked until files are rewritten (REORG ... PURGE or OPTIMIZE) and vacuumed.
D. Yes, as soon as the transaction commits, since readers can no longer see the rows.

Q4 A query filtering on `diagnosis_code` reads every file of a table clustered by `member_id`. Why?

A. Diagnosis codes are spread across all files, so no file's min/max excludes the value.
B. String columns never get statistics, so any filter on a code column reads all files.
C. Data skipping requires Photon, which the cluster running this query does not use.
D. Delta supports skipping on one column per query, and the clustering column takes priority.

Q5 Which change lets a table's clustering keys change from `(date)` to `(date, region)` without rewriting existing data immediately?

A. A CTAS into a new table clustered by both columns, then swapping the table names.
B. `ALTER TABLE t PARTITIONED BY (date, region)` followed by an incremental OPTIMIZE.
C. `ALTER TABLE t CLUSTER BY (date, region)`; a later OPTIMIZE applies the layout.
D. `OPTIMIZE t ZORDER BY (date, region)` on the existing liquid-clustered table.

Q6 A gold materialized view aggregates a 3 TB silver table that gets a few thousand updates per hour. The MV fully recomputes every run and misses its SLA. Which redesign lowers latency with the least cost?

A. Refresh the materialized view every minute so each recompute has less new data to handle.
B. Enable Change Data Feed on silver and drive an incremental flow from the changes only.
C. Convert silver to an external table so the MV can read its files without catalog overhead.
D. Z-order the silver table by the aggregation key so the full scan finishes faster.

Q7 Which `_change_type` values does Change Data Feed emit for a row modified by an `UPDATE`?

A. A single `update` row carrying the new values of the changed columns

B. A `delete` of the old row followed by an `insert` of the new row

C. `update_preimage` and `update_postimage`, with the same commit version

D. `modify`, with a map of old and new values for the columns that changed

Q8 A Query Profile shows a broadcast nested loop join dominating runtime. The SQL is `JOIN dim d ON f.region_code = d.code OR f.legacy_code = d.code`. What should be done?

A. Add DISTINCT after the join to remove the duplicate matches the OR produces.

B. Rewrite the OR as two equi-joins combined with UNION so a hash join can be used.

C. Increase the warehouse size so the nested loop has more cores to compare row pairs.

D. Disable AQE so the planner keeps the original sort-merge strategy for the join.

Q9 An engineer sets `delta.deletedFileRetentionDuration` to 1 hour and runs VACUUM hourly to save storage. A downstream stream now fails intermittently with file-not-found errors. Why?

A. Streaming reads require deletion vectors to be disabled whenever VACUUM runs more than daily.

B. The checkpoint must be stored in DBFS root, otherwise VACUUM treats it as unreferenced data.

C. VACUUM rewrites the transaction log, which invalidates the offsets stored in the stream's checkpoint.

D. VACUUM removed files that the lagging stream still needed; retention must exceed reader lag.

Q10 A scheduled ETL job runs 10 minutes every hour on an all-purpose cluster with 120-minute auto-termination. Which change most reduces cost without hurting the SLA?

A. Enable Photon on the all-purpose cluster so each run finishes in fewer minutes.

B. Raise auto-termination to 240 minutes so the cluster never cold-starts between runs.

C. Add worker nodes so the job finishes faster and the cluster can terminate sooner.

D. Run it on job compute or serverless, created only for the duration of each run.

Q11 Which statement about OPTIMIZE on a liquid-clustered table is true?

A. It requires a ZORDER BY clause listing the clustering columns in the same order.

B. It is incremental: only data not yet clustered (mostly new writes) is rewritten.

C. It rewrites the entire table each time, which is why it should run only weekly.

D. It cannot run while a streaming job is appending to the table, so streams must pause.

Q12 A streaming job writes micro-batches every 30 seconds to a Delta table, producing thousands of 2 MB files a day, and downstream queries slow down. Which remedy set fits?

A. Partition the target by ingestion minute so each folder holds only one batch's files.
B. Lower the trigger interval to 5 seconds so each batch writes less data per file.
C. Convert the target to plain Parquet so that small files can be merged by the reader.
D. Optimized writes and auto compaction (automatic on managed tables), plus regular OPTIMIZE.

Q13 Thirty dashboards scan the same 300 GB gold table every few minutes on classic compute; the profile shows time dominated by cloud storage reads. Which change most directly improves repeated-read latency?

A. SSD-backed instances so the disk cache serves repeat reads; pre-warm with `CACHE SELECT`.
B. Call `df.cache()` in a notebook each morning so the dashboards share its memory.
C. Partition the table by dashboard name so that each dashboard reads only its own folder.
D. Increase `spark.sql.shuffle.partitions` so the scan is split across more tasks.

Q14 A platform team manages 4,000 managed tables and cannot hand-pick clustering keys. Query patterns vary by team and change over time. Which option minimises tuning while keeping skipping effective?

A. Predictive Optimization with `CLUSTER BY AUTO`, which picks keys from query predicates.
B. Turn off statistics collection so that maintenance jobs finish faster.
C. Z-order every table by its primary key in a nightly maintenance job.
D. Partition every table by date, since most analytical queries filter on some date column.

Domain 5 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Answer B

Managed tables let Databricks run maintenance and layout decisions for you, removing maintenance jobs.

A Time travel works on managed tables.

C External Delta tables can use DVs.

D Managed tables are readable by external engines too; that is not the reason.

Q2 Answer D

High-cardinality partitioning is the anti-pattern; liquid clustering handles both keys at any cardinality.

A Each partition is too small to compact meaningfully.

B Multiplies the number of tiny files.

C Keeps the high-cardinality anti-pattern.

Q3 Answer C

Deletion vectors are merge-on-read; physical removal needs a rewrite and VACUUM after retention.

A REORG PURGE plus VACUUM removes them.

B That is copy-on-write, not DVs.

D Invisible to readers is not physically removed.

Q4 Answer A

Skipping depends on co-location of values in files.

B Strings get truncated min/max stats.

C Skipping is independent of Photon.

D Skipping can use statistics on many columns.

Q5 Answer C

Clustering keys are metadata; later OPTIMIZE (or OPTIMIZE FULL) applies the new layout.

A Rewrites everything immediately.

B Partitioning cannot be changed in place.

D Z-order cannot be combined with liquid clustering.

Q6 Answer B

CDF turns an updated table into an incremental change stream so only affected groups are recomputed.

A Every refresh still recomputes 3 TB.

C Irrelevant to recompute cost.

D Still a full recompute each run.

Q7 Answer C

Updates emit before and after images with the same commit version.

A CDF records both images.

B Those types are used for real deletes and inserts.

D No such change type exists.

Q8 Answer B

OR-joined keys block hash-based joins; splitting into equi-joins restores an efficient plan.

A Does not change the join strategy.

C Scales a quadratic algorithm.

D AQE did not cause this; OR conditions did.

Q9 Short retention lets VACUUM delete files still referenced by slow readers or streams.

Answer D

A Unrelated.

B Checkpoints are not table files.

C VACUUM deletes data files, not the log.

Q10 Job/serverless compute has a lower rate and no idle time; the all-purpose cluster idles most of each hour.

Answer D

A Keeps the highest rate and idle cost.

B More idle billing.

C Auto-termination still waits 120 minutes.

Q11 Incremental clustering is the main operational advantage over Z-order.

Answer B

A Z-order is not used with liquid clustering.

C That describes Z-order; FULL is the explicit full rewrite.

D Row-level concurrency allows both.

Q12 Write-time optimisation and compaction fix small files; shorter triggers make them worse.

Answer D

A Explodes the file count further.

B More, smaller files.

C Loses Delta and does not merge anything.

Q13 The disk cache keeps copies of remote files on local SSDs, turning repeated cloud reads into local reads.

Answer A

B Spark cache is per application, not shared with warehouses or other sessions.

C Meaningless for the data.

D Shuffle partitions do not affect scans.

Q14 Automatic liquid clustering selects and adapts keys from observed query predicates.

Answer A

B Removes the basis for skipping.

C Costly rewrites that ignore query predicates.

D Ignores real patterns and creates small files on small tables.

Notes · Domain 5

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 6

6 Ensuring Data Security and Compliance

Least privilege, row filters and column masks, ABAC with governed tags, anonymisation vs pseudonymisation, PII pipelines and GDPR-grade deletion.

8% of the exam · about 5 questions

What you will be able to do

- Apply least privilege on Unity Catalog securables and workspace objects
- Implement row filters and column masks, and know when ABAC policies replace them
- Tell anonymisation from pseudonymisation and pick techniques for each
- Design a PII pipeline and a provable erasure process with Delta

Matching free course

- Databricks Data Privacy

Study checklist

- [ ] Read 6.1–6.6
- [ ] Lab 6: governed tag + ABAC mask + purge
- [ ] Practice questions (14)

Official objectives: least-privilege ACLs on UC securables and workspace objects; ABAC policies with governed tags; anonymisation and pseudonymisation; PII detection and masking pipelines; purging data for retention and erasure requirements.

> **In plain words**
> Security answers "who may see what", compliance answers "can we prove it, and can we make data disappear when the law says so". In a hospital, both are daily work: analysts need data, but not every analyst needs every SSN, and a patient's erasure request must actually erase.

6.1 Access control layers

Layer	Controls	Examples
Unity Catalog privileges	Data objects	`USE CATALOG`, `USE SCHEMA`, `SELECT`, `MODIFY`, `EXECUTE`, `READ VOLUME`, `BROWSE`, `MANAGE`
Row filters, column masks	Rows and values inside a table	`SET ROW FILTER`, `SET MASK`
ABAC policies	Filters/masks across many tables by tag	`CREATE POLICY ... ON CATALOG`
Workspace object ACLs	Jobs, notebooks, clusters, warehouses	Jobs: `CAN VIEW`, `CAN MANAGE RUN`, `CAN MANAGE`, `IS OWNER`
Secrets	Credentials	`dbutils.secrets.get` (values redacted in output)

Least privilege in practice: grant to **groups**, never individuals; analysts get `SELECT` on de-identified views or gold, not raw PHI; operators get `CAN MANAGE RUN` (run, cancel, repair) not `CAN MANAGE`; production jobs run as service principals; nobody gets `ALL PRIVILEGES` on a catalog "to reduce tickets".

6.2 Row filters and column masks

Both are UC SQL functions bound to a table. They apply to **every** reader on UC-enabled compute, whatever the language or tool. Readers do not need `EXECUTE` on the function.

```
CREATE FUNCTION prod.sec.facility_filter(fac STRING) RETURNS BOOLEAN
RETURN is_account_group_member('phi_admins')
    OR fac IN (SELECT facility_id FROM prod.sec.user_facility WHERE user_email = current_user());
ALTER TABLE prod.clinical.encounters SET ROW FILTER prod.sec.facility_filter ON (facility_id);

CREATE FUNCTION prod.sec.ssn_mask(ssn STRING) RETURNS STRING
RETURN CASE WHEN is_account_group_member('hr_admins') THEN ssn ELSE CONCAT('***-**-', RIGHT(ssn, 4))
END;
ALTER TABLE prod.hr.employees ALTER COLUMN ssn SET MASK prod.sec.ssn_mask;
```

Use `is_account_group_member()` for account groups; `is_member()` checks workspace-local groups only. A mask can read other columns with `USING COLUMNS (...)`.

6.3 ABAC: govern by tag, not by table

Per-table masks do not scale: every new table needs its own `ALTER TABLE`, and nobody can see "the rule". **Attribute-based access control** defines a policy once at catalog or schema scope; the policy finds columns and tables by **governed tags**, so new tables are protected as soon as they are tagged.

```
-- steward tags data (column tags must be on the column itself; they do not inherit)
ALTER TABLE prod.clinical.patients ALTER COLUMN ssn SET TAGS ('pii' = 'ssn');
-- governance admin writes one policy for the whole catalog
CREATE POLICY mask_ssn ON CATALOG prod
  COLUMN MASK prod.sec.last_n
  TO `account users` EXCEPT `phi_admins`
  FOR TABLES
  MATCH COLUMNS has_tag_value('pii', 'ssn') AS c
  ON COLUMN c USING COLUMNS (4);
SHOW EFFECTIVE POLICIES ON TABLE prod.clinical.patients;
```

	ABAC policy	Table filter/mask	Dynamic view
Scope	Catalog/schema, matched by tags	One table	One view
New tables covered	Yes, once tagged	No	No
Protects base table	Yes	Yes	No: base must be locked separately
Best for	Many tables, consistent rules	One special table	A curated view for one audience

Facts to remember: governed tags are account-level with allowed values and assign permissions; table/schema tags inherit downward, **column tags do not**; data classification can detect PII and apply tags; ABAC needs **serverless or DBR 16.4+** (older runtimes cannot read protected tables); several policies can apply to one table.

6.4 Anonymisation vs pseudonymisation

	Pseudonymisation	Anonymisation
Re-identifiable?	Yes, with extra information (key, salt, mapping)	No, irreversibly
Still personal data (GDPR)?	Yes	No
Techniques	Keyed hashing, tokenisation with a vault, encryption	Generalisation (age bands, 3-digit ZIP), suppression, aggregation, noise, k-anonymity
Keeps joins?	Yes (same key → same token)	Usually not

```
-- deterministic keyed hash keeps joins across datasets without exposing emails
SELECT sha2(concat(secret('pii','salt'), lower(trim(email))), 256) AS email_token, ... FROM
bronze.members;
```

Crypto-shredding: if pseudonyms can only be resolved through a mapping table or key, deleting a person's mapping row (or key) makes their historical pseudonymised records effectively anonymous without rewriting every table.

6.5 A compliant PII pipeline

1. Land raw data in a restricted bronze schema (few principals).
2. Detect PII (classification, `ai_mask` /regex for free text), tag columns with governed tags.
3. Pseudonymise identifiers in silver; keep the mapping in a locked vault table.
4. Serve analysts from gold or de-identified views; ABAC masks anything tagged that slips through.
5. Audit with `system.access.audit`; review grants to broad groups regularly.

6.6 Purging for retention and erasure

Deleting rows in Delta is not enough: old files (and, with deletion vectors, the same files) still hold the data until rewritten and vacuumed.

```
DELETE FROM prod.silver.patients WHERE patient_id = 'P123';
ALTER TABLE prod.silver.patients SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = 'interval 1 days',
  'delta.logRetentionDuration' = 'interval 1 days');
REORG TABLE prod.silver.patients APPLY (PURGE);     -- rewrite files that have deletion vectors
VACUUM prod.silver.patients;                          -- after the retention window has passed
```

Then: repeat for every layer and derived table (bronze, gold, feature tables), full-refresh streaming tables that copied the rows, remove the person from the pseudonym mapping, and handle shared copies. Downstream streams need `skipChangeCommits` to survive the deletes. For scheduled retention (delete data older than 7 years), cluster or partition by a time key so deletes remove whole files cheaply.

Compliance distractors

"DELETE is enough" (files remain for 7 days by default). "OPTIMIZE removes the data" (old files remain until VACUUM). "Hashing = anonymisation" (it is pseudonymisation). "Grant BROWSE to satisfy erasure" (visibility is not deletion). "Masks only apply in SQL warehouses" (they apply on all UC compute).

Remember

- Groups, least privilege, service principals; `CAN MANAGE RUN` for operators.
- Filters and masks are UC functions bound to tables, enforced everywhere on UC compute; use `is_account_group_member`.
- ABAC = governed tags + one policy per scope; covers future tables; column tags do not inherit; serverless or DBR 16.4+.
- Hashing/tokenising/encrypting = pseudonymisation; generalising/suppressing/aggregating = anonymisation.
- Erasure = DELETE + REORG PURGE (with DVs) + shortened retention + VACUUM, in every copy; crypto-shred mappings.

Lab 6 · One rule, every table (40 min)

1. Create two tables with an `email` column. Tag both columns `pii = email` (use a free-form tag if governed tags are not available in Free Edition).
2. Create a mask function and either an ABAC policy on the schema or a per-table mask; query as yourself and as a test group member.
3. Create a third table, tag it, and check whether it is masked without further work (that is the ABAC advantage).
4. Delete one row, run `DESCRIBE HISTORY`, time-travel to see it is still readable, then reason about what REORG + VACUUM would change.

Domain 6 practice questions

Answer first, check later. Score: _____ / 14 · Date: _____ · Retake date: _____

Q1 Analysts query `clinical.encounters` from SQL warehouses, notebooks and a BI tool, and must see only their own facility's rows. Which implementation enforces this on every access path?

A. A dynamic view per facility, with analysts granted SELECT on their facility's view only.

B. A row filter function attached with `ALTER TABLE ... SET ROW FILTER f ON (facility_id)`.

C. A WHERE clause in each BI dashboard based on the logged-in user's facility attribute.

D. Separate tables per facility, with notebook CAN READ permissions limiting who opens what.

Q2 A view uses `CASE WHEN is_member('phi_admins') THEN ssn ELSE 'REDACTED' END`. A member of the account group `phi_admins` still sees REDACTED. Why?

A. `is_member()` checks workspace-local groups; use `is_account_group_member()`.
B. A column mask on `ssn` overrides the view logic for every user except the owner.
C. Views cannot call group functions, so the CASE expression always takes the ELSE branch.
D. The user lacks MODIFY on the base table, which is required to see unmasked values.

Q3 Which technique produces data that is no longer personal data under GDPR?

A. Salted SHA-256 hashing of email addresses with a secret salt stored in a vault.
B. Generalising ages to 10-year bands and suppressing direct identifiers, irreversibly.
C. Tokenisation that replaces each identifier with a random token kept in a mapping table.
D. AES encryption of the SSN column with keys managed in the cloud key service.

Q4 Two departments must join datasets on customer email without either seeing plain emails. Which method keeps joinability?

A. Suppress the email column in both datasets and join on customer name and birth date instead.
B. The same keyed hash of the normalised email in both pipelines, joining on the hash.
C. Mask emails to `***@domain` with a column mask and join on the masked value.
D. Generalise emails to their domain so the datasets can be joined at company level.

Q5 A DELETE removed a patient's rows from a table with deletion vectors and default retention. Legal needs the data unrecoverable within 24 hours. Which sequence achieves this?

A. Disable Change Data Feed so the deleted rows are no longer stored in change files.
B. Run `VACUUM t RETAIN 168 HOURS` immediately after the delete completes.
C. Run OPTIMIZE, which rewrites the files and removes the deleted rows from storage.
D. Shorten retention, `REORG ... APPLY (PURGE)`, then VACUUM after the window.

Q6 A job operator must trigger and repair production runs but must not change the job definition. Which permission?

A. CAN VIEW and CAN ATTACH TO
B. CAN MANAGE
C. IS OWNER
D. CAN MANAGE RUN

Q7 A notebook prints a JDBC password obtained with `dbutils.secrets.get`. What appears in the output?

A. An error
B. The plain-text password
C. `[REDACTED]`
D. A base64-encoded value

Q8 Which column mask reveals the full value only to `hr_admins` and the last four characters to everyone else?

A. `ALTER TABLE t SET ROW FILTER m ON (ssn)` with a function that returns the last four characters.
B. `CREATE VIEW v AS SELECT RIGHT(ssn, 4) AS ssn FROM t`, granted to every analyst group.
C. A UC mask function returning `s` for `hr_admins` and `CONCAT('****', RIGHT(s,4))` otherwise.
D. `GRANT SELECT (ssn) ON TABLE t TO hr_admins` plus a view with `RIGHT(ssn, 4)` for other users.

Q9 A policy requires deleting events older than 7 years from a 40 TB append-only table monthly, cheaply, while downstream streams keep running. Which design is best?

A. Take a SHALLOW CLONE each month and delete old rows from the clone instead.
B. Cluster by event month, delete old months, VACUUM; streams use `skipChangeCommits`.
C. Recreate the table each month with CTAS that excludes rows older than seven years.
D. Set `delta.logRetentionDuration` to 7 years so older data expires automatically.

Q10 Which statement about row filters and column masks is correct?

A. Readers need EXECUTE on the filter or mask function for their queries to succeed.
B. A mask can reference only the masked column, never other columns of the row.
C. They apply only when data is queried through Databricks SQL warehouses.
D. They are enforced for all readers on UC-enabled compute, in any language.

Q11 A pipeline pseudonymises members and stores `member_id → token` mappings. A member exercises the right to erasure. What makes historical pseudonymised records effectively anonymous without rewriting every downstream table?

A. Crypto-shred: delete the member's mapping and purge bronze identifiers.
B. Enable deletion vectors on downstream tables so the member's rows are masked.
C. Re-hash every downstream table with a new salt so that old tokens no longer match.
D. Replace SELECT with BROWSE on all tables that contain the member's tokens.

Q12 Hundreds of tables across several schemas contain PII, and more will come. Governance wants one centrally maintained rule masking any column tagged `pii = email` for everyone except `privacy_admins`, including next year's tables. What should be implemented?

A. Revoke SELECT on all tables and grant access case by case through tickets.
B. `ALTER TABLE ... SET MASK` on every existing table, reapplied by a nightly job for new tables.
C. A governed `pii` tag plus one ABAC column-mask policy at catalog scope matching `email`.
D. Dynamic views duplicating every base table with masking logic in each view.

Q13 After an ABAC policy was created on `prod.clinical`, analysts on a DBR 14.3 cluster in standard access mode get errors reading protected tables, while serverless users are fine. What is the cause?

A. ABAC needs serverless or DBR 16.4+; older runtimes cannot read ABAC-protected tables.
B. Governed tags are only replicated to serverless compute, not to classic clusters.
C. The policy must be created on each table rather than on the schema to work on clusters.
D. Analysts need EXECUTE on the masking function used by the policy.

Q14 A steward tagged a schema `sensitivity = 'high'` and expected a policy matching `has_tag_value('pii','ssn')` to mask `ssn` in a new table. Nothing is masked. Why?

A. Policies only apply to tables that existed when the policy was created.
B. Column masks cannot be expressed as ABAC policies, only row filters can.
C. The schema must be recreated for the new tag to propagate to its tables.
D. Column tags do not inherit; `pii = ssn` must be set on the column itself.

Domain 6 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Row filters bind to the table and apply to every engine and tool on UC compute.

Answer B

A Protects only the views, and the approach does not scale.

C Protects one tool only.

D Notebook ACLs do not protect data; does not scale.

Q2 Unity Catalog groups are account-level; is_member() only sees workspace-local groups.

Answer A

B Nothing indicates a mask exists; the function scope is the issue.

C Views can call these functions.

D MODIFY is a write privilege.

Q3 Only irreversible techniques (generalisation, suppression, aggregation) anonymise.

Answer B

A Keyed hashing is pseudonymisation: re-identifiable with the salt.

C Reversible through the mapping.

D Reversible with the key.

Q4 A deterministic keyed hash preserves equality for joins while hiding the value.

Answer B

A Weak, error-prone key and still identifying.

C Masked values collide.

D Destroys person-level joins.

Q5 Purge rewrites DV-masked files; VACUUM with a shortened retention removes the old files.

Answer D

A Does not purge data files.

B 168 hours is the 7-day default.

C Old files remain until VACUUM, 7 days by default.

Q6 CAN MANAGE RUN allows run, cancel and repair without editing settings.

Answer D

A Circumvents least privilege.

B CAN MANAGE allows editing the job.

C Owners can edit everything.

Q7 Databricks redacts secret values in notebook output.

Answer C

A Secrets can be assigned to variables; only the output is redacted.

B Values are redacted for everyone in output.

D Not how redaction works.

Q8 Column masks are UC functions bound to a column; the CASE decides per principal.

Answer C

A Row filters return booleans and remove rows.

B Nobody sees the full value, and the base table is unprotected.

D Column-level grants do not exist in UC.

Q9
Answer B

Deleting by a coarse time key drops whole files; streams must be told to ignore the delete commits.

A The original still holds the data.
C Rewrites 40 TB monthly and breaks streams.
D Log retention does not delete rows.

Q10
Answer D

Filters and masks are bound to the table and enforced engine-agnostically.

A Readers do not need EXECUTE.
B USING COLUMNS passes other columns.
C Enforced on all UC compute.

Q11
Answer A

Without the mapping, tokens cannot be resolved back to the person.

B Rows are not deleted; tokens remain resolvable.
C Rewrites everything; unnecessary.
D Visibility is not erasure.

Q12
Answer C

ABAC policies key on governed tags and cover future tables automatically.

A That is no access, not least privilege.
B Does not scale and leaves gaps between runs.
D Base tables stay exposed; heavy upkeep.

Q13
Answer A

ABAC enforcement requires recent runtimes or serverless compute.

B Tags are metastore metadata.
C Schema scope is valid.
D Readers do not need EXECUTE.

Q14
Answer D

The policy matches columns by their own tags; schema tags never reach columns.

A Policies cover later tables.
B Column-mask policies exist.
C Tags apply immediately.

Notes · Domain 6

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 7

7 Data Governance

Making data findable and understandable, and knowing exactly how permissions flow through Unity Catalog.

5% of the exam · about 3 questions

What you will be able to do

- Predict who can access what from grants on catalogs, schemas and tables
- Use comments, tags, lineage and classification so data is discoverable
- Explain ownership, MANAGE and who may grant

Matching free course

- Data Management and Governance with Unity Catalog
- Databricks Data Privacy (audit and isolation)

Study checklist

- [ ] Read 7.1–7.3
- [ ] Lab 7: grants and inheritance
- [ ] Practice questions (11)

Official objectives: use Unity Catalog tags and comments for discoverability; understand the permission inheritance model.

7.1 Permission inheritance: the rules

In plain words

Grants flow downward like water. A privilege granted on a catalog applies to every schema and table inside it, including ones created tomorrow. There is no "deny": to stop access you must remove the grant where it was given. And to reach a table you need a "key" for every door on the way: `USE CATALOG` and `USE SCHEMA`, plus `SELECT` on the table (or above it).

Rule	Consequence
Privileges inherit to current and future children	`GRANT SELECT ON SCHEMA prod.gold` covers tables created next month
The USE chain is required	`SELECT` without `USE CATALOG` + `USE SCHEMA` fails
No DENY in grants	Revoking on a table does nothing if `SELECT` was granted on the catalog
Views run as their owner	Readers need SELECT on the view only; the view owner needs access to base tables
`BROWSE`	See metadata (names, comments, columns) without reading data
Who can grant	Object owner, principals with `MANAGE`, owners of parent objects, metastore admins
Ownership	Transfer to groups or service principals, not people

```
GRANT USE CATALOG ON CATALOG prod TO `bi_users`;
GRANT USE SCHEMA, SELECT ON SCHEMA prod.gold TO `bi_users`;   -- all current and future gold tables
GRANT BROWSE ON CATALOG prod TO `all_analysts`;                -- discoverable, not readable
ALTER TABLE prod.gold.kpi OWNER TO `data_platform_team`;
SHOW GRANTS ON SCHEMA prod.gold;
```

The revoke that does nothing

`GRANT SELECT ON CATALOG prod TO analysts; REVOKE SELECT ON TABLE prod.gold.salaries FROM analysts;` Analysts can still read salaries: the catalog grant still applies. Restructure (move sensitive tables to a separate schema/catalog), grant at a lower level, or protect with a view, filter or mask.

7.2 Discoverability

Mechanism	Use
Comments on tables and columns	Human descriptions; searchable; used by AI assistants to answer questions correctly
Tags (key/value)	Structured labels: owner, domain, refresh cadence, PII class; searchable and queryable
Governed tags	Account-level tag keys with allowed values and assignment permissions; inherit catalog → schema → table (not to columns); used by ABAC
Lineage	Automatic table and column lineage from UC-enabled compute; shows upstream sources and downstream consumers
Data classification	Detects sensitive columns and suggests or applies tags
information_schema	Query tags, comments, grants programmatically

```
COMMENT ON TABLE prod.gold.claims_monthly IS 'Paid claims by plan and month. Refreshed daily 06:00 UTC. Owner: claims-data team.';
ALTER TABLE prod.gold.claims_monthly ALTER COLUMN paid_amount COMMENT 'Sum of allowed minus member responsibility, USD';
ALTER TABLE prod.gold.claims_monthly SET TAGS ('domain' = 'claims', 'refresh' = 'daily', 'pii' = 'false');
SELECT * FROM system.information_schema.column_tags WHERE tag_name = 'pii' AND tag_value <> 'false';
```

Lineage gaps usually mean the writing workload ran on compute without Unity Catalog (legacy no-isolation mode) or wrote through paths instead of table names.

Governance that people actually use

Make comments and tags part of the definition of done: a PR that adds a gold table without a table comment, column comments on business measures, an owner tag and a refresh tag is not finished. Put this in the bundle (table properties and comments in the pipeline code) so it cannot drift.

Remember

- Inheritance flows down to future children; no DENY; revoke where granted.
- Need USE CATALOG + USE SCHEMA + SELECT.
- Views run as owner; readers need only the view.
- BROWSE = metadata only. Owners, MANAGE holders, parent owners and admins can grant.
- Comments for meaning, tags for structure, governed tags for policy, information_schema for queries.

Lab 7 · Prove inheritance (20 min)

1. Create a group (or use a colleague's account in a trial) and grant `SELECT ON SCHEMA` only: the query fails without `USE CATALOG`.
2. Add `USE CATALOG` / `USE SCHEMA`, then create a new table in the schema: readable without new grants.
3. Try revoking `SELECT` on that single table and confirm it is still readable.
4. Add comments and tags, then find the table through Catalog Explorer search and `information_schema.table_tags`.

Domain 7 practice questions

Answer first, check later. Score: _____ / 11 · Date: _____ · Retake date: _____

Q1 A user was granted `SELECT ON SCHEMA prod.silver` but gets a permission error querying `prod.silver.orders`. What is most likely missing?

A. `EXECUTE` on the catalog, which authorises running queries against its objects
B. `USE CATALOG` on `prod` and/or `USE SCHEMA` on `prod.silver`
C. Ownership of the schema, since schema-level grants only apply to the owner's tables
D. `MODIFY` on the table, which is required whenever a query scans more than one file

Q2 An admin grants `SELECT ON CATALOG prod` to `analysts`, then runs `REVOKE SELECT ON TABLE prod.gold.salaries FROM analysts`. Can analysts still read `salaries`?

A. No, because a table-level revoke overrides any privilege inherited from its catalog.
B. Only if they own the table, because revokes do not apply to owners of objects.
C. Only through SQL warehouses, since clusters evaluate table revokes before inherited grants.
D. Yes: the catalog grant still applies; revoke there or move the table elsewhere.

Q3 Which privilege lets a steward see table names, comments and columns in Catalog Explorer without querying data?

A. BROWSE on the catalog
B. READ VOLUME
C. SELECT with a row filter
D. USE SCHEMA on each schema

Q4 View `gold.v_patients_deid` is owned by alice and reads `silver.patients`. What does analyst bob need to query the view without base-table access?

A. SELECT on `silver.patients` as well, since views run with the caller's permissions.
B. MODIFY on the view for both alice and bob so the view can be evaluated.
C. Ownership of the view transferred to bob so that it runs with his rights.
D. USE CATALOG/SCHEMA and SELECT on the view; alice needs SELECT on the base table.

Q5 Which statement about Unity Catalog privileges is true?

A. Grants must be repeated for each new table, because inheritance covers only existing tables.
B. Only the metastore admin can grant privileges on objects in any catalog.
C. Privileges granted on a schema apply to tables created in it later.
D. A DENY on a table blocks privileges inherited from its schema or catalog.

Q6 Every gold table must show its business owner, refresh cadence and a PII flag in search results. Which combination is most appropriate?

A. A README file in the Git folder of the pipeline that creates each table.
B. A separate Delta table listing metadata for every table, maintained by hand.
C. Comments for descriptions plus tags such as `owner`, `refresh` and `pii`.
D. Encode the owner and cadence in each table name, such as `kpi__finance__daily`.

Q7 Which SQL correctly documents a column?

A. `DESCRIBE COLUMN t.claim_status 'Status: OPEN, PAID, DENIED'`
B. `ALTER TABLE t ALTER COLUMN claim_status COMMENT 'Status: OPEN, PAID, DENIED'`
C. `SET COLUMN COMMENT t.claim_status 'Status: OPEN, PAID, DENIED'`
D. `COMMENT ON COLUMN t.claim_status = 'Status: OPEN, PAID, DENIED'`

Q8 Compliance wants to find all columns tagged `pii = 'true'` across the metastore programmatically. Where should they query?

A. `DESCRIBE HISTORY` on each table, looking for SET TAGS operations
B. `system.access.audit`, filtered to tag assignment events on columns
C. The pipeline event log of each pipeline that writes PII tables
D. `system.information_schema.column_tags` (or each catalog's own view)

Q9 Who can grant SELECT on `prod.gold.kpi` to a group?

A. Only the user who created the metastore that contains the catalog
B. Any user who has SELECT on the table, since read access includes sharing it
C. Owners, MANAGE holders, parent owners, or a metastore admin
D. Only workspace admins of the workspace where the table was created

Q10 Lineage for a gold table shows no upstream sources, although a job populates it nightly. What is the most likely reason?

A. The job runs on compute without Unity Catalog, such as no-isolation access mode.
B. Lineage is captured only for tables that are also shared with Delta Sharing.
C. Lineage requires column comments on the target table before it is recorded.
D. Lineage is captured only for queries run on SQL warehouses, not for jobs.

Q11 What distinguishes governed tags from ordinary tags?

A. Governed tags can be applied only to catalogs, while ordinary tags go on tables and columns.
B. Governed tags are account-level with allowed values and permissions; ABAC can match them.
C. Ordinary tags are enforced by row filters, while governed tags are for documentation.
D. Governed tags replace comments and are shown instead of descriptions in search.

Domain 7 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Answer B

Access needs the USE chain as well as SELECT.

A EXECUTE is for functions.

C Grants apply to all tables in the schema.

D MODIFY is for writes.

Q2 Answer D

Inherited privileges persist until revoked where they were granted.

A UC has no DENY semantics.

B They are not owners; the catalog grant is the reason.

C Engine does not matter.

Q3 Answer A

BROWSE grants metadata visibility without data access.

B Volumes hold files, not table metadata.

C Still a data grant, just filtered.

D Traversal only; it does not reveal every object's metadata.

Q4 Answer D

Views execute with the owner's privileges; readers need only the view.

A UC views run with the owner's rights on base tables.

B MODIFY is irrelevant to reading.

C Unnecessary and over-grants.

Q5 Answer C

Inheritance includes objects created in the future.

A Inheritance covers future children.

B Owners and MANAGE holders can grant too.

D There is no DENY.

Q6 Answer C

Comments and tags are the native, searchable metadata mechanisms.

A Not visible in catalog search.

B Drifts from reality; not native search.

D Brittle and not structured metadata.

Q7 Answer B

ALTER TABLE ... ALTER COLUMN ... COMMENT (or COMMENT ON COLUMN ... IS) sets column comments.

A DESCRIBE reads metadata.

C Not valid syntax.

D The syntax uses IS, not =.

Q8 Answer D

Tags are exposed through information_schema views.

A Per table and incomplete.

B Shows changes over time, not the current state reliably.

C Pipelines do not record tags.

Q9 Ownership and MANAGE (and parent ownership or admin) confer the right to grant.

Answer C

A Too narrow.

B SELECT does not include granting.

D Workspace admin is not a UC grant right by itself.

Q10 Lineage capture requires UC-enabled compute and table-name access.

Answer A

B Unrelated.

C Comments do not affect lineage.

D Jobs on UC compute capture lineage.

Q11 Governed tags are a controlled taxonomy that policies can rely on.

Answer B

A Governed tags apply at all levels.

C Reversed: governed tags drive policies.

D They complement comments.

Notes · Domain 7

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 8

8 Debugging and Deploying

Finding the cause of failures quickly, recovering cheaply, and shipping code safely with bundles and CI/CD.

10% of the exam · about 6 questions

What you will be able to do

- Know where to look first for each kind of failure
- Recognise common errors and their fixes
- Repair runs with parameter overrides
- Deploy with Declarative Automation Bundles and CI from Git

Matching free course

- Automated Deployment with Declarative Automation Bundles
- DevOps Essentials for Data Engineering (Associate refresher)

Study checklist

- ☐ Read 8.1–8.5
- ☐ Lab 8: deploy a bundle to dev and prod-like targets
- ☐ Practice questions (11)

Official objectives: diagnose with the Spark UI, cluster logs, system tables and query profiles; repair jobs and override parameters; use event logs; deploy with Declarative Automation Bundles; CI/CD with Git folders.

8.1 Where to look first

Symptom	First place	Then
Task failed	Task run output / error in the Jobs UI	Driver logs (stdout/stderr/log4j)
Slow stage	Spark UI stages or Query Profile	Task distribution, spill, shuffle
Cluster died, "executor lost"	Cluster event log (spot loss, termination reason)	`system.compute.node_timeline`
Pipeline flow failed	Pipeline UI, then `event_log()` WHERE level = 'ERROR'	Flow's input schema, expectations
Numbers changed unexpectedly	`DESCRIBE HISTORY table` (who, when, operation, metrics)	`RESTORE` if needed
Recurring failures across jobs	`system.lakeflow.job_task_run_timeline`	Audit log for config changes

Short-lived job clusters lose their logs when they terminate unless **cluster log delivery** to a volume is configured.

8.2 Common errors and what they mean

Error	Meaning	Fix
`ConcurrentAppendException` / `ConcurrentDeleteReadException`	Two writers touched the same files	Disjoint layouts, deletion vectors (row-level concurrency), retries
`UnknownFieldException` (Auto Loader)	New column detected	Restart (schema already evolved) or change mode
Streaming source "detected a data update"	Non-append change on a Delta source	`skipChangeCommits`, full refresh, or CDF
MERGE "multiple source rows matched"	Duplicate keys in source	Deduplicate source
`FileNotFoundException` on a Delta read	Files vacuumed while needed	Longer retention; restart stream carefully
Driver OOM	collect/toPandas, big broadcast	Keep distributed; high memory; smaller broadcast
Executor OOM / heavy spill	Skew, too few partitions	AQE skew, salting, more partitions
`ModuleNotFoundError`	Dependency not installed where code runs	Environment spec (serverless), task libraries
`PERMISSION_DENIED` / `USE CATALOG` missing	Grant chain incomplete	Grant USE CATALOG/SCHEMA

8.3 Repair runs

Repair re-runs only failed tasks (and the tasks that depend on them) of an existing run, keeping successful tasks' results, and accepts new job parameters. It is almost always cheaper than rerunning the whole job.

```
databricks jobs repair-run 987 --rerun-all-failed-tasks --json '{"job_parameters": {"run_date": "2026-10-01"}}'
```

There are no cross-task transactions: tasks that succeeded before the failure keep their commits. Design tasks to be idempotent so a repair is always safe.

8.4 Declarative Automation Bundles

```
databricks bundle init                  # from a template
databricks bundle validate -t dev       # schema + config checks
databricks bundle deploy -t dev         # upload code, build wheel, create/update resources
databricks bundle run -t dev claims_job # run a resource
databricks bundle deploy -t prod        # normally from CI only
databricks bundle destroy -t dev        # remove dev resources
```

	`mode: development`	`mode: production`
Names	Prefixed `[dev <user>]`	As defined
Schedules/triggers	Paused	Active
Identity	Deploying user	`run_as` required (service principal)
Checks	Relaxed, concurrent runs allowed	Validates run_as, Git branch settings, no dev overrides

Bundles are **declarative**: the deployed state is reconciled to the YAML on every deploy. A change made by hand in the UI is overwritten on the next deploy. Make changes in the source.

8.5 Git folders and CI/CD

Git folders give each developer a working copy in the workspace for branches, commits and pull requests. Production should not run from someone's Git folder. The standard flow:

1. Developer works on a branch in a Git folder or locally (VS Code with the Databricks extension or Databricks Connect).
2. PR triggers CI: unit tests, `bundle validate`, deploy to a staging target, integration tests.
3. Merge to `main` triggers CD: `bundle deploy -t prod` as a service principal with OAuth.
4. Jobs in prod run the deployed, reviewed code (or a job configured with a Git source pinned to a branch or tag).

```
# .github/workflows/deploy.yml (excerpt)
- uses: databricks/setup-cli@main
- run: databricks bundle deploy -t prod
  env: {DATABRICKS_HOST: ${{ secrets.HOST }}, DATABRICKS_CLIENT_ID: ${{ secrets.SP_ID }},
DATABRICKS_CLIENT_SECRET: ${{ secrets.SP_SECRET }}}
```

Incident habits of senior engineers

Stop the bleeding first (pause the schedule, stop downstream publishes), then find the first failing thing, not the loudest. Use `DESCRIBE HISTORY` and the event log to establish a timeline, repair instead of rerun, and write a short blameless note: what happened, why it was not caught, what check now catches it. The check (an expectation, an alert, a test) is the real deliverable.

Remember

- Task error → driver logs; slow → Spark UI/Query Profile; cluster died → cluster event log; data changed → DESCRIBE HISTORY.
- Repair re-runs failed + dependent tasks with parameter overrides; no cross-task rollback.
- Bundles: validate/deploy/run/destroy; dev mode = prefixed, paused; prod mode = run_as required.
- UI edits are overwritten by the next deploy; CI deploys prod as a service principal from main.
- Trigger or rate-limit changes keep the checkpoint; log delivery keeps job-cluster logs.

Lab 8 · Your first bundle (60 min)

1. Install the Databricks CLI locally, authenticate to Free Edition, run `databricks bundle init default-python`.
2. Look at `databricks.yml`, `resources/` and `src/`. Run the unit tests locally.
3. `bundle validate`, `bundle deploy -t dev`, `bundle run`. Find the `[dev you]` prefix in the Jobs UI.
4. Change the schedule in the UI, redeploy, and watch it revert.
5. Break a task on purpose, then fix it with a repair run and a parameter override.

Domain 8 practice questions

Answer first, check later. Score: _____ / 11 · Date: _____ · Retake date: _____

Q1 A 12-task job failed at task 9 after 3 hours; tasks 10–12 were skipped. The fix is a parameter typo. What is the fastest, cheapest recovery?

A. Increase retries on task 9 to 3 and wait for the scheduler to try it again.
B. Run the whole job again from the start with the corrected parameter value.
C. A repair run with the corrected parameter, re-running only task 9 onward.
D. Create a temporary job containing only tasks 9 to 12 and run it once.

Q2 A nightly MERGE now intermittently fails with `ConcurrentAppendException` since a new streaming job writes to the same table. What reduces conflicts without serialising the workloads?

A. Convert the table to Parquet so the writers no longer compete on a transaction log.
B. Deletion vectors (row-level concurrency), disjoint layouts, and retry logic.
C. Disable AQE on the MERGE job so its file selection is deterministic across attempts.
D. Run VACUUM before each MERGE so the streaming job's new files are not scanned by it.

Q3 A job cluster terminates mid-run with "executor lost" errors. Where should the engineer look first to confirm spot reclamation?

A. The SQL query history of the warehouse used by the dashboards reading the output.
B. The pipeline event log, which records node loss for all compute types.
C. The cluster event log, and `system.compute.node_timeline`.
D. `DESCRIBE HISTORY` on the target table to see when writes stopped.

Q4 A gold table's values changed unexpectedly overnight. What is the quickest way to find which job or user wrote the change?

A. Open the Spark UI of every cluster that ran overnight and check their SQL tabs.
B. Read all notebooks in the workspace, looking for statements that write to the table.
C. Run VACUUM with a dry run to list which files were removed during the night.
D. `DESCRIBE HISTORY` on the table to see writer, job and operation; RESTORE if needed.

Q5 A pipeline update fails and the UI shows a red flow for `silver_claims`. Which query gives the error details programmatically?

A. `SHOW TBLPROPERTIES prod.silver.claims` and look for failure properties
B. `SELECT * FROM system.billing.usage WHERE usage_metadata.dlt_pipeline_id = '...'`
C. `event_log()` on the table, filtered to `level = 'ERROR'`
D. `DESCRIBE DETAIL prod.silver.claims` and read the `lastError` field

Q6 After `databricks bundle deploy -t prod`, a schedule that an operator changed in the UI reverted to the YAML value. What explains this?

A. The operator lacked CAN MANAGE, so the UI change was never saved.
B. Bundles reconcile deployed resources to the source; UI changes are overwritten.
C. A known CLI defect that resets schedules when the job has more than one task.
D. Production mode pauses all schedules and restores them from a backup.

Q7 Which bundle target configuration is appropriate for production?

A. No explicit target, deployed from each engineer's laptop after local testing.
B. `mode: production` deployed as a personal user with a token stored in the repository.
C. `mode: development` with `default: true`, so every deploy lands in production.
D. `mode: production` with `run_as` a service principal, deployed by CI from `main`.

Q8 A stage shows 2,000 tasks each reading about 1 MB, with high scheduling overhead. What is the likely problem and fix?

A. Too few executors for the number of tasks; add worker nodes to the cluster.
B. Data skew on the scan key; salt the key before reading the table.
C. Driver OOM caused by task metadata; increase driver memory.
D. Small files; compact with OPTIMIZE and let AQE coalesce.

Q9 Which CI pattern ensures a job always runs reviewed code from `main` rather than a developer's working copy?

A. On merge to main, CI deploys the bundle to prod as a service principal.
B. Ask developers to push their branches before each scheduled run.
C. Export the notebooks to DBFS every week and run the job from the exported copies.
D. Point the job's notebook path at the lead developer's Git folder, which tracks main.

Q10 A stream's trigger changes from 1 minute to 5 minutes and `maxFilesPerTrigger` is reduced. What must the engineer do about the checkpoint?

A. Nothing; trigger and rate limits can change with the same checkpoint.
B. Start a new checkpoint and set `startingVersion` to the latest version.
C. Delete it and reprocess from the beginning with the new settings.
D. Migrate the job to a declarative pipeline, which rewrites checkpoints automatically.

Q11 How can driver and executor logs of short-lived job clusters be kept for later analysis?

A. Increase auto-termination so logs stay available on the cluster longer.

B. Configure cluster log delivery to a Unity Catalog volume in the job's cluster spec.

C. Set `spark.eventLog.enabled` to false so logs are written to the driver disk instead.

D. Export notebook outputs as HTML at the end of every run.

Domain 8 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Repair re-runs only failed and dependent tasks and accepts parameter overrides.

Answer C

A Retries reuse the wrong parameter.

B Repeats 3 hours of successful work.

D Extra work and loses run context.

Q2 Row-level concurrency and non-overlapping file sets reduce conflicts; retries handle the rest.

Answer B

A Loses ACID guarantees entirely.

C AQE is unrelated.

D VACUUM does not change conflict detection.

Q3 Termination and instance-loss reasons are in the cluster event log.

Answer C

A Unrelated compute.

B Only for pipelines.

D Shows commits, not why nodes vanished.

Q4 Delta history records operation, user, job/notebook and metrics for every version.

Answer D

A Slow and logs may be gone.

B Slow and incomplete.

C Does not identify the writer.

Q5 The event log table-valued function exposes error events with details.

Answer C

A Properties are configuration.

B Billing has no errors.

D No such field; DETAIL shows metadata.

Q6 Declarative deployment makes the bundle source the truth; edit the YAML.

Answer B

A The change was saved, then overwritten.

C It is by design.

D Production mode keeps schedules active.

Q7 Production mode plus a service principal plus CI is the documented pattern.

Answer D

A Uncontrolled and personal.

B Personal identity and leaked secret.

C Development mode is for individual iteration.

Q8 Many tiny tasks signal small files or over-partitioning.

Answer D

A More nodes do not fix tiny files.

B Skew shows one long task.

C The symptom is overhead, not OOM.

Q9 Deploying from CI decouples production from personal folders.

Answer A

B Process, not a guarantee.

C Legacy and unreviewed.

D Still a personal working copy.

Q10 Trigger and rate limits are not part of checkpointed state.

Answer A

B Risks skipping data; unnecessary.

C Unnecessary reprocessing.

D Not needed.

Q11 Log delivery persists logs beyond cluster termination.

Answer B

A Job clusters end with the run.

C Disables event logging.

D Captures output, not logs.

Notes · Domain 8

What clicked, what did not, commands to try, questions to ask a colleague.

Domain 9

9 Data Modeling

Physical layouts for Delta and Iceberg, dimensional models, and a gold layer built from materialized views and Metric Views.

5% of the exam · about 3 questions

What you will be able to do

- Choose partitioning, clustering and compaction from grain and access patterns
- Model facts and dimensions, including SCD Type 2 and surrogate keys
- Know which constraints Delta enforces and what informational keys are for
- Serve consistent KPIs with Metric Views and fast ones with materialized views

Matching free course

- Data Modeling Strategies (Data Warehousing Practitioner pathway)
- Building Semantic Models with UC Metric Views

Study checklist

- [ ] Read 9.1–9.4
- [ ] Lab 9: star schema + metric view
- [ ] Practice questions (12)

Official objectives: Delta and Iceberg layouts (partitioning to grain, clustering to access patterns, compaction); dimensional models with materialized views and Unity Catalog Metric Views.

9.1 Layout decisions

In plain words

Layout is about putting rows that are read together in the same files. Two different forces decide it: how data **arrives and expires** (its grain and lifecycle, which can justify a partition), and how data is **read** (filters and joins, which decide clustering).

1. **Default:** no partitioning; liquid clustering on the top filter/join columns (or `CLUSTER BY AUTO` on managed tables).
2. **Partition only if** the table is large (roughly 1 TB+), the column is low-cardinality with large partitions (each ≥ ~1 GB), and it matches load/delete lifecycle (month, region). Never partition by IDs or timestamps.
3. **Compaction** keeps file sizes healthy: automatic on managed tables; scheduled OPTIMIZE otherwise; for Iceberg, UC-managed compaction and snapshot expiration.
4. **Generated columns** (`event_date DATE GENERATED ALWAYS AS (CAST(event_ts AS DATE))`) let queries that filter on the timestamp prune by the derived date.

9.2 Dimensional modelling on the lakehouse

Element	Lakehouse practice
Fact table	Narrow, long, one row per event at a declared grain; clustered by date and main join key

Element	Lakehouse practice
Dimension	Descriptive attributes; SCD Type 1 (overwrite) or Type 2 (history) via AUTO CDC
Surrogate keys	`GENERATED ALWAYS AS IDENTITY` (single writer at a time) or deterministic hashes of natural keys
SCD Type 2	`effective_from`, `effective_to`, `is_current`; the fact stores the surrogate key valid at event time for point-in-time joins
Constraints	`NOT NULL` and `CHECK` are **enforced**; `PRIMARY KEY` / `FOREIGN KEY` are **informational** (use `RELY` to let the optimizer eliminate joins or aggregations)
Views	Rename/subset columns for different consumers without copies

```
CREATE TABLE prod.gold.dim_member (
  member_sk BIGINT GENERATED ALWAYS AS IDENTITY,
  member_id STRING NOT NULL, plan_code STRING, state STRING,
  effective_from TIMESTAMP, effective_to TIMESTAMP, is_current BOOLEAN,
  CONSTRAINT pk_member PRIMARY KEY (member_sk) RELY) CLUSTER BY (member_id);

CREATE TABLE prod.gold.fact_claims (
  claim_id STRING NOT NULL, member_sk BIGINT, service_date DATE, paid_amount DECIMAL(12,2),
  CONSTRAINT paid_nonneg CHECK (paid_amount >= 0),
  CONSTRAINT fk_member FOREIGN KEY (member_sk) REFERENCES prod.gold.dim_member) CLUSTER BY
(service_date, member_sk);
```

9.3 The gold layer: materialized views and Metric Views

Two different problems, two tools, usually used together:

	Materialized view	Metric view
Solves	Slow dashboards recomputing big joins and aggregates	Inconsistent KPI definitions across reports
What it is	Stored query result, refreshed by a pipeline (incrementally when possible)	A governed semantic definition (YAML): source, joins, dimensions, measures
Query	Like a table	`SELECT dim, MEASURE(m) FROM mv GROUP BY ALL`
Bonus	Cheap reads	Ratios computed correctly at any grain; used by AI/BI dashboards and Genie

```
CREATE OR REPLACE VIEW prod.gold.claims_metrics WITH METRICS LANGUAGE YAML AS $$
version: 1.1
source: prod.gold.fact_claims
joins:
  - name: member
    source: prod.gold.dim_member
    on: source.member_sk = member.member_sk
dimensions:
  - name: Service Month
    expr: DATE_TRUNC('MONTH', service_date)
  - name: State
    expr: member.state
measures:
  - name: Paid Amount
    expr: SUM(paid_amount)
  - name: Paid per Member
    expr: SUM(paid_amount) / COUNT(DISTINCT member_sk)
$$;
SELECT `State`, `Service Month`, MEASURE(`Paid per Member`) FROM prod.gold.claims_metrics GROUP BY ALL;
```

Modelling for trust

Declare the grain of every fact table in its comment ("one row per claim line"). Most wrong numbers in healthcare analytics are grain mistakes: joining claim headers to lines, or members to multiple coverage rows, and summing twice. A metric view that defines "paid per member" once, with the correct distinct count, prevents a whole class of these.

Remember

- Cluster to access patterns; partition only to coarse grain/lifecycle on large tables.
- Generated columns let timestamp filters prune by derived dates.
- SCD2 with surrogate keys for point-in-time joins; identity columns need a single writer.
- NOT NULL and CHECK enforced; PK/FK informational (RELY helps the optimizer).
- MVs precompute; metric views define KPIs once (`MEASURE()`).

Lab 9 · A tiny star with a semantic layer (45 min)

1. Build `dim_member` (SCD2 from Lab 1 if you have it) and a `fact_claims` table of 1,000 generated rows.
2. Add a CHECK constraint and try to insert a negative amount.
3. Create the metric view above and query it grouped by state and by month; verify "paid per member" is right at both grains.
4. Create a materialized view at the state × month grain and compare query times.

Domain 9 practice questions

Answer first, check later. Score: _____ / 12 · Date: _____ · Retake date: _____

Q1 A new 8 TB fact table will be filtered by `service_date` and joined on `member_sk` (millions of values). Which layout?

- A. No layout; rely on the disk cache to make repeated scans fast
- B. Partition by `member_sk` so joins read one folder per member
- C. Partition by `member_sk` and Z-order by `service_date` inside each partition
- D. `CLUSTER BY (service_date, member_sk)`, no partitioning

Q2 If partitioning must be used on a 3 TB events table, which column is the best candidate?

- A. `event_id`, since it is unique and spreads data evenly across partitions
- B. `event_date`: about 1,000 values, several GB each, matching the lifecycle
- C. `user_id` (10 million values), since most queries filter on a user
- D. `event_timestamp`, so that time-range queries can prune at the finest level

Q3 A dimension must keep the full history of address changes and support point-in-time joins from a fact table. Which design?

- A. SCD Type 2 with surrogate keys and validity dates
- B. A separate copy of the dimension for each year, joined on the fact's year.
- C. SCD Type 1 with an `updated_at` column recording when each address last changed.
- D. SCD Type 3 with a `previous_address` column next to the current address.

Q4 Which constraint types does Delta Lake enforce on write?

- A. UNIQUE constraints on any column with statistics
- B. PRIMARY KEY and FOREIGN KEY, when declared with RELY
- C. All constraints declared in the table definition
- D. NOT NULL and CHECK constraints (rejecting bad writes)

Q5 Queries filter on `event_ts` (timestamp) but the table is clustered by a date. Without changing queries, how can pruning improve?

- A. Increase the number of statistics columns to 64 so the timestamp gets min/max values.
- B. A generated column `event_date AS (CAST(event_ts AS DATE))` used for layout.
- C. Disable deletion vectors so file statistics are recomputed on every write.
- D. Cast `event_ts` to STRING so it sorts together with the date values in files.

Q6 What is the main purpose of declaring `PRIMARY KEY ... RELY` on a gold table?

- A. Create an index that speeds up point lookups by key.
- B. Document the model and let the optimizer assume uniqueness to eliminate work.
- C. Enable liquid clustering on the key column automatically.
- D. Reject duplicate keys on insert, just like a relational database would.

Q7 Two teams need the same aggregate data with different column names and a subset of columns. What is the simplest correct solution?

- A A view with aliased, selected columns for each team
- B A CTAS copy per team plus a nightly sync job
- C A second write path in the pipeline producing a renamed copy
- D A DEEP CLONE for each team, refreshed hourly by a job

Q8 Which statement about Z-ORDER is accurate?

- A It changes the table's partitioning scheme to match the Z-ordered columns.
- B It is incremental and only touches files written since the last OPTIMIZE.
- C It rewrites files to co-locate columns; incompatible with liquid clustering.
- D It is required for any data skipping to happen on a Delta table.

Q9 Surrogate keys use `GENERATED ALWAYS AS IDENTITY`. Two jobs insert concurrently and one fails. Why?

- A Identity columns require the table to be Z-ordered by the key column.
- B Identity columns are only supported on external tables, not managed tables.
- C Deletion vectors must be disabled on tables that have identity columns.
- D Identity allocation does not support concurrent writers; use one writer.

Q10 A gold model must give dashboards sub-second filters on region and month over billions of rows; aggregates are deterministic. Which approach fits best?

- A A Python UDF that caches aggregated results inside the dashboard session.
- B Query the silver fact table directly using the largest warehouse size.
- C A materialized view at region × month grain, clustered by region and month.
- D A shallow clone of silver clustered by region for the dashboard team.

Q11 Finance dashboards compute "net revenue" with slightly different SQL, producing inconsistent numbers, and each recomputes large joins. Which implementation addresses both?

- A A metric view for the definition plus materialized views for speed.
- B A comment on each dashboard describing the official formula for net revenue.
- C A shared SQL snippet file that every analyst copies into their queries.
- D A Python UDF computing net revenue, copied into each team's notebooks.

Q12 An Iceberg table maintained in Unity Catalog and read by Trino is loaded daily and purged by month after seven years; queries filter on `member_id`. Which layout follows the principles?

A. Partition by `member_id` so Trino can prune directly to each member's files.

B. No partitioning and no compaction, to avoid rewriting files that Trino may be reading.

C. Partition by month, cluster by `member_id` within it, with UC-managed compaction.

D. Partition by day and by `member_id` for the most precise pruning possible.

Domain 9 practice questions — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

Q1 Liquid clustering covers filter and join columns at any cardinality.

Answer D

A Caching does not reduce what is scanned the first time.

B High-cardinality partitioning explodes files.

C Keeps the anti-pattern.

Q2 Low cardinality, large partitions and a natural lifecycle key.

Answer B

A One file per ID.

C Far too many partitions.

D Near-unique values.

Q3 Type 2 with validity intervals is the standard for history and point-in-time joins.

Answer A

B Coarse and duplicative.

C History is overwritten.

D Keeps only one prior value.

Q4 NOT NULL and CHECK reject violating writes; keys are informational.

Answer D

A Not supported/enforced.

B Informational only; RELY is an optimizer hint.

C PK/FK are not enforced.

Q5 Delta maps filters on the source column to the generated column for pruning.

Answer B

A Stats alone do not co-locate data.

C Unrelated.

D Breaks types and does not help.

Q6 RELY tells the optimizer it may trust the unenforced constraint.

Answer B

A Delta has no such index.

C Unrelated.

D Not enforced.

Q7 Views present renamed/subset columns with no copies.

Answer A

B Copies and staleness.

C Duplicated logic.

D Copies data.

Q8 Z-order is a rewrite-based OPTIMIZE option and is incompatible with liquid clustering.

Answer C

A Partitioning is separate.

B Z-order rewrites broadly.

D Skipping works without it.

Q9 Identity allocation conflicts between concurrent transactions.

Answer D

A Unrelated.

B Supported on managed tables.

C Unrelated.

Q10 Pre-aggregation at the query grain plus clustering gives dashboard latency at low cost.

Answer C

A Not a data model.

B Expensive and still slow.

D Same row count, no aggregation.

Q11 Metric view = one governed definition; materialized view = precomputed aggregates.

Answer A

B Documentation is not enforcement.

C Copies drift.

D Not governed; slow.

Q12 Partition to grain/lifecycle, cluster to access pattern, compact to keep file sizes healthy.

Answer C

A High-cardinality partitioning.

B Small files and expensive purges.

D Explodes file counts.

Notes · Domain 9

What clicked, what did not, commands to try, questions to ask a colleague.

Part III

Reference and practice

Use this part in the final two weeks: one-page cheat sheets, the renamed-product glossary, the timed mock exam, and a path for growing into a senior role after the certificate.

Cheat sheets

Glossary

Final mock exam

Beyond the exam: the senior path

Reference

Cheat sheets

Everything you should recognise on sight. Read the night before, not instead of the chapters.

Delta Lake commands

Command	Does	Watch out
`DESCRIBE HISTORY t`	Versions with operation, user, metrics	First stop when data changed
`DESCRIBE DETAIL t`	Location, format, size, files, properties	
`SELECT ... VERSION AS OF n` / `TIMESTAMP AS OF`	Time travel	Only while files exist
`RESTORE TABLE t TO VERSION AS OF n`	New commit equal to version n	History kept
`VACUUM t [RETAIN n HOURS]`	Deletes unreferenced files older than retention (7 days)	Breaks time travel and lagging readers
`OPTIMIZE t [FULL]`	Compaction + clustering (incremental; FULL reclusters all)	`ZORDER BY` is legacy
`REORG TABLE t APPLY (PURGE)`	Rewrites files with deletion vectors	Needed for physical erasure
`ALTER TABLE t CLUSTER BY (...)` / `AUTO` / `NONE`	Change clustering keys	Metadata only until OPTIMIZE
`CONVERT TO DELTA parquet.\` path``	In-place upgrade	No data rewrite
`CREATE TABLE ... SHALLOW CLONE` / `DEEP CLONE`	Metadata copy / full copy	Shallow depends on source files
`table_changes('t', v)`	Change Data Feed	Needs CDF enabled
`ANALYZE TABLE t COMPUTE STATISTICS`	Optimizer statistics	Automatic with Predictive Optimization
`UNDROP TABLE t`	Recover a dropped managed table	Within the recovery window

Streaming quick reference

Topic	Rule
Checkpoint	One per query; offsets + commits + state; keep across restarts
Safe to change	Trigger interval, `maxFilesPerTrigger`, `maxBytesPerTrigger`
Needs new checkpoint	Different source, sink, or stateful operator shape
`availableNow`	Process all available, then stop
Watermark	Required to bound state for aggregations, dedup, stream-stream joins
Output modes	append (Delta default), update (foreachBatch/MERGE), complete (aggregations, unbounded)

Topic	Rule
Delta source changes	Fail by default; `skipChangeCommits`, full refresh or CDF
foreachBatch	At-least-once; idempotent via MERGE or `txnAppId` / `txnVersion`

Declarative Pipelines quick reference

Construct	Use
`CREATE OR REFRESH STREAMING TABLE`	Append-only incremental
`CREATE OR REFRESH MATERIALIZED VIEW`	Correct over any source
`CREATE FLOW ... AS INSERT INTO`	Several sources into one table
`AUTO CDC INTO ... KEYS ... SEQUENCE BY ... STORED AS SCD TYPE 1/2`	CDC
`AUTO CDC FROM SNAPSHOT`	Snapshot sources
`EXPECT` / `DROP ROW` / `FAIL UPDATE`	Warn / drop / stop
`event_log(TABLE(t))`	Errors, expectations, refresh decisions
Full refresh	Reprocess from scratch (clears streaming state)

Configs to recognise

Setting	Meaning
`spark.sql.shuffle.partitions`	Partitions after a shuffle (200 default; AQE adjusts)
`spark.sql.autoBroadcastJoinThreshold`	Max size to broadcast (10 MB default)
`spark.sql.adaptive.enabled` / `.skewJoin.enabled`	AQE and skew handling
`spark.sql.files.maxPartitionBytes`	Input split size (128 MB default)
`delta.deletedFileRetentionDuration`	VACUUM retention (7 days)
`delta.logRetentionDuration`	History retention (30 days)
`delta.enableChangeDataFeed`	CDF on/off
`delta.enableDeletionVectors`	DVs on/off
`delta.universalFormat.enabledFormats = 'iceberg'`	UniForm Iceberg metadata
`cloudFiles.schemaEvolutionMode`	addNewColumns / rescue / failOnNewColumns / none
`cloudFiles.useNotifications` / managed file events	Event-driven discovery

Which system table?

Question	Table
How much did it cost?	`system.billing.usage` × `system.billing.list_prices`

Question	Table
Who accessed or changed it?	`system.access.audit`
Who reads/writes this table?	`system.access.table_lineage`
Which runs failed?	`system.lakeflow.job_run_timeline`, `job_task_run_timeline`
Slow SQL?	`system.query.history`
Node or warehouse behaviour?	`system.compute.node_timeline`, `warehouse_events`
Which tags/grants exist?	`system.information_schema.*`

Last-day checklist

- Read every "Remember" box and every "Exam trap" box.
- Re-answer the questions you got wrong the first time, without looking.
- Sleep. Check ID, webcam, clean desk and WebAssessor login the evening before.
- In the exam: read the last sentence first, find the constraint, eliminate impossible behaviour, flag and move on after 2 minutes.

Reference

Glossary and renamed products

Old names still appear in blogs, colleagues' code and older docs. The exam uses the new ones.

Renamed products

Current name (exam)	Formerly	Notes
Lakeflow Spark Declarative Pipelines (SDP)	Delta Live Tables (DLT)	Python module `pyspark.pipelines` (legacy `dlt`)
AUTO CDC	APPLY CHANGES INTO	`create_auto_cdc_flow` (legacy `apply_changes`)
Declarative Automation Bundles	Databricks Asset Bundles (DABs)	Same `databricks bundle` CLI
Lakeflow Jobs	Databricks Workflows / Jobs	
Lakeflow Connect	(new) managed ingestion connectors	SaaS and database connectors
OpenSharing	Delta Sharing	D2D and D2O (open)
Git folders	Repos	
Data quality monitoring	Lakehouse Monitoring	Anomaly detection + profiling
Standard access mode	Shared access mode	
Dedicated access mode	Single user access mode	
Free Edition	Community Edition	

Terms

Term	Meaning
ABAC	Attribute-based access control: policies that match objects by governed tags
AQE	Adaptive Query Execution: runtime re-planning from shuffle statistics
CDC	Change data capture: a feed of inserts, updates and deletes
CDF	Change Data Feed: Delta's row-level change log for a table
Checkpoint	A stream's stored progress (offsets, commits, state)
Clean Room	Shared environment where parties run approved code on combined data
D2D / D2O	Sharing to a Databricks (UC) recipient / to any client via the open protocol
Deletion vector	File marking deleted rows; merge-on-read DML
Disk (Delta) cache	Local SSD copies of remote data files
Expectation	Data-quality rule in a pipeline (warn, drop, fail)

Term	Meaning
Federation	Querying external databases through UC foreign catalogs
Governed tag	Account-level tag with allowed values and assignment permissions
Liquid clustering	Incremental, changeable data layout (`CLUSTER BY`)
Materialized view	Stored query result maintained by a pipeline
Metric view	Governed definition of dimensions and measures, queried with `MEASURE()`
Photon	Native vectorised execution engine
Predictive Optimization	Automatic OPTIMIZE/VACUUM/ANALYZE for managed tables
SCD 1 / 2	Overwrite current values / keep history with validity intervals
Service principal	Non-human identity for automation
Skew	Uneven key distribution making one task much larger
Spill	Data written to disk when execution memory runs out
Streaming table	Append-only incremental table maintained by a pipeline
UniForm	Delta tables exposing Iceberg metadata for Iceberg readers
VARIANT	Binary semi-structured type for JSON-like data
Volume	Governed folder for non-tabular files in UC
Watermark	Event-time lateness threshold that bounds state

Practice

Final mixed mock exam

Timed, mixed, rebalanced. Treat it like the real thing: 40 minutes, no notes.

Scoring guide: **18–20** book the exam. **15–17** re-read the chapters behind the misses, and retake in a few days. **Below 15** do the labs of your weakest domains hands-on first.

Record your attempts here:

Attempt	Date	Score	Weakest domain	Action
1		/ 20		
2		/ 20		
3		/ 20		

Final mixed mock exam

20 questions across all domains in roughly the exam's proportions. Take it timed: 40 minutes. Score 17 or more before booking.

Answer first, check later. Score: _____ / 20 · Date: _____ · Retake date: _____

M1 A stream with a 10-second trigger normally processes each batch in 3 seconds. At peak, batches take up to 30 seconds, and records must still be processed within 10 seconds. Holding cluster size constant, which change is most likely to meet the requirement?

A. Decrease the trigger interval to 5 seconds so each batch is smaller.

B. Increase the trigger interval to 60 seconds so peaks are absorbed in fewer batches.

C. Tune shuffle partitions, since the trigger cannot change without a new checkpoint.

D. Use `availableNow` and schedule the job every 8 seconds from a job trigger.

M2 Bronze ingests JSON with Auto Loader. Silver must hold one current row per `device_id`, applying updates and deletes ordered by `event_ts`. Which construct?

A. A view with `SELECT DISTINCT` over bronze.

B. A streaming table using `dropDuplicates(["device_id"])`.

C. AUTO CDC into a streaming table with KEYS, SEQUENCE BY, APPLY AS DELETE and SCD TYPE 1.

D. A materialized view with `ROW_NUMBER()` per device over the entire bronze history.

M3 A 1 TB JSON dataset must be written as Parquet files of about 512 MB in one pass, without an extra shuffle. Which setting achieves the target size?

- A `repartition(2048)` before the write
- B `spark.sql.shuffle.partitions = 2048`
- C `coalesce(2048)` before the write
- D `spark.sql.files.maxPartitionBytes = 512MB`

M4 An external Delta table has a nightly OPTIMIZE and weekly VACUUM job that often fails and delays other work. Leadership wants to eliminate this maintenance. What is the recommended change?

- A Convert the table to Parquet so it no longer needs VACUUM or OPTIMIZE.
- B Migrate it to a Unity Catalog managed table with Predictive Optimization.
- C Run `VACUUM ... RETAIN 0 HOURS` so each maintenance run is faster.
- D Give the maintenance job more retries and a dedicated larger cluster.

M5 Queries by `customer_id` on a 5 TB table read 95% of files. The table was created with `CLUSTER BY (event_date)`. What is the lowest-effort fix?

- A Enable Photon on the warehouses used by the analysts.
- B Recreate the table partitioned by `customer_id`.
- C Add a broadcast hint to every query that filters on `customer_id`.
- D `ALTER TABLE t CLUSTER BY (event_date, customer_id)`, then `OPTIMIZE t FULL` once.

M6 A job runs task A, then B and C in parallel. A and B succeed; C fails midway after writing some rows to a Delta table. What is the state afterwards?

- A All commits stay, including C's partial ones: no cross-task rollback.
- B All writes from A, B and C are rolled back because the job run failed.
- C No writes become visible until every task in the run succeeds.
- D A's and B's writes are rolled back, but C's partial commits remain.

M7 Which grants let `bi_users` read all current and future tables in `prod.gold` and nothing else in `prod`?

- A `SELECT` on schema `prod.gold` only
- B `SELECT` on catalog `prod`
- C `USE CATALOG` on `prod`; `USE SCHEMA` and `SELECT` on `prod.gold`
- D `ALL PRIVILEGES` on schema `prod.gold`

M8 After a patient's erasure request, no copy of their identifiers may be recoverable within 48 hours. Which plan is **incomplete**?

A. DELETE, retention of 1 day, REORG PURGE, VACUUM after 24 h, full-refresh dependent streaming tables.
B. DELETE in bronze, silver and gold, purge with REORG and VACUUM, and remove the data from shares.
C. DELETE the rows in each table and rely on the default VACUUM retention to remove old files later.
D. DELETE, REORG PURGE, shorten retention, VACUUM, remove the token mapping, refresh derived tables.

M9 Bronze stores raw JSON of varying shape in a STRING column. The team wants efficient storage and fast field access without a fixed schema. What should they do?

A. Keep the STRING and use `regexp_extract` for each field at query time.
B. Parse into a VARIANT column with `parse_json` and use path access.
C. Explode the JSON into key/value rows with a fixed STRUCT schema per firmware version.
D. Convert to Parquet with a strict schema before ingestion.

M10 A notebook reads a masked column and sees full SSNs. The cluster uses a legacy access mode. What is the explanation?

A. Non-UC compute does not enforce masks or grants; use standard, dedicated or serverless.
B. The mask function must be owned by the data scientist for it to be evaluated.
C. Column masks apply only to SQL warehouses and must be recreated as views for notebooks.
D. Masks require Photon, which is not available on legacy clusters.

M11 Which Auto Loader configuration ingests CSV, infers types, keeps unexpected columns without failing, and records their content for review?

A. `mode = "DROPMALFORMED"` on the CSV reader
B. `schemaEvolutionMode = "none"` with `inferColumnTypes = true`
C. `inferColumnTypes = true` with `schemaEvolutionMode = "rescue"`
D. `schemaEvolutionMode = "failOnNewColumns"` with job retries enabled

M12 A bundle deploy to prod fails validation saying `run_as` must be specified. Why?

A. Bundles cannot deploy to production workspaces, only to development ones.
B. The target was set to development mode, which blocks deployments to prod.
C. Production mode requires an explicit `run_as` identity.
D. The wheel artifact was not built before `bundle deploy` was called.

M13 A join between a 1 TB fact table and a 12 MB dimension shows 900 GB of shuffle write and a SortMergeJoin. Which change removes most of the shuffle?

A. Cache the fact table in memory before the join.
B. Broadcast the dimension (hint, or raise the threshold above 12 MB).
C. Coalesce the fact table to 10 partitions before joining.
D. Join in a Python UDF that looks up the dimension from a dictionary.

M14 Hundreds of PII tables need one centrally managed rule masking any column tagged `pii`, across current and future tables, without altering each table. What should be implemented?

A. An ABAC column-mask policy matching the governed `pii` tag.
B. Dynamic views with masking logic duplicating every base table.
C. A per-table `ALTER COLUMN ... SET MASK` on every existing table.
D. A row filter function attached to each schema.

M15 A pipeline validates `member_id IS NOT NULL`. Missing ids mean a broken upstream extract, and continuing would corrupt gold joins. Which expectation behaviour fits?

A. Remove the constraint and fix the nulls in gold
B. `ON VIOLATION DROP ROW` with a quarantine table for the dropped rows
C. Warn only, and report the violation count in the event log
D. `ON VIOLATION FAIL UPDATE`, stopping the pipeline update

M16 Which Jobs API call re-runs only the failed tasks of run 987 with a corrected parameter?

A. `POST /api/2.2/jobs/runs/cancel` followed by `runs/submit` for the failed task
B. `POST /api/2.2/jobs/runs/repair` with `rerun_all_failed_tasks` and `job_parameters`
C. `POST /api/2.2/jobs/run-now` with the job id and the new parameters
D. `GET /api/2.2/jobs/runs/get-output?run_id=987` with an override flag

M17 An SDP materialized view joins three silver tables that receive updates and deletes. A colleague proposes a streaming table instead, "for lower cost". What is the correct assessment?

A. Correct if `skipChangeCommits` is set, since it applies the upstream changes.
B. Incorrect, because materialized views are the only dataset type that supports joins.
C. Correct: streaming tables are always cheaper and give the same results for joins.
D. Incorrect: streaming tables cannot apply the sources' updates and deletes correctly.

M18 Which practice violates least privilege?

A. Making the ETL service principal the owner of production jobs.
B. Granting ALL PRIVILEGES on `prod` to `account users`.
C. Giving operators CAN MANAGE RUN on production jobs.
D. Granting analysts SELECT on a de-identified view instead of the base table.

M19 An engineer must continuously replicate inserts, updates and deletes from an operational PostgreSQL database with minimal custom code. Which approach fits?

A. A Lakeflow Connect managed PostgreSQL CDC connector pipeline.
B. A nightly JDBC read of each table that overwrites the target.
C. OpenSharing the PostgreSQL tables into the lakehouse.
D. CSV exports to cloud storage ingested with Auto Loader.

M20 A daily job must publish a gold table only when a freshness alert is not triggered, and otherwise notify on-call, inside the job DAG. Which native design?

A. A notebook that polls the alert API in a loop until it returns OK.
B. Two jobs on the same schedule: one publishes, one checks.
C. A job health rule on run duration that cancels the publish task.
D. A SQL alert task followed by an If/else task on its result.

Final mixed mock exam — answer key

Every option is explained. Read the wrong-answer notes too: the exam re-uses the same traps.

M1
Answer A

Smaller, more frequent batches reduce per-batch volume and backlog; trigger changes keep the checkpoint.

B Latency grows.
C The trigger can change freely.
D Job start-up overhead alone exceeds the budget.

M2
Answer C

AUTO CDC SCD Type 1 handles ordering, updates and deletes.

A Not current state.
B Keeps the first row, not the latest, and no deletes.
D Recomputes everything and ignores delete semantics.

M3
Answer D

For narrow pipelines, input split size drives output file size.

A Adds a shuffle.
B Affects only wide operations.
C Cannot increase partitions and is unrelated to size.

M4
Answer B

Managed tables let Databricks run maintenance automatically.

A Loses Delta entirely.
C Dangerous and still manual.
D Keeps the maintenance burden.

M5
Answer D

Clustering keys change as metadata; FULL reclusters existing data.

A Faster scanning, same files read.
B High-cardinality partitioning.
C Hints do not change scans.

M6
Answer A

Delta commits are per table and immediate; jobs are not transactions.

B No distributed transactions.
C Each commit is visible immediately.
D Nothing is rolled back.

M7
Answer C

Schema-level SELECT plus the USE chain covers future tables in that schema only.

A Missing the USE chain.
B Grants every schema.
D Over-grants write rights.

M8
Answer C

Default retention keeps files for 7 days, and DVs keep data in place until rewritten.

A Complete for the tables covered.
B Covers copies and purge.
D Complete.

M9 VARIANT keeps any shape in an efficient binary form with fast path queries.

Answer B

A Slow and fragile.

C Rigid and multiplies rows.

D Loses flexibility.

M10 Governance is enforced only by UC-enabled compute.

Answer A

B Ownership is not the issue.

C Masks work on all UC compute.

D Photon is unrelated.

M11 Rescue mode keeps the schema stable and captures unexpected data in `_rescued_data`.

Answer C

A Discards rows.

B Ignores new columns.

D Fails on new columns.

M12 Production-mode validation enforces run_as.

Answer C

A False.

B Development mode does not require run_as.

D Different error.

M13 Broadcasting the small side means the large side is not shuffled.

Answer B

A Still shuffled.

C Destroys parallelism.

D Slow and unnecessary.

M14 ABAC policies key on governed tags and cover future tables.

Answer A

B Base tables remain exposed.

C Per table; misses new ones.

D Row filters do not mask values, and attach to tables.

M15 Invariant violations that signal systemic breakage should stop the update.

Answer D

A Pushes the problem downstream.

B Hides a systemic upstream failure.

C Bad data flows downstream.

M16 runs/repair re-runs failed tasks and accepts parameter overrides.

Answer B

A Awkward and not a repair.

C Starts a new full run.

D Read-only endpoint.

M17 Keep the MV (incremental on serverless), or build CDF-driven flows deliberately.

Answer D

A It skips them.

B Streaming tables can join; the issue is changing inputs.

C Correctness breaks with changing sources.

M18 Blanket privileges for everyone are the opposite of least privilege.

Answer B

A Recommended.

C Recommended.

D Recommended.

M19 Managed database connectors deliver CDC including deletes.

Answer A

B Full reloads, no continuous CDC.

C Sharing is for Delta/UC data between parties.

D No deletes, extra operations.

M20 The SQL alert task exposes its state for If/else branching.

Answer D

A Custom polling.

B No dependency.

C Watches duration, not data.

Beyond the exam

The senior path

The certificate proves you know the platform. Seniority is what you do with it: design, reliability, cost, and people.

What you will be able to do

- See what separates a senior data engineer from a strong mid-level one
- Follow a six-month plan of free courses and practice after the exam
- Turn your daily work into evidence for promotion

Matching free course

- Apache Spark Developer pathway (all four courses)
- Data Warehousing Practitioner pathway (all four courses)

Study checklist

- [ ] Pick one capability per month
- [ ] Keep a brag document
- [ ] Teach one topic to your team each quarter

What changes at senior level

Area	Mid-level	Senior
Scope	Delivers the pipeline in the ticket	Designs the data flow across teams; says what should not be built
Quality	Adds tests when asked	Defines contracts, expectations and alerts before building
Reliability	Fixes failures	Makes failures cheap: idempotent tasks, repairable runs, runbooks
Cost	Makes it work	Knows what it costs per month and why, and makes it cheaper
Security	Follows the rules	Designs PHI handling so the safe way is the easy way
People	Asks for reviews	Reviews others, mentors, writes the design doc everyone reads

Habits worth building now

1. **Write a one-page design note before any new pipeline:** sources, grain, SLAs, volume, failure modes, PHI classification, cost estimate, owner. Ask a colleague to poke holes in it.
2. **Data contracts with upstream teams:** schema, freshness and quality expectations written down, with alerts that fire when they are broken.
3. **Make everything idempotent:** re-running any task on any day must be safe. This single habit removes most 2 a.m. incidents.
4. **Own a cost dashboard** from system tables for your team's jobs; review it monthly.
5. **Blameless incident notes** that end with a new check, not a promise to be careful.
6. **Keep a brag document:** each month, write what you shipped, what it saved or enabled, and who it helped. Promotions are decided on evidence.

A six-month plan after the exam (all free)

Month	Focus	Free resource	Practice at work
1	Spark internals	Introduction to Apache Spark; Developing Applications with Apache Spark	Profile your slowest job; write up the fix
2	Streaming depth	Stream Processing and Analysis with Apache Spark	Convert one batch job to incremental with availableNow or a pipeline
3	Performance and monitoring	Monitoring and Optimizing Apache Spark Workloads on Databricks	Build the team's cost and failure dashboard from system tables
4	Modelling	Data Modeling Strategies	Document the grain of every gold table you own
5	Semantic layer	Building Semantic Models with UC Metric Views	Replace one inconsistent KPI with a metric view
6	Architecture	Architecting Data Warehouses for Large-Scale Deployments; SQL Programming and Procedural Logic	Write a design doc for your next large project and present it

For a healthcare data engineer specifically

Become the person who understands the data, not only the platform: claim lifecycles (submitted, adjudicated, paid, adjusted, reversed), member eligibility spans, provider identifiers (NPI), code systems (ICD-10, CPT, HCPCS), and why the same number differs between finance and clinical reports. That domain knowledge, combined with platform depth, is rare and valued.

Notes · My senior goals and evidence

What clicked, what did not, commands to try, questions to ask a colleague.

Notes

Extra notes pages

Commands to try, questions for colleagues, things to look up in the docs.

More notes

What clicked, what did not, commands to try, questions to ask a colleague.

More notes

What clicked, what did not, commands to try, questions to ask a colleague.